Contextual Equivalence for Simply Typed Lambda Calculi



Jan Gałek

St Catherine's College

University of Oxford

A dissertation submitted in partial fulfilment of the requirements for the degree of

MSc in Mathematics and Foundations of Computer Science

Trinity Term 2026

Acknowledgements

The list of people who made writing this work possible is very long, too long for one page. I cannot express the profound gratitude I feel, but at least I will try to name some of you.

First, I want to thank my academic mentors. My supervisor introduced me to this fascinating topic and provided me with his invaluable guidance and patience, helping me turn my general interests into a concrete, compelling project. Benedict and Walter showed me the charm of foundations and gave me the confidence to pursue an academic career in this direction.

Second, I am incredibly lucky to have my friends, who made it all matter. My beloved Hania, who always remained close, even when far away. My tribe from Chata, to which I can always come back. Donek, who gave me shelter in the wilderness. Irena, who patiently kept reminding me what is important. Marcel and Łucja, my brother and sister in arms.

Finally, my family, which made this whole venture possible. My father, who was the first to show me the world. My stepfather, who gave me the strength and courage to explore it. My aunt, who took care of me and made Oxford within reach. And my mother, who was always there for me with her unconditional love.

Abstract

Contextual equivalence identifies programs that cannot be distinguished by any program context, but its definition quantifies over an infinite family of contexts and therefore gives no immediate decision procedure. This dissertation studies contextual equivalence in simply typed λ-calculi using complementary syntactic, denotational, and type-theoretic methods.

For calculi with a finite ground signature, we first review Padovani's finite representative approach, which yields decidability of contextual equivalence. We then develop a new denotational construction. Starting from Laird's fully abstract bistable model for a binary ground signature, we show that contextual equivalence over a finite signature with at least two constants is determined by its surjective binary recolourings. Interpreting a term simultaneously in the corresponding binary models yields a finite, effectively presentable product model which is fully abstract, and hence gives a semantic decision procedure for contextual equivalence.

Our second main contribution is a general retraction-based framework for higher-order contextual equivalence. We introduce finite-component retractions and retraction bases, and prove transfer principles showing how decidability, universality, and full abstraction at arbitrary types can be reduced to corresponding properties at simpler basis types. The framework isolates and generalises retraction techniques appearing in earlier work, separating a uniform structural reduction from the language-specific construction of compression retractions. We apply it to finite ground signatures, unary PCF, unary PCF with parallel composition, and Boolean PCF, where Loader's undecidability theorem yields an obstruction to effective low-order compression.

Introduction

The question of when two programs should be regarded as equivalent is fundamental to programming language theory. A standard answer is given by *contextual equivalence*: two programs are equivalent when no surrounding program context can distinguish them by their observable behaviour. The idea is captured by the following informal definition from [12]:

> Two phrases of a programming language are contextually equivalent if any occurrences of the first phrase in a complete program can be replaced by the second phrase without affecting the observable results of executing the program.
>
> It is really a family of notions, parameterised by the particular choices one takes for what constitutes a 'complete program' in the language and what are the 'observable results' of executing such programs.

Contextual equivalence is conceptually natural, but its definition presents an immediate difficulty: it quantifies over an infinite and highly structured collection of program contexts. Thus, even for a simple language, it is not clear how contextual equivalence can be characterised mathematically or decided effectively. A central theme of this dissertation is that, for sufficiently restricted simply typed languages, this apparently infinitary notion can nevertheless be captured by finite or structurally controlled data.

We study this phenomenon in the setting of simply typed λ-calculi. The simply typed λ-calculus is sufficiently expressive to exhibit genuinely higher-order behaviour, while remaining simple enough for syntactic, semantic, and type-theoretic techniques to be compared in a common setting. Two problems will be of particular importance throughout the dissertation:

- *Decidability:* given two terms of the same type, can we decide whether they are contextually equivalent?
- *Full abstraction:* given a denotational model of the language, does equality of denotations coincide exactly with contextual equivalence?

These questions are closely related. A fully abstract model replaces quantification over program contexts by equality of mathematical objects. If those objects are effectively representable and their equality is decidable, the model may itself yield a decision procedure for contextual equivalence. Conversely, syntactic descriptions of contextual equivalence constrain what a fully

abstract semantic model can look like. A further perspective is provided by *definable retractions*, which allow questions at complicated higher-order types to be reduced to corresponding questions at simpler types. Accordingly, the dissertation develops three complementary approaches to contextual equivalence. The first is *syntactic*: one seeks finite families of terms representing all observable behaviours at a given type. The second is *semantic*: one interprets terms in mathematical structures whose equality captures precisely the distinctions observable by program contexts. The third is *type-theoretic*: one uses definable retractions to compress the observable information contained at arbitrary higher-order types into simpler families of types. Although these techniques originate in rather different parts of the literature, one aim of the dissertation is to show that they express closely related ways of controlling the apparently infinitary nature of contextual equivalence.

Contributions The dissertation has both expository and original components. On the expository side, it brings together a collection of results concerning contextual equivalence in simply typed λ-calculi which are usually presented using rather different terminology and techniques. In particular, we compare syntactic characterisations of the minimal model, denotational accounts based on sequentiality, and retraction-based reductions between types. Throughout, we indicate where the presentation follows the existing literature and where arguments are reformulated or extended.

The main original contributions are the following:

- We extend the bistable model of Laird [10], originally formulated for a two-element ground signature, to arbitrary finite ground signatures. The resulting interpretation is fully abstract. Since the interpretation of every type is finite and effectively presentable, this yields a semantic proof of decidability of contextual equivalence for arbitrary finite ground signatures. In this way we obtain a denotational route to the decidability problem raised by Abramsky and Lenisa in [1].

- We isolate and generalise the retraction-based methodology underlying several arguments of Laird [10, 9, 11]. We introduce a framework of finite-component retractions and *retraction bases*, in which arbitrary types can be compressed into finite collections of simpler types. This separates a uniform structural reduction from the language-specific problem of constructing suitable compression retractions. The framework gives transfer principles for decidability, universality, and full abstraction, and applies uniformly to several simply typed languages considered in the dissertation.

Alongside these two main contributions, the dissertation contains several smaller original arguments and expository developments, which are explicitly indicated in the relevant sections.

In particular, several arguments which are standard, implicit, or compressed in the literature are supplied in full in order to make the development self-contained. These include the compatibility argument relating applicative and contextual equivalence, the detailed reconstruction of Padovani's finite-representative argument, and the elimination of finite products used in the retraction analysis.

Prerequisites and structure We assume basic familiarity with the λ-calculus, category theory, and order theory.

Chapter 1 establishes the formal setting. We define contextual equivalence for simply typed λ-calculi with finite ground signatures and compare several equivalent formulations, including observational and applicative characterisations. We also introduce the maximal theory and the corresponding minimal model.

Chapter 2 develops the syntactic perspective. We present Padovani's cellularisation method [16], which replaces arbitrary terms by a restricted class of cellular representatives. The central consequence is that, at every type over a finite ground signature, only finitely many observable behaviours need be represented. This yields a decision procedure for contextual equivalence and provides the first answer to the apparent infinitary nature of its definition.

Chapter 3 turns to denotational semantics and asks whether the same observable distinctions can instead be represented by semantic objects. After recalling the cartesian closed interpretation of STLC and formulating full abstraction, we review Laird's bistable model for a binary ground signature [10]. We prove its universality by recasting the decomposition method of Schmidt-Schauß used in [13]. We then extend the model to arbitrary non-singleton finite ground signatures by means of a product construction. The resulting model is fully abstract and gives a semantic decision procedure for contextual equivalence, providing a denotational counterpart to the syntactic result of Chapter 2.

Chapter 4 broadens the setting from finite ground signatures to simply typed languages with higher-type constants and additional operational structure. We study unary PCF and its stable semantics following Laird [9], and contrast it with Boolean PCF. These examples exhibit a sharp boundary: unary PCF retains strong sequentiality and decidability properties, whereas Loader's undecidability theorem for Boolean PCF shows that analogous effective descriptions cannot persist unrestrictedly.

Finally, Chapter 5 develops the type-theoretic perspective suggested by the preceding examples. We extract the retraction-based techniques used by Laird [10, 9, 11] and formulate them as a general theory of finite-component retractions and retraction bases. The framework reduces questions at arbitrary higher-order types to corresponding questions on a simpler basis of types, yielding general transfer principles for decidability, universality, and full abstraction.

We instantiate the framework for finite-ground-signature STLC and unary PCF with and without parallel composition, and use Boolean PCF to identify an obstruction to uniform effective compression. In this way the final chapter brings together the syntactic and semantic themes of the dissertation in a common structural account.

Contents

1 Preliminaries **11**

1.1 Simply Typed λ-Calculus . . . 11

1.2 Ground and higher-order observations . . . 12

1.3 Ground contextual equivalence . . . 13

1.4 Many faces of contextual equivalence . . . 14

1.5 Maximal theory and the minimal model . . . 15

2 Syntactic methods **17**

2.1 Syntactic forms . . . 18

2.2 Finite representatives . . . 20

2.3 Deciding contextual equivalence . . . 24

2.4 Examples . . . 25

3 Semantics **27**

3.1 Cartesian closed semantics of STLC . . . 28

3.1.1 CCC interpretation of STLC . . . 28

3.2 Denotational semantics and full abstraction . . . 29

3.3 Syntactic constraints on fully abstract semantics . . . 31

3.4 The set-theoretic model . . . 32

3.5 Laird's two-point model . . . 33

3.5.1 Bistable biorders and interpretation . . . 33

3.5.2 The structure of binary semantic elements . . . 35

3.5.3 Semantic adaptation of Loader's decomposition . . . 37

3.5.4 Universality and full abstraction . . . 40

3.6 A fully abstract semantics for finite ground signatures . . . 42

3.6.1 Obstructions to a direct extension . . . 43

3.6.2 Reduction to binary observations . . . 43

3.6.3 The product interpretation . . . 45

3.7 Semantic decidability of contextual equivalence . . . 47

4 Beyond ground-only signatures **49**

4.1 Typed signatures and equational theories 49

4.2 Unary PCF and its models 50

4.2.1 The language and observational equivalence 50

4.2.2 The Scott model and parallel behaviour 52

4.2.3 Standard order-extensional models 52

4.2.4 The stable-biorder model 53

4.2.5 Universality of the stable model 55

4.2.6 Failure of full abstraction of the Scott model 56

4.2.7 Concrete structure at first-order types 57

4.3 A hierarchy of two-point calculi and models 58

4.4 Boolean PCF 58

4.4.1 The language and observational equivalence 59

4.4.2 Failure of boundedness 60

4.4.3 Undecidability and effective semantics 60

4.4.4 The stable-biorder interpretation 61

5 Retractions **63**

5.1 Definable retractions 64

5.2 Finite-component retractions 65

5.2.1 Products as a metatheoretical device 65

5.3 General retraction-based framework 66

5.3.1 Retraction bases and compression 67

5.3.2 Transfer principles 68

5.3.3 Effective retraction bases 68

5.4 Recovering universality of the bistable model 70

5.5 Finite ground signatures 70

5.5.1 First-order compression 70

5.5.2 An effective first-order retraction basis 71

5.5.3 Decidability at the basis types 73

5.5.4 Decidability of the maximal theory 73

5.6 Unary PCF 74

5.6.1 An effective retraction basis 74

5.6.2 Decidability 74

5.6.3 Universality 75

5.7 Unary PCF with parallel composition 75

5.7.1 First-order compression . . . 75
5.7.2 An effective ground-type retraction basis . . . 76
5.7.3 Decidability . . . 77
5.8 Boolean PCF and obstruction to effective compression . . . 77
5.8.1 Undecidability obstruction . . . 77
5.8.2 Finite separation without reconstruction . . . 78

6 Conclusions **80**
6.1 Future work . . . 81

A Auxiliary results on observational equivalence **85**
A.1 Higher-order and ground observations . . . 85
A.2 Compatibility of observational equivalence . . . 86
A.3 Observational and contextual equivalence . . . 88

B Obstructions to direct extensions of the bistable model **91**
B.1 Total coherence . . . 91
B.2 Discrete coherence . . . 92

C Products and canonical type isomorphisms **94**
C.1 Elimination of finite products . . . 94
C.2 Effective witnesses for canonical type isomorphisms . . . 95

Chapter 1

Preliminaries

1.1 Simply Typed λ-Calculus

We work in simply typed lambda calculus with a single ground type o. Types are generated by

$$A, B ::= o \mid A \to B.$$

We write $[A_1, \ldots, A_n]$ for $A_1 \to \cdots \to A_n \to o$, where the case $n = 0$ denotes the ground type o.

Let $\mathcal{C}$ be a non-empty finite set of ground constants, each of type o. We refer to $\mathcal{C}$ as a *finite ground signature*, and to the language as $\Lambda_{\mathcal{C}}$. Given a countable set of variables $\mathcal{V}$, terms are generated by

$$M, N ::= x \mid c \mid \lambda x^A.\, M \mid MN, \qquad x \in \mathcal{V},\ c \in \mathcal{C}.$$

We identify terms modulo α-equivalence. A typing context Γ is a finite assignment of types to variables. Typing judgments have the form $\Gamma \vdash M : A$ and are derived by

$$\frac{x : A \in \Gamma}{\Gamma \vdash x : A}, \qquad \frac{c \in \mathcal{C}}{\Gamma \vdash c : o}, \qquad \frac{\Gamma, x : A \vdash M : B}{\Gamma \vdash \lambda x^A.\, M : A \to B}, \qquad \frac{\Gamma \vdash M : A \to B \qquad \Gamma \vdash N : A}{\Gamma \vdash MN : B}.$$

Reduction is generated by β- and η-reduction,

$$(\lambda x^A.\, M)N \longrightarrow_\beta M[N/x], \qquad \lambda x^A.\, Mx \longrightarrow_\eta M \quad \text{if } x \notin \mathrm{FV}(M),$$

with both relations closed under term formation. We write

$$\longrightarrow_{\beta\eta} := \longrightarrow_\beta \cup \longrightarrow_\eta,$$

and write $=_{\beta\eta}$ for the reflexive, symmetric, and transitive closure of $\longrightarrow_{\beta\eta}$. Finally, we write

$$M \Downarrow N \text{ if } M \longrightarrow^*_{\beta\eta} N \text{ and } N \text{ is } \beta\eta\text{-normal.}$$

We use Church-style typing, so that the type of a bound variable is specified at its abstraction. We nevertheless make typing contexts explicit, since they will be useful when reasoning about open terms and, later, when dealing with type-theoretic technicalities. Thus $\Gamma \vdash M : A$ records the types of the free variables of M, while for closed terms we write simply $\vdash M : A$, or just $M : A$ when there is no risk of confusion. We assume standard results such as confluence or strong normalisation. For the general theory of simply typed λ-calculus, the reader is referred to [2].

One-hole contexts will be used to formalise the placement of a term within a larger program.

Definition 1.1.1 (One-hole contexts)**.** One-hole contexts are generated by the grammar

$$C[-] ::= [-] \mid \lambda x^A.C[-] \mid C[-]\ M \mid M\ C[-].$$

We write $\Gamma \vdash C[-] : A \Rightarrow B$ when, for a fresh variable z, we have $\Gamma, z : A \vdash C[z] : B$.

1.2 Ground and higher-order observations

Before defining contextual equivalence formally, it is useful to make explicit what we take to be an observable result. For comparison, first consider the corresponding constant-free calculus. In the pure simply typed λ-calculus, with no constants at the ground type o, there are no closed terms of type o. Hence taking complete programs to be closed terms of ground type would yield no non-trivial observations.

One possibility is instead to designate an inhabited higher type as the type of observable results. The simplest example is the Church Boolean type $\mathbb{B} := o \to o \to o$, whose two closed inhabitants, up to $\beta\eta$-equivalence, are $\mathsf{tt} := \lambda x^o.\lambda y^o.x$ and $\mathsf{ff} := \lambda x^o.\lambda y^o.y$. Thus, in the pure calculus, terms may be distinguished by placing them in contexts of result type $\mathbb{B}$ and observing which Boolean is produced.

Alternatively, one may enrich the language with primitive constants at ground type. After adjoining two distinct constants $a, b : o$, terms may instead be distinguished by ground contexts according to whether they return a or b. The following lemma shows that these two presentations induce the same equivalence over constant-free closed terms.

Let M_1, M_2 be closed terms of the same type θ in the constant-free calculus. Write $M_1 \equiv_{\mathbb{B}} M_2$ if, for every closed context $\vdash C[-] : \theta \Rightarrow \mathbb{B}$, we have $C[M_1] \Downarrow \mathsf{tt}$ iff $C[M_2] \Downarrow \mathsf{tt}$. Since tt and ff are the only closed normal forms of type $\mathbb{B}$, observing one of them is sufficient.

Now extend the calculus by two fresh and distinct constants $a, b : o$. Write $M_1 \equiv_{a,b} M_2$ if, for every closed ground context $\vdash_{\{a,b\}} D[-] : \theta \Rightarrow o$, we have $D[M_1] \Downarrow a$ iff $D[M_2] \Downarrow a$.

Lemma 1.2.1. *For closed terms $M_1, M_2 : \theta$ of the pure simply typed λ-calculus, $M_1 \equiv_{\mathbb{B}} M_2$ iff $M_1 \equiv_{a,b} M_2$.*

The proof is given in Appendix A.1. The lemma justifies taking primitive ground constants as the basic observations in what follows. We build the observable outcomes directly into the ground type, while higher-order terms are observed through the ground behaviour that contexts can elicit from them.

Remark 1.2.2. The same correspondence extends to any finite number of observations. Given distinct constants $a_1, \ldots, a_n : o$, one may instead work in the pure calculus with the observation type $\mathbb{B}_n := \underbrace{o \to \cdots \to o}_{n \text{ arguments}} \to o$, whose closed normal forms are precisely the projections $\lambda x_1^o \ldots x_n^o . x_i$, for $1 \leq i \leq n$. Replacing each a_i by x_i, and conversely applying such an observation to $a_1, \ldots, a_n$, yields the analogous correspondence.

Finite ground signatures may therefore be viewed as providing primitive names for a finite collection of observations that could alternatively be represented at higher type. We adopt this presentation because it separates the primitive notion of observable outcome from the higher-order structure of programs.

1.3 Ground contextual equivalence

A *closed ground context* from A to o is a one-hole context satisfying $\vdash C[-] : A \Rightarrow o$.

Definition 1.3.1 (Ground contextual equivalence for closed terms)**.** If M and N are closed terms of type A, then $M \simeq_{\mathrm{ctx}} N$ means that for every closed ground context $\vdash C[-] : A \Rightarrow \circ$, we have $C[M] =_{\beta\eta} C[N]$.

This specifies the informal idea that a complete program must return an observable value. In finite signature calculi, the only observable values are the ground constants. Function-typed terms are compared extensionally, by applying them to closed arguments and observing the resulting ground behaviour. It is also possible to extend this definition to open terms.

Definition 1.3.2 (Closing substitutions)**.** Let $\Gamma = x_1 : A_1, \ldots, x_n : A_n$ be a typing context. A *closing substitution* for Γ is a substitution $\sigma = [M_1/x_1, \ldots, M_n/x_n]$ such that each M_i is closed and has the appropriate type, i.e. $\vdash M_i : A_i$ for every $1 \leq i \leq n$. We write $\sigma \in \mathsf{ClSub}(\Gamma)$ for the set of all closing substitutions for Γ.

If $\Gamma \vdash M : A$ and $\sigma = [M_1/x_1, \ldots, M_n/x_n] \in \mathsf{ClSub}(\Gamma)$, then $M\sigma$ denotes the capture-avoiding simultaneous substitution $M[M_1/x_1, \ldots, M_n/x_n]$. In particular, $\vdash M\sigma : A$.

Definition 1.3.3 (Ground contextual equivalence for open terms)**.** Let $\Gamma \vdash M : A,\ \Gamma \vdash N : A$. We say that M and N are contextually equivalent, written $\Gamma \vdash M \simeq_{\mathrm{ctx}} N$, if for every closing substitution $\sigma \in \mathsf{ClSub}(\Gamma)$ and every closed ground context $\vdash C[-] : A \Rightarrow o$, $C[M\sigma] =_{\beta\eta} C[N\sigma]$.

Since open terms are compared only after applying a closing substitution, the subsequent insertion into a closed ground context involves closed terms. Hence variable capture does not arise.

1.4 Many faces of contextual equivalence

The notion of program equivalence admits several equivalent characterisations. In this section we show that the applicatively defined observational equivalence coincides, on closed terms, with the ground contextual equivalence introduced above. We then derive the two characterisations used later in the dissertation: testing equivalence by closed applications, and testing open terms by closing substitutions.

Definition 1.4.1. Two closed terms t, t' of the same type are *observationally equivalent*, written $t \equiv t'$, if and only if:

- if $t, t' : o$, then $t =_\beta t'$;
- if $t, t' : A \to B$, then for every closed $u : A$, $t\,u \equiv t'\,u$.

We will use the following standard compatibility property of observational equivalence. Its proof is recalled in Appendix A.2.

Proposition 1.4.2 (Context Lemma)**.** *If $\vdash u, u' : A$ and $u \equiv u'$, then for every closed $t : A \to B$ we have $t\,u \equiv t\,u'$.*

The equivalence of the two notions now follows from a compatibility argument. Since the proof is somewhat technical and would interrupt the preliminary discussion, we defer it to Appendix A.3.

Theorem 1.4.3. *For closed terms, observational equivalence and ground contextual equivalence coincide.*

The preceding theorem allows us to use the applicative definition of observational equivalence as a characterisation of contextual equivalence.

Corollary 1.4.4 (Applicative characterisation)**.** *Let $M, N : A \to B$ be closed. Then $M \simeq_{\mathrm{ctx}} N$ if and only if, for every closed $P : A$, we have $M\,P \simeq_{\mathrm{ctx}} N\,P$.*

Proof. By the preceding theorem, contextual equivalence coincides with observational equivalence on closed terms. The result therefore follows directly from the definition of observational equivalence at function type. □

The same result gives a convenient characterisation for open terms.

Corollary 1.4.5 (Closing-substitution characterisation)**.** *Let* $\Gamma \vdash M, N : A$*. Then* $\Gamma \vdash M \simeq_{\mathrm{ctx}} N : A$ *if and only if* $M\sigma \equiv N\sigma$ *for every* $\sigma \in \mathsf{ClSub}(\Gamma)$*.*

Proof. By the definition of contextual equivalence for open terms, $\Gamma \vdash M \simeq_{\mathrm{ctx}} N : A$ iff $M\sigma \simeq_{\mathrm{ctx}} N\sigma$ for every closing substitution $\sigma \in \mathsf{ClSub}(\Gamma)$. Each $M\sigma$ and $N\sigma$ is closed, so the preceding theorem gives $M\sigma \simeq_{\mathrm{ctx}} N\sigma$ iff $M\sigma \equiv N\sigma$. □

The preceding results allow us to move freely between the formulations of program equivalence used later in the dissertation. On closed terms, the observational equivalence $\equiv$ used later in Padovani's syntactic arguments coincides with contextual equivalence, while the applicative characterisation reduces equivalence at function type to equivalence after closed applications. When the signature needs to be made explicit, we write $\equiv_{\mathcal{C}}$ for observational equivalence in $\Lambda_{\mathcal{C}}$. For open terms, contextual equivalence can be checked after all closing substitutions. These characterisations will be used repeatedly in the syntactic decidability argument of Chapter 2.

1.5 Maximal theory and the minimal model

We conclude the preliminaries by recalling the notions of maximal theory and minimal model.

Definition 1.5.1 (λ-theory)**.** A *λ-theory* T for $\Lambda_{\mathcal{C}}$ is an equivalence relation $=_T$ on well-typed terms, relating only terms of the same type in the same typing context, such that it contains $\beta\eta$-equivalence and is compatible with the term constructors.

The theory T is *consistent* if it does not identify distinct ground constants.

Definition 1.5.2 (Maximal theory)**.** A consistent λ-theory T is *maximal* if it is the greatest, with respect to inclusion, among consistent λ-theories containing $\beta\eta$-equivalence. Thus, if S is any other consistent λ-theory containing $\beta\eta$-equivalence, then

$$\Gamma \vdash M =_S N : A \quad \Longrightarrow \quad \Gamma \vdash M =_T N : A.$$

Lemma 1.5.3. *Ground contextual equivalence is a consistent λ-theory containing $\beta\eta$-equivalence.*

Proof. It is immediate from the definition that contextual equivalence is an equivalence relation and contains $\beta\eta$-equivalence.

It is compatible with the term constructors because contexts may be composed: if replacing a subterm by a contextually equivalent one could change the result of some closed ground context, composing the two contexts would distinguish the original terms.

It is consistent because distinct ground constants are distinguished by the identity context. □

Proposition 1.5.4. *Ground contextual equivalence is the greatest consistent λ-theory containing $\beta\eta$-equivalence.*

Proof. Let S be any consistent λ-theory containing $\beta\eta$-equivalence, and suppose $\Gamma \vdash M =_S N : A$. Assume for contradiction that $M \not\simeq_{\mathrm{ctx}} N$. Then there exist $\sigma \in \mathsf{ClSub}(\Gamma)$ and a closed ground context $\vdash C[-] : A \Rightarrow o$ such that $C[M\sigma] \neq_{\beta\eta} C[N\sigma]$. Since these are closed ground terms, they normalise to distinct ground constants $a \neq b$.

Compatibility of S, together with β-conversion, gives $M\sigma =_S N\sigma$ and hence $C[M\sigma] =_S C[N\sigma]$. Since S contains $\beta\eta$-equivalence, we also have $C[M\sigma] =_S a$ and $C[N\sigma] =_S b$. Thus $a =_S b$, contradicting consistency. Therefore $\Gamma \vdash M \simeq_{\mathrm{ctx}} N : A$. □

Definition 1.5.5 (Minimal model)**.** The *minimal model* of $\Lambda_{\mathcal{C}}$ is the quotient of well-typed terms by contextual equivalence. In particular, at each type A its closed elements are

$$\mathcal{M}_{\mathcal{C}}(A) = \big\{ [M]_{\simeq_{\mathrm{ctx}}} \mid \vdash M : A \big\},$$

where $[M]_{\simeq_{\mathrm{ctx}}} = \{N \mid \vdash N : A \text{ and } N \simeq_{\mathrm{ctx}} M\}$. More generally, open terms may be quotiented in the same way, so that a judgement $\Gamma \vdash M : A$ is represented by its contextual-equivalence class.

Remark 1.5.6. The terms *maximal theory* and *minimal model* describe the same quotient from two complementary viewpoints. Contextual equivalence is maximal because it identifies as many terms as possible without collapsing distinct observations. The corresponding quotient model is minimal because it retains only distinctions that are observable by program contexts.

Chapter 2

Syntactic methods

Contextual equivalence is defined by quantifying over all closing program contexts of observable result type. Even in the simply typed setting, there are infinitely many such contexts, and their syntax can be arbitrarily complicated. Taken literally, the definition therefore gives no immediate reason to expect contextual equivalence to be decidable.

The first approach considered in this dissertation replaces this infinite family of observations by finitely many *syntactic representatives*. For the simply typed λ-calculus with a finite ground signature, Padovani showed that every closed term is observationally equivalent to one belonging to a restricted syntactic class, and that at each fixed type only finitely many observable behaviours need be represented [16]. Combined with the applicative characterisation of contextual equivalence from the preceding chapter, these representatives yield a finite decision procedure.

The argument has two stages. First, arbitrary terms are transformed, up to observational equivalence, into *cellular terms*. Their syntax decomposes computation into local experiments, called *cells*, each of which can either produce a ground constant or select one of finitely many continuations. The Shrinking Lemma identifies a form of redundancy among nested occurrences of the same cell.

Second, this restricted syntax is used to construct, recursively on the order of types, a finite set $\mathcal{R}(A, \mathcal{C})$ containing a representative of every observational-equivalence class at type A. Contextual equivalence then reduces to finitely many applications to representative arguments and, ultimately, to equality of ground normal forms.

The presentation is based on Padovani's argument [16], with two modifications. We work modulo $\beta\eta$-equivalence rather than assuming terms to be η-long throughout, and we make explicit how the closed-term decision procedure extends to contextual equivalence of open terms. The proof of the finite representative theorem follows Padovani's construction closely, but his presentation of this step is very compressed. We therefore spell out several points left implicit

there, in particular the bound on the number of observable holes, the well-founded construction of the finite representative set, and the final use of the Shrinking Lemma. These are expository elaborations of Padovani's construction.

2.1 Syntactic forms

Padovani works throughout with η-long terms. In our presentation, terms are not assumed to be η-long; instead we work modulo $\beta\eta$-equivalence. This causes no loss of generality. For every closed well-typed term t, let t^ℓ be the η-long expansion of its β-normal form. Then $t =_{\beta\eta} t^\ell$, hence $t \simeq_{\mathrm{ctx}} t^\ell$. Since ground contextual equivalence coincides with Padovani's observational equivalence on closed terms, we also have $t \equiv t^\ell$. We may therefore apply Padovani's syntactic construction to t^ℓ and transfer the resulting observational-equivalence statements back to t.

In what follows, whenever Padovani's syntactic arguments are invoked, terms are tacitly replaced by η-long representatives. We recall the main definitions and results needed for the decidability argument, but omit their technical proofs. These can be found in [16]. Padovani's proof of the Cellularisation Theorem also uses the Stretching Lemma and an intermediate class of semi-cellular terms. Since we do not reproduce that proof here, we omit these auxiliary notions.

Definition 2.1.1 (Cell)**.** A *cell* is a well-typed multiple-hole context of the form

$$\Sigma[-]_1 \cdots [-]_K = \big(y\,(C_1[-]_1 \cdots [-]_K) \cdots (C_p[-]_1 \cdots [-]_K)\big),$$

where $y : A_1 \to \cdots \to A_p \to o$, each hole $[-]_i$ has type o, and each $C_j[-]_1 \cdots [-]_K$ is a context of type A_j with no free variables other than the holes.

Remark 2.1.2. A cell may be viewed as a local experiment performed by its head variable y. Replace its holes by fresh ground variables $x_1, \ldots, x_K$, and substitute a closed term for y. The resulting term has ground type and no free variables other than the x_i. Since the language contains no constants of functional type, its β-normal form is either a ground constant or one of the variables x_i. Thus a closed instance of a cell either produces a constant or selects one of its holes.

Definition 2.1.3 (Cellular term)**.** A term t is *cellular* if and only if it has one of the following forms:

1. $\lambda y_1 \cdots y_n.\, a$, where a is a ground constant;
2. $\lambda y_1 \cdots y_n.\, \Sigma[w_1]_1 \cdots [w_K]_K$, where Σ is a cell whose head variable is among $y_1, \ldots, y_n$, and each $\lambda y_1 \cdots y_n.w_j$ is cellular.

Remark 2.1.4. Informally, a cellular term is built recursively from local experiments: a cell interrogates one of the outer variables and, depending on the result of that interaction, either terminates with a constant or continues with one of its cellular subterms.

Lemma 2.1.5 (Cell evaluation)**.** *Let $\Sigma[-]_1 \cdots [-]_K$ be a cell whose head variable is among $y_1, \ldots, y_n$. Replace the holes by fresh variables $x_1, \ldots, x_K : o$, and substitute closed terms of the appropriate types for $y_1, \ldots, y_n$. Then the β-normal form of the resulting term is either one of the variables x_j or a ground constant.*

The Cell Evaluation Lemma gives the basic operational picture of a cell: once its head variable has been closed, it can observe its continuations only by selecting one of them. The following result shows that this selection behaviour makes certain nested occurrences redundant.

Proposition 2.1.6 (Shrinking Lemma)**.** *Let Σ be a cell whose head variable is among $y_1, \ldots, y_n$, and let $M[-]$ and $N[-]$ be well-typed contexts such that the following terms are well-typed. Then*

$$\lambda y_1 \cdots y_n.\, M\big[\Sigma[w_1] \cdots [w_{k-1}]\big[N[\Sigma[v_1] \cdots [v_K]]\big][w_{k+1}] \cdots [w_K]\big]$$

is observationally equivalent to

$$\lambda y_1 \cdots y_n.\, M\big[\Sigma[w_1] \cdots [w_{k-1}]\big[N[v_k]\big][w_{k+1}] \cdots [w_K]\big].$$

The reason is that, after closing the head variable of Σ, the outer cell either returns a constant or selects one of its holes. If it selects the k-th hole, then only the k-th component of the inner occurrence can contribute to the final observation; all its other components are invisible.

Example 2.1.7 (A shrinking phenomenon). Let $\mathcal{C} = \{a, b\}$ and consider $P = \lambda g^{o\to o\to o}.g\,a\,(g\,b\,a)$ and $Q = \lambda g^{o\to o\to o}.g\,a\,a$, both of type $(o \to o \to o) \to o$.

Every closed $\beta\eta$-normal term of type $o \to o \to o$ is one of

$$\lambda x^o y^o.a, \qquad \lambda x^o y^o.b, \qquad \lambda x^o y^o.x, \qquad \lambda x^o y^o.y.$$

Testing P and Q on these four possible behaviours gives

g	$P\,g$	$Q\,g$
$\lambda x^o y^o.a$	a	a
$\lambda x^o y^o.b$	b	b
$\lambda x^o y^o.x$	a	a
$\lambda x^o y^o.y$	a	a

and hence $P \equiv Q$.

This is a concrete instance of the Shrinking Lemma. The outer occurrence of g forms the cell $\Sigma[-]_1[-]_2 = g\,[-]_1\,[-]_2$, whose second hole is filled in P by the inner occurrence $g\,b\,a$. If the outer cell selects this second hole, then only the second component of the inner cell can be observed, namely a. The inner occurrence may therefore be replaced by a, yielding Q.

The Shrinking Lemma will later be used to eliminate cyclic dependencies when constructing a finite grammar of cellular representatives. The following theorem first ensures that restricting attention to cellular terms loses no observable behaviour.

Theorem 2.1.8 (Padovani's Cellularisation Theorem)**.** *Every closed term t is observationally equivalent to a cellular term.*

2.2 Finite representatives

The Cellularisation Theorem reduces arbitrary terms, up to observational equivalence, to a restricted syntactic class. We now use this restriction to obtain the finiteness result underlying decidability. The key point is that a closed instance of a cell can only return a ground constant or select one of its holes. Although a cell may syntactically contain arbitrarily many holes, at any fixed type only finitely many of them can correspond to observationally distinct behaviours.

The construction proceeds by induction on the order of types.

Definition 2.2.1. The *order* of a type is defined by $\mathrm{ord}(o) = 0$ and $\mathrm{ord}(A \to B) = \max(\mathrm{ord}(A)+1, \mathrm{ord}(B))$.

Recall that every type can be written as $A = [A_1, \ldots, A_n] = A_1 \to \cdots \to A_n \to o$, and each A_i has strictly smaller order than A.

Lemma 2.2.2 (Arrow decomposition)**.** *For every type A, there exist $n \geq 0$ and types $A_1, \ldots, A_n$ such that $A = [A_1, \ldots, A_n]$. If $A \neq o$, then $n \geq 1$.*

Before giving the construction, we isolate its main finiteness phenomenon. Suppose $t = \lambda y_1 \cdots y_n.\Sigma[s_1]_1 \cdots [s_L]_L$ is cellular and the head variable of Σ is $y_i : A_i$. Call a hole $[-]_l$ *selectable* if there exists a closed $u : A_i$ such that $\mathsf{nf}_\beta(\Sigma[u/y_i]) = [-]_l$.

Distinct selectable holes require observationally inequivalent selectors. Indeed, if $[-]_l$ and $[-]_m$ are selected by equivalent terms $u_l, u_m : A_i$, fill the two holes with distinct constants $a, b \in \mathcal{C}$ and all remaining holes with a fixed constant. The resulting ground context returns a on u_l and b on u_m, contradicting $u_l \equiv_{\mathcal{C}} u_m$. Hence the number of selectable holes is bounded by the number of observational-equivalence classes at type A_i.

Lemma 2.2.3 (Substitution of fresh constants)**.** *Let $\mathcal{D} = \{d_1, \ldots, d_k\}$ be a finite set of fresh ground constants disjoint from $\mathcal{C}$. Suppose that $M, N : A$ are closed terms over the enlarged signature $\mathcal{C} \cup \mathcal{D}$ and $M \equiv_{\mathcal{C}\cup\mathcal{D}} N$.*

If $\Gamma \vdash R_j : o$ *is a term over* $\mathcal{C}$ *for each* $1 \leq j \leq k$*, then*

$$\Gamma \vdash M[R_1/d_1, \ldots, R_k/d_k] \simeq_{\mathrm{ctx}} N[R_1/d_1, \ldots, R_k/d_k] : A,$$

where the notation denotes simultaneous replacement of the fresh constants d_j *by the terms* R_j*.*

Proof. By the equivalence of observational and contextual equivalence on closed terms, we may regard the hypothesis as contextual equivalence in the enlarged signature.

Let $\sigma \in \mathsf{ClSub}(\Gamma)$ and let $\vdash C[-] : A \Rightarrow o$ be a closed ground context over $\mathcal{C}$. Put $P_j = R_j\sigma$. Then each P_j is a closed term of type o.

Since $M \simeq_{\mathrm{ctx}} N$ in the enlarged signature and $C[-]$ is also a context over that signature, we have

$$C[M] =_{\beta\eta} C[N].$$

Simultaneously replacing each fresh constant d_j by P_j preserves $\beta\eta$-equality. Hence

$$C[M[P_1/d_1, \ldots, P_k/d_k]] =_{\beta\eta} C[N[P_1/d_1, \ldots, P_k/d_k]].$$

Since σ and $C[-]$ were arbitrary, the required contextual equivalence follows. □

Theorem 2.2.4 (Finite representative theorem)**.** *For every finite non-empty ground signature* $\mathcal{C}$ *and every type* A*, one can effectively construct a finite set* $\mathcal{R}(A, \mathcal{C})$ *of closed terms of type* A *containing a representative of every observational-equivalence class.*

Proof. We proceed by induction on $\mathrm{ord}(A)$, uniformly over finite ground signatures.

If $|\mathcal{C}| = 1$, say $\mathcal{C} = \{a\}$, every type has a single observational-equivalence class, so for $A = [A_1, \ldots, A_n]$ we may take $\mathcal{R}(A, \mathcal{C}) = \{\lambda y_1^{A_1} \cdots y_n^{A_n}.a\}$. Hence assume $|\mathcal{C}| \geq 2$.

If $A = o$, every closed term normalises to a ground constant, so we take $\mathcal{R}(o, \mathcal{C}) = \mathcal{C}$.

Now let $A = [A_1, \ldots, A_n]$ and assume the result for all types of smaller order. For each i, write $A_i = [B_1^i, \ldots, B_{p_i}^i]$ and set $k_j^i = |\mathcal{R}(B_j^i, \mathcal{C})|$.

A closed term of type A_i is observationally determined by its values on tuples of representatives from the B_j^i. Hence the number of observational-equivalence classes at A_i is bounded by

$$K_i = |\mathcal{C}|^{\prod_{j=1}^{p_i} k_j^i},$$

where the empty product is 1. By the preceding observation, a cell headed by y_i therefore has at most K_i selectable holes.

Introduce fresh ground constants $D_i = \{d_1^i, \ldots, d_{K_i}^i\}$. For each j, let $W_j^i = \mathcal{R}(B_j^i, \mathcal{C} \cup D_i)$,

which is finite by the induction hypothesis, and define

$$V_i = \{\, y_i\, w_1 \cdots w_{p_i} \mid w_j \in W_j^i \,\}.$$

If $p_i = 0$, this means $V_i = \{y_i\}$. Finally let $V = \bigcup_i V_i$.

We now define a finite grammar of representatives. Write $\bar{y} = y_1^{A_1} \cdots y_n^{A_n}$. For $S \subseteq V$, define R_S by induction on $|S|$. Let $R_\varnothing = \{\lambda\bar{y}.a \mid a \in \mathcal{C}\}$, and set

$$R_S = R_\varnothing \cup \bigcup_i \left\{ \lambda\bar{y}.\, v[u_1/d_1^i, \ldots, u_{K_i}/d_{K_i}^i] \;\middle|\; \begin{array}{l} v \in S \cap V_i, \\ \lambda\bar{y}.u_m \in R_{S\setminus\{v\}} \text{ for all } m \end{array} \right\}.$$

Finally define $\mathcal{R}(A,\mathcal{C}) = R_V$. Since V is finite and the recursion always passes to $S \setminus \{v\}$, this set is finite and effectively computable. By induction on $|S|$, every term in R_S is cellular.

It remains to show completeness. Let $t : A$ be closed. By Cellularisation, we may assume that t is cellular. We proceed by induction on the size of t.

The constant case $t = \lambda\bar{y}.a$ is immediate. Otherwise write $t = \lambda\bar{y}.\Sigma[s_1]_1 \cdots [s_L]_L$, where the head variable of Σ is $y_i : A_i$, and

$$\Sigma[-]_1 \cdots [-]_L = y_i(C_1[-]_1 \cdots [-]_L) \cdots (C_{p_i}[-]_1 \cdots [-]_L).$$

First remove non-selectable holes. Since such a hole can never be returned by a closed instance of the head variable, its filler may be replaced by any fixed ground constant without changing the observational class. After doing this and reindexing, all remaining holes are selectable. Hence $L \leq K_i$.

Replace these holes by $d_1^i, \ldots, d_L^i$. Each $C_j[d_1^i, \ldots, d_L^i]$ has type B_j^i over the enlarged signature $\mathcal{C} \cup D_i$. Choose $w_j \in W_j^i$ such that $C_j[d_1^i, \ldots, d_L^i] \equiv_{\mathcal{C}\cup D_i} w_j$, and let $v_\Sigma = y_i w_1 \cdots w_{p_i} \in V_i$.

To make the subsequent substitution of the fresh constants precise, let $z : A_i$ be fresh, and let Σ_z be the cell obtained from Σ by replacing its head variable y_i by z. Put $v_{\Sigma,z} = z\, w_1 \cdots w_{p_i}$. By compatibility,

$$\lambda z^{A_i}.\Sigma_z[d_1^i]_1 \cdots [d_L^i]_L \equiv_{\mathcal{C}\cup D_i} \lambda z^{A_i}.v_{\Sigma,z}.$$

By the induction hypothesis on the size of t, for each $1 \leq l \leq L$ choose r_l such that $\lambda\bar{y}.s_l \equiv \lambda\bar{y}.r_l \in R_V$. If $L < K_i$, fix some $a \in \mathcal{C}$ and put $r_{L+1} = \cdots = r_{K_i} = a$. Then $\lambda\bar{y}.r_m \in R_\varnothing \subseteq R_V$ also for $L < m \leq K_i$.

We now apply the Substitution of Fresh Constants Lemma to the preceding equivalence, replacing each d_m^i by r_m, for $1 \leq m \leq K_i$. It is important to substitute also for $d_{L+1}^i, \ldots, d_{K_i}^i$: although these constants do not occur in the original cell, they may occur in the chosen representatives w_j.

Writing $\Gamma = y_1 : A_1, \ldots, y_n : A_n$, the lemma gives

$$\Gamma \vdash \lambda z^{A_i}.\Sigma_z[r_1]_1 \cdots [r_L]_L \simeq_{\mathrm{ctx}} \lambda z^{A_i}.v_{\Sigma,z}[r_1/d_1^i, \ldots, r_{K_i}/d_{K_i}^i].$$

By compatibility, we may apply both sides to y_i. After β-conversion this gives

$$\Gamma \vdash \Sigma[r_1]_1 \cdots [r_L]_L \simeq_{\mathrm{ctx}} v_\Sigma[r_1/d_1^i, \ldots, r_{K_i}/d_{K_i}^i] : o.$$

Abstracting the variables $\bar{y}$ and using the equivalence of contextual and observational equivalence on closed terms, we obtain

$$\lambda\bar{y}.\Sigma[r_1]_1 \cdots [r_L]_L \equiv \lambda\bar{y}.\, v_\Sigma[r_1/d_1^i, \ldots, r_{K_i}/d_{K_i}^i].$$

On the other hand, since $\lambda\bar{y}.s_l \equiv \lambda\bar{y}.r_l$ for every $1 \leq l \leq L$, compatibility gives

$$\lambda\bar{y}.\Sigma[s_1]_1 \cdots [s_L]_L \equiv \lambda\bar{y}.\Sigma[r_1]_1 \cdots [r_L]_L.$$

Therefore

$$t \equiv \lambda\bar{y}.\, v_\Sigma[r_1/d_1^i, \ldots, r_{K_i}/d_{K_i}^i].$$

The only possible obstruction is that some r_l may itself have been constructed using v_Σ, whereas the definition of R_V requires the fillers of an outer occurrence of v_Σ to lie in $R_{V\setminus\{v_\Sigma\}}$.

We first note that the construction is monotone in the indexing set: if $S \subseteq T \subseteq V$, then $R_S \subseteq R_T$. This follows immediately by induction on $|S|$ from the definition of the sets R_S.

Choose, for each $1 \leq m \leq K_i$, a derivation witnessing $\lambda\bar{y}.r_m \in R_V$. Let μ be the total number of applications of the recursive clause in these derivations whose chosen element $v \in V$ is v_Σ.

Suppose $\mu > 0$. Then for some l, the chosen derivation of r_l contains an application of the recursive clause using v_Σ. Consequently, the corresponding subterm of r_l has the form

$$M[v_\Sigma[r'_1/d_1^i, \ldots, r'_{K_i}/d_{K_i}^i]]$$

for suitable $M[-]$, where the terms r'_m are supplied by the premises of that application. Hence the corresponding part of the whole term has the form

$$v_\Sigma[r_1/d_1^i, \ldots, M[v_\Sigma[r'_1/d_1^i, \ldots, r'_{K_i}/d_{K_i}^i]]/d_l^i, \ldots, r_{K_i}/d_{K_i}^i].$$

Viewing the fresh constants $d_1^i, \ldots, d_{K_i}^i$ as the holes of the cell represented by v_Σ, the Shrink-

ing Lemma gives the observationally equivalent term

$$v_\Sigma[r_1/d_1^i, \ldots, M[r_l']/d_l^i, \ldots, r_{K_i}/d_{K_i}^i].$$

At the level of the chosen derivation of r_l, this transformation amounts to replacing the corresponding node labelled v_Σ by its l-th premise. That premise derives a term in a set of the form $R_{S\setminus\{v_\Sigma\}}$, and hence, by monotonicity, may be used in place of the original node in the remainder of the derivation. Thus the modified filler again has a derivation in R_V, but the total number of occurrences of v_Σ in the chosen derivations has decreased by one.

Repeating this transformation strictly decreases μ, and hence terminates after finitely many steps. We therefore obtain fillers $q_1, \ldots, q_{K_i}$ admitting derivations in R_V which contain no application of the recursive clause labelled by v_Σ. Such derivations use only elements of $V\setminus\{v_\Sigma\}$, and hence witness $\lambda\bar{y}.q_m \in R_{V\setminus\{v_\Sigma\}}$ for every $1 \leq m \leq K_i$. Moreover, at every step the whole outer term is replaced by an observationally equivalent one, so

$$t \equiv \lambda\bar{y}.\, v_\Sigma[q_1/d_1^i, \ldots, q_{K_i}/d_{K_i}^i].$$

By the definition of R_V,

$$\lambda\bar{y}.\, v_\Sigma[q_1/d_1^i, \ldots, q_{K_i}/d_{K_i}^i] \in R_V.$$

Hence R_V contains a representative of the observational-equivalence class of t, completing the proof. □

The theorem gives not merely finiteness of the quotient at each type, but a uniform effective construction of representatives. This turns the applicative characterisation of contextual equivalence into a finite test.

2.3 Deciding contextual equivalence

Corollary 2.3.1. *Contextual equivalence in STLC with a single ground type and finitely many ground constants is decidable, both for closed and open terms.*

Proof. First let $t, t' : A_1 \to \cdots \to A_n \to o$ be closed. Compute the finite sets $\mathcal{R}(A_i, \mathcal{C})$ and test, for every tuple $r_i \in \mathcal{R}(A_i, \mathcal{C})$, whether $t\, r_1 \cdots r_n =_\beta t'\, r_1 \cdots r_n$. This is a finite test, and each comparison is decidable by normalisation.

Soundness is immediate. For completeness, let $u_i : A_i$ be arbitrary closed arguments and choose $r_i \in \mathcal{R}(A_i, \mathcal{C})$ with $u_i \equiv r_i$. Repeated use of the Context Lemma gives $t\, u_1 \cdots u_n \equiv$

$t\, r_1 \cdots r_n$ and $t'\, u_1 \cdots u_n \equiv t'\, r_1 \cdots r_n$. Hence agreement on every representative tuple implies agreement on every closed tuple, and the applicative characterisation yields $t \simeq_{\mathrm{ctx}} t'$.

Now let $\Gamma = x_1 : C_1, \ldots, x_m : C_m \vdash M, N : A$. Enumerate the finitely many substitutions $\rho = [r_1/x_1, \ldots, r_m/x_m]$ with $r_i \in \mathcal{R}(C_i, \mathcal{C})$, and apply the closed-term decision procedure to $M\rho$ and $N\rho$.

Conversely, let $\sigma \in \mathsf{ClSub}(\Gamma)$ be arbitrary. Choose $r_i \in \mathcal{R}(C_i, \mathcal{C})$ with $\sigma(x_i) \equiv r_i$, and write $\rho = [r_1/x_1, \ldots, r_m/x_m]$. Compatibility under substitution gives $M\sigma \equiv M\rho$ and $N\sigma \equiv N\rho$. Thus, if every representative substitution passes the test, then $M\sigma \equiv N\sigma$ for every closing substitution σ. By the open-term characterisation of contextual equivalence, $\Gamma \vdash M \simeq_{\mathrm{ctx}} N : A$. □

Thus the infinite family of contexts in the definition of contextual equivalence is replaced by a finite and effectively computable family of syntactic behaviours.

2.4 Examples

Throughout the following examples let $\mathcal{C} = \{a, b\}$.

Example 2.4.1 (A first finite representative set). Consider $o \to o$. Here $A_1 = o$, so $p_1 = 0$ and $V_1 = \{y\}$. The constant clause gives $K_a = \lambda x^o.a$ and $K_b = \lambda x^o.b$, while the single non-constant cell gives $I = \lambda x^o.x$. Hence we may take

$$\mathcal{R}(o \to o, \mathcal{C}) = \{K_a, I, K_b\}.$$

Thus every closed $F : o \to o$ is observationally equivalent to one of these three terms. In particular, $F \equiv G$ iff $F\, a =_\beta G\, a$ and $F\, b =_\beta G\, b$.

For example, $I \not\equiv K_a$: although $I\, a =_\beta K_a\, a = a$, we have $I\, b =_\beta b$ while $K_a\, b =_\beta a$.

Example 2.4.2 (A non-syntactic equivalence). Let $T = \lambda f^{o\to o}.f\, a$ and $U = \lambda f^{o\to o}.f(f\, a)$. They have distinct $\beta\eta$-normal forms, but it is enough to test them on I, K_a, K_b:

f	$T\, f$	$U\, f$
I	a	a
K_a	a	a
K_b	b	b

Hence $T \equiv U$. Thus observational equivalence is strictly coarser than $\beta\eta$-equality.

Example 2.4.3 (A separating representative). Let $T = \lambda f^{o\to o}.f\, a$ and $V = \lambda f^{o\to o}.f\, b$. The representative $I = \lambda x^o.x$ separates them, since $T\, I =_\beta a$ while $V\, I =_\beta b$. Therefore $T \not\equiv V$.

The result of this chapter gives a purely syntactic answer to the apparently infinitary nature of contextual equivalence. In the next chapter we ask whether the same distinctions can instead be captured by equality in a denotational model.

Chapter 3

Semantics

As the previous chapter illustrated, syntactic methods can be highly effective in the study of contextual equivalence. Nevertheless, programs are not merely pieces of syntax: like expressions in natural language or formulae in a formal logical system, they invite the question of what they mean. This motivates the search for semantic accounts of programming languages, in which programs are interpreted as mathematical objects whose structure reflects their computational behaviour. In this chapter, we focus on one such account: denotational semantics.

The central question is then *full abstraction*: when does equality of denotations coincide exactly with contextual equivalence? This asks for a model which contains neither too few distinctions, identifying programs which can be separated operationally, nor too many, distinguishing programs which no context of the language can tell apart.

The chapter has two main stages. We first develop the semantic theory of the calculus with two ground constants. After recalling the standard cartesian closed interpretation of simply typed λ-calculus and formulating full abstraction, we consider the failure of the set-theoretic model and introduce Laird's bistable model [10]. We prove universality, derive full abstraction, and show independently that each type interpretation is finite and effectively presentable.

The second stage contains the main construction of the chapter. We show that a naive enlargement of Laird's ground object is obstructed by algebraic consequences of the Shrinking Lemma. Instead, we reduce finite ground observations with $|\mathcal{C}| \geq 2$ to binary ones. For $P_{\mathcal{C}} = \{p : \mathcal{C} \twoheadrightarrow \mathcal{C}_2\}$, we prove

$$M \equiv_{\mathcal{C}} N \quad \Longleftrightarrow \quad p_*(M) \equiv_{\mathcal{C}_2} p_*(N) \text{ for every } p \in P_{\mathcal{C}}.$$

This leads to the product interpretation $[\![M]\!]_{\mathcal{C}} = \big([\![p_*(M)]\!]_{\mathrm{Laird}}\big)_{p \in P_{\mathcal{C}}}$, which we prove fully abstract for every finite ground signature with $|\mathcal{C}| \geq 2$. Since each binary component is finite and effectively presentable, the product model yields a semantic decision procedure for contextual equivalence.

For simplicity, we formulate the semantic results below for closed terms. The corresponding statements for open terms are obtained by passing to their λ-closures.

3.1 Cartesian closed semantics of STLC

We begin with the standard categorical interpretation of simply typed λ-calculus. We assume familiarity with basic category theory and recall only the structure needed below; see, for example, [3, Chapter 6] or [7] for a general treatment.

Definition 3.1.1. A *cartesian closed category* (CCC) is a category $\mathbf{C}$ with finite products and, for every pair of objects X, Y, an exponential object Y^X equipped with an evaluation morphism $\mathsf{ev}_{X,Y} : Y^X \times X \to Y$. The exponential satisfies the universal property that every morphism $f : Z \times X \to Y$ has a unique curried form $\Lambda(f) : Z \to Y^X$ such that

$$\mathsf{ev}_{X,Y} \circ \langle \Lambda(f) \circ \pi_1, \pi_2 \rangle = f.$$

The exponential Y^X plays the role of an abstract function space. Its universal property gives the correspondence $\mathbf{C}(Z \times X, Y) \cong \mathbf{C}(Z, Y^X)$ between maps of several arguments and their curried forms. In **Set**, exponentials are the usual sets of functions. In other cartesian closed categories, morphisms may be required to preserve additional structure, allowing different notions of semantic function space for the same typed language.

3.1.1 CCC interpretation of STLC

Let $\mathbf{C}$ be a cartesian closed category. To interpret $\Lambda_{\mathcal{C}}$, choose an object $D = [\![o]\!]$ and, for every ground constant $c \in \mathcal{C}$, a global element $[\![c]\!] : 1 \to D$. The remaining types and terms are interpreted using the cartesian closed structure.

- **Types.** Function types are interpreted by exponentials: $[\![A \to B]\!] = [\![B]\!]^{[\![A]\!]}$.
- **Contexts.** A typing context $\Gamma = x_1 : A_1, \ldots, x_n : A_n$ is interpreted by $[\![\Gamma]\!] = [\![A_1]\!] \times \cdots \times [\![A_n]\!]$, with $[\![\varnothing]\!] = 1$.
- **Terms.** A judgement $\Gamma \vdash M : A$ is interpreted by a morphism $[\![M]\!] : [\![\Gamma]\!] \to [\![A]\!]$.

  Variables are interpreted by the corresponding product projections. A ground constant c in a non-empty context is interpreted by the composite $[\![\Gamma]\!] \to 1 \xrightarrow{[\![c]\!]} D$.

  If $\Gamma \vdash M : A \to B$ and $\Gamma \vdash N : A$, then

$$[\![MN]\!] = \mathsf{ev} \circ \langle [\![M]\!], [\![N]\!] \rangle.$$

If $\Gamma, x : A \vdash M : B$, then $[\![\lambda x^A.M]\!] = \Lambda([\![M]\!])$.

The universal properties of products and exponentials ensure that this interpretation validates the β- and η-laws. In particular, $M =_{\beta\eta} N$ implies $[\![M]\!] = [\![N]\!]$. Thus every such interpretation gives a sound semantics for the equational theory of STLC. The question relevant to contextual equivalence is stronger: whether semantic equality captures *exactly* the distinctions observable by program contexts. We turn to this next.

3.2 Denotational semantics and full abstraction

Assigning denotations to terms is only useful for our purposes if the semantic interpretation is related to the operational behaviour of the language. In particular, we would like semantic equality to provide a sound method for proving contextual equivalence, and ultimately to ask whether it characterises contextual equivalence exactly.

We isolate three standard properties of a denotational semantics. The presentation follows the usual development of compositionality, soundness, and adequacy; see, for example, [12].

Definition 3.2.1. A semantics is *compositional* if semantic equality is preserved by program contexts: if $[\![M]\!] = [\![N]\!]$, then $[\![C[M]]\!] = [\![C[N]]\!]$ whenever $C[M]$ and $C[N]$ are well-typed.

For the categorical interpretations considered above, compositionality is built into the interpretation of terms: the denotation of each term constructor is determined from the denotations of its immediate subterms.

Recall that $\Lambda_{\mathcal{C}}$ is strongly normalising and that its observable results are the constants of the finite ground signature $\mathcal{C}$. Thus every closed term $P : o$ has a unique ground normal form $c \in \mathcal{C}$.

Definition 3.2.2. A denotational interpretation is *sound with respect to β-conversion* if $M =_\beta N$ implies $[\![M]\!] = [\![N]\!]$.

Definition 3.2.3. A denotational interpretation is *adequate at ground type* if, for every closed $P : o$ and every $c \in \mathcal{C}$, $[\![P]\!] = [\![c]\!]$ implies $P \Downarrow c$.

Soundness says that computation does not change denotation, while adequacy gives the converse at observable types: if a closed ground program has the denotation of a ground constant, then it evaluates to that constant. Together with compositionality, these properties ensure that semantic equality cannot identify contextually distinguishable programs.

Theorem 3.2.4 (Semantic equality implies contextual equivalence)**.** *Suppose that an interpretation is compositional, sound with respect to β-conversion, and adequate at ground type. Then,*

for all closed terms $M, N : A$*,*

$$[\![M]\!] = [\![N]\!] \quad \Longrightarrow \quad M \equiv_{\mathcal{C}} N.$$

Proof. Assume $[\![M]\!] = [\![N]\!]$, and let $C[-]$ be an arbitrary closing context of ground result type. By compositionality, $[\![C[M]]\!] = [\![C[N]]\!]$.

Since the calculus is terminating, $C[M] \Downarrow c$ for some $c \in \mathcal{C}$. Hence $C[M] =_\beta c$, so soundness gives $[\![C[M]]\!] = [\![c]\!]$. Therefore $[\![C[N]]\!] = [\![c]\!]$, and adequacy implies $C[N] \Downarrow c$. Thus $C[M]$ and $C[N]$ have the same observable result.

Since $C[-]$ was arbitrary, $M \equiv_{\mathcal{C}} N$. □

The theorem shows that equality of denotations is a sound method for establishing contextual equivalence. The natural question is whether this method is also complete: if two programs cannot be distinguished by any context, must they have equal denotations?

Definition 3.2.5. A denotational semantics is *fully abstract* for contextual equivalence if, for all closed terms $M, N : A$,

$$M \equiv_{\mathcal{C}} N \quad \Longleftrightarrow \quad [\![M]\!] = [\![N]\!].$$

Under the hypotheses of the preceding theorem, the right-to-left implication is already guaranteed. The substantive requirement of full abstraction is therefore the converse

$$M \equiv_{\mathcal{C}} N \quad \Longrightarrow \quad [\![M]\!] = [\![N]\!].$$

It says that the model introduces no distinctions between programs which the language itself cannot observe. Equivalently, every semantic distinction between denotations of programs must be reflected by some program context.

A fully abstract model therefore contains exactly the distinctions relevant to contextual equivalence: semantic equality is neither too coarse, by identifying distinguishable programs, nor too fine, by separating programs with identical observable behaviour.

Remark 3.2.6. Some denotational models carry an order representing operational approximation, in which case one may instead seek an inequational correspondence

$$M \precsim N \quad \Longleftrightarrow \quad [\![M]\!] \sqsubseteq [\![N]\!].$$

This is usually called *inequational full abstraction*. We are concerned here only with the equational form. Although the bistable model introduced below carries an extensional order, the observational correspondence studied in this chapter is between contextual equivalence and equality of denotations.

3.3 Syntactic constraints on fully abstract semantics

From this point, we formulate semantic arguments elementwise, as appropriate for the concrete cartesian closed categories **Set**, **BBO** considered below. Thus, if $D = [\![o]\!]$, we write $u \in D^n \Rightarrow D$ and $\vec{x} \in D^n$ for elements of the corresponding semantic objects.

The preceding syntactic analysis does more than establish finiteness of contextual equivalence classes. Padovani's Shrinking Lemma exposes equations satisfied by observable program behaviour, and any semantics complete for observational equivalence must validate these equations. They therefore give necessary algebraic conditions for completeness, and hence for full abstraction, providing a systematic test for candidate semantic function spaces. These constraints rely essentially on the ground-only signature assumed throughout this chapter.

For $n \geq 1$, recall that $F_n = \underbrace{[o, \ldots, o]}_{n \text{ arguments}}$. Writing $D = [\![o]\!]$, we have $[\![F_n]\!] \cong D^n \Rightarrow D$. For $\vec{x} = (x_1, \ldots, x_n) \in D^n$ and $a \in D$, write $\vec{x}[i := a] = (x_1, \ldots, x_{i-1}, a, x_{i+1}, \ldots, x_n)$.

Proposition 3.3.1 (Necessary shrinking equations)**.** *Suppose that the interpretation is complete with respect to observational equivalence. Then, for every $n \geq 1$, every $u \in D^n \Rightarrow D$, every $1 \leq i \leq n$, and all $\vec{x}, \vec{y} \in D^n$,*

$$u\big(\vec{x}[i := u(\vec{y})]\big) = u\big(\vec{x}[i := y_i]\big).$$

Proof. Fix $n \geq 1$ and $1 \leq i \leq n$, and let $p : F_n$ and $x_1, \ldots, x_n, y_1, \ldots, y_n : o$. Define

$$T_{n,i} = \lambda p^{F_n}.\lambda x_1^o \cdots x_n^o.\lambda y_1^o \cdots y_n^o.\, p\, x_1 \cdots x_{i-1} (p\, y_1 \cdots y_n) x_{i+1} \cdots x_n,$$
$$S_{n,i} = \lambda p^{F_n}.\lambda x_1^o \cdots x_n^o.\lambda y_1^o \cdots y_n^o.\, p\, x_1 \cdots x_{i-1} y_i x_{i+1} \cdots x_n.$$

Both have type $F_n \to \underbrace{o \to \cdots \to o}_{2n \text{ arguments}} \to o$.

Consider the n-hole cell $\Sigma[-]_1 \cdots [-]_n = p\,[-]_1 \cdots [-]_n$. The body of $T_{n,i}$ contains

$$\Sigma[x_1] \cdots [x_{i-1}] \big[\Sigma[y_1] \cdots [y_n]\big] [x_{i+1}] \cdots [x_n].$$

By the Shrinking Lemma, the inner occurrence may be replaced by its i-th argument y_i. Hence $T_{n,i} \equiv S_{n,i}$, and completeness gives $[\![T_{n,i}]\!] = [\![S_{n,i}]\!]$.

Now let $u \in D^n \Rightarrow D$ and $\vec{x}, \vec{y} \in D^n$. Evaluating these equal denotations at $u, \vec{x}, \vec{y}$ gives

$$u\big(\vec{x}[i := u(\vec{y})]\big) = u\big(\vec{x}[i := y_i]\big),$$

as required. □

Corollary 3.3.2. *If, for some $n \geq 1$, the semantic function space $D^n \Rightarrow D$ contains an element violating a shrinking equation, then the interpretation is not complete with respect to observational equivalence, and therefore cannot be fully abstract.*

Proof. If for some $u \in D^n \Rightarrow D$, $1 \leq i \leq n$, and $\vec{x}, \vec{y} \in D^n$,

$$u\big(\vec{x}[i := u(\vec{y})]\big) \neq u\big(\vec{x}[i := y_i]\big),$$

then $T_{n,i} \equiv S_{n,i}$ but $[\![T_{n,i}]\!] \neq [\![S_{n,i}]\!]$. Thus completeness fails. □

Example 3.3.3. For $n = 1$, the shrinking equation is $u(u(y)) = u(y)$. Hence every element $u \in D \Rightarrow D$ admitted by the semantic function space must be idempotent.

3.4 The set-theoretic model

The most direct denotational interpretation of simply typed λ-calculus is obtained in the category **Set**. For a finite ground signature $\mathcal{C}$, interpret the ground type by $D_o = \mathcal{C}$ and function types by full function spaces $D_{A\to B} = D_B^{D_A}$. Ground constants are interpreted by the corresponding elements of $\mathcal{C}$, while abstraction and application are interpreted by ordinary function formation and application.

This model is natural, but its function spaces are too large for full abstraction: they contain elements which violate equations satisfied by all observable program behaviours and can therefore distinguish contextually equivalent terms.

Proposition 3.4.1. *Let $\mathcal{C}$ be a finite ground signature with $|\mathcal{C}| \geq 2$. The set-theoretic interpretation of $\Lambda_{\mathcal{C}}$ is not fully abstract.*

Proof. The unary shrinking equation gives

$$\lambda f^{o\to o}.\lambda x^o.f(fx) \equiv_{\mathcal{C}} \lambda f^{o\to o}.\lambda x^o.fx.$$

Thus completeness would require every $u \in [\![o \to o]\!]$ to be idempotent.

In the set-theoretic interpretation, $[\![o \to o]\!] = \mathcal{C}^{\mathcal{C}}$, so every function $\mathcal{C} \to \mathcal{C}$ is admitted. Choose distinct $a, b \in \mathcal{C}$, and let $s : \mathcal{C} \to \mathcal{C}$ transpose a and b, fixing every other element. Then $s(s(a)) = a \neq b = s(a)$, so s is not idempotent. Consequently,

$$[\![\lambda f.\lambda x.f(fx)]\!](s)(a) = a \qquad \text{but} \qquad [\![\lambda f.\lambda x.fx]\!](s)(a) = b.$$

The two contextually equivalent terms therefore have distinct denotations. Hence the set-theoretic interpretation fails completeness and is not fully abstract. □

The failure is not caused merely by the presence of non-denotable elements. A fully abstract model may contain such elements provided that they introduce no distinctions between definable programs beyond those observable by program contexts. The problem with the set-theoretic interpretation is more specific: its unrestricted function spaces admit behaviours which violate equations forced by the syntax.

This suggests replacing full set-theoretic function spaces by more restricted spaces whose structure reflects the sequential behaviour of programs. Laird's bistable model provides precisely such a refinement.

3.5 Laird's two-point model

In this section we consider an order-theoretic model provided by Laird in [10]. Laird gives a denotational account of STLC with product types and a binary ground signature (denoted by $\Lambda_{\perp}^{\top}$) using the cartesian closed category **BBO** of bistable biorders and monotone bistable functions. For the sake of coherence with the previous chapter, we use the restriction of Laird's interpretation to the arrow-only fragment for now, to which we refer as $\Lambda_{\mathcal{C}_2}$. Finite categorical products are still used to interpret typing contexts, as in any cartesian closed semantics, but no product constructor is added to the object language.

3.5.1 Bistable biorders and interpretation

The model refines the standard set-theoretic interpretation by replacing full function spaces with spaces of functions preserving additional order-theoretic structure intended to capture sequential behaviour.

Definition 3.5.1 (Bistable biorder)**.** A *bistable biorder* is a triple $(D, \sqsubseteq, \smile)$, where:

- $(D, \sqsubseteq)$ is a partial order, called the *extensional order*,
- $\smile$ is an equivalence relation over D, called *bistable coherence*, such that every coherence class is a distributive lattice with respect to $\sqsubseteq$, and its binary meet and join are also respectively the greatest lower bound and least upper bound of the two elements in D.

Definition 3.5.2 (Monotone bistable function)**.** Let D and E be bistable biorders. A function $f : D \to E$ is:

- *monotone* if $x \sqsubseteq y$ implies $f(x) \sqsubseteq f(y)$.
- *bistable* if, whenever $x \smile y$, one has $f(x) \smile f(y)$, and $f(x \wedge y) = f(x) \wedge f(y)$, $f(x \vee y) = f(x) \vee f(y)$. Equivalently, the restriction of f to each coherence class of D is a lattice homomorphism into a coherence class of E.

With these definitions we are in position to introduce the category **BBO** which allows us to formulate the bistable model.

Definition 3.5.3. The category **BBO** has bistable biorders as objects, and its morphisms are monotone bistable functions.

Definition 3.5.4. Given bistable biorders D, E, we define the function-space $D \Rightarrow E$ to be the set of monotone and bistable functions $f : D \to E$. Furthermore:

- extensional order is defined pointwise by $f \sqsubseteq g \iff \forall x \in D,\ f(x) \sqsubseteq g(x)$.
- two functions $f, g : D \to E$ are coherent, written $f \smile g$, if $\forall x \in D,\ f(x) \smile g(x)$, and $x \smile y$ implies that $f(x) \wedge g(y) = f(y) \wedge g(x)$ and $f(x) \vee g(y) = f(y) \vee g(x)$.

Lemma 3.5.5 ([10], Lemma 2.6.)**.** *$D \Rightarrow E$ is a bistable biorder.*

Proposition 3.5.6 ([10], Proposition 2.7.)**. BBO** *is cartesian closed.*

Definition 3.5.7 (The bistable model)**.** The bistable model consists of the following interpretations in **BBO**.

- o is interpreted by the *Sierpiński space* given by $(\Sigma = \{\bot, \top\},\ \bot \sqsubseteq \top,\ \bot \smile \top)$. For the sake of brevity we abuse notation and simply write Σ. Thus Σ consists of a single coherence class, whose meet and join are determined by the extensional order.
- Function types are interpreted recursively by $[\![A \to B]\!] = [\![A]\!] \Rightarrow [\![B]\!]$.
- The two constants are interpreted by $[\![\bot]\!] = \bot,\ [\![\top]\!] = \top$.
- Variables are interpreted by projections, abstraction by currying, and application by the evaluation morphism.

Remark 3.5.8. In the terminating calculus considered here, $\bot$ and $\top$ should be regarded simply as the names of the two ground constants.

Example 3.5.9. Already at the first function type, the bistable interpretation is strictly smaller than the full set-theoretic function space. Indeed, $\Sigma \Rightarrow \Sigma = \{c_\bot, \mathrm{id}_\Sigma, c_\top\}$, where $c_\bot$ and $c_\top$ are the two constant functions. The transposition s satisfying $s(\bot) = \top,\ s(\top) = \bot$ is absent because it is not monotone with respect to the extensional order $\bot \sqsubseteq \top$. A quick check shows that there are two coherence classes: $\{c_\bot, c_\top\}$ and $\{\mathrm{id}\}$. This has a direct computational reading in terms of how the corresponding terms treat their input: $\lambda x.\bot$ and $\lambda x.\top$ ignore their input and return their final value, whereas $\lambda x.x$ inspects and returns its input.

Remark 3.5.10. Observe that the discrete interpretation of the ground type would lead to the inclusion of the transposition in the model, and thus immediate failure of full abstraction as is the case for the set-theoretic model. At higher types, extensional order and bistability impose further restrictions which exclude forms of non-sequential behaviour present in unrestricted set-theoretic function spaces.

For the development later in this chapter, the essential consequence of Laird's binary interpretation is that it is fully abstract.

We establish the binary result here through *universality*: every element of the interpretation of a type is denoted by a closed term. Laird proves universality by reducing arbitrary types to first-order types using definable retractions [10], to which we turn in Chapter 5. We give instead an alternative proof based on a semantic adaptation of the strictness-based listing method discussed below.

Besides giving a self-contained route to full abstraction, the argument makes explicit a close correspondence between syntactic and semantic presentations of the minimal model. To this end, we need the following definition.

Definition 3.5.11 (Strictness in an argument)**.** Let $A = [A_1, \ldots, A_m]$ and let $a \in [\![A]\!]$. By uncurrying, we regard a as a monotone bistable function $\bar{a} : [\![A_1]\!] \times \cdots \times [\![A_m]\!] \longrightarrow \Sigma$.

For $1 \leq i \leq m$, we say that a is *i-strict* if

$$x_i = \bot_{A_i} \Longrightarrow \bar{a}(x_1, \ldots, x_m) = \bot, \text{ and } x_i = \top_{A_i} \Longrightarrow \bar{a}(x_1, \ldots, x_m) = \top.$$

The argument we present has two ingredients. First, a property called *strong sequentiality* classifies every non-constant semantic element according to an argument in which it is strict. Second, a family of definable combinators reconstructs an element from semantic elements at smaller types. We introduce the necessary tools in the next two subsections.

3.5.2 The structure of binary semantic elements

Strong sequentiality

Lemma 3.5.12 (Pointedness of type interpretations)**.** *For every arrow-only type A, the bistable biorder $[\![A]\!]$ is pointed: it has a least element $\bot_A$ and a greatest element $\top_A$, and $\bot_A \smile \top_A$. Moreover, these are definable.*

Proof. Define $\bot_A$ and $\top_A$ recursively on A. At ground type, $\bot_o = \bot$ and $\top_o = \top$. At a function type $A = B \to C$, let $\bot_{B\to C} = c_{\bot_C}$ and $\top_{B\to C} = c_{\top_C}$, where $c_{\bot_C}(x) = \bot_C$ and $c_{\top_C}(x) = \top_C$ for all $x \in [\![B]\!]$. These constant maps are monotone and bistable, and are respectively least and greatest since the extensional order on exponentials is pointwise.

They are coherent because, for every $x \smile y$, $c_{\perp_C}(x) \wedge c_{\top_C}(y) = \perp_C = c_{\perp_C}(y) \wedge c_{\top_C}(x)$ and $c_{\perp_C}(x) \vee c_{\top_C}(y) = \top_C = c_{\perp_C}(y) \vee c_{\top_C}(x)$. Hence $c_{\perp_C} \smile c_{\top_C}$.

If $A = [A_1, \ldots, A_m]$, define $K^A = \lambda u^o x_1^{A_1} \cdots x_m^{A_m}.\, u : o \to A$. Then $[\![K^A \perp]\!] = \perp_A$ and $[\![K^A \top]\!] = \top_A$. □

The following is the strong-sequentiality property of bistable functions.

Lemma 3.5.13 (Strong sequentiality)**.** *Let $D_1, \ldots, D_m$ be pointed bistable biorders, where $m \geq 1$, and let $f : D_1 \times \cdots \times D_m \longrightarrow \Sigma$ be monotone and bistable. Suppose that f is strict, in the sense that $f(\perp, \ldots, \perp) = \perp$, $f(\top, \ldots, \top) = \top$. Then f is i-strict for some $1 \leq i \leq m$.*

Proof. For each j, define

$$u_k^j = \begin{cases} \perp_{D_j}, & k = j, \\ \top_{D_k}, & k \neq j, \end{cases} \quad \text{and} \quad v_k^j = \begin{cases} \top_{D_j}, & k = j, \\ \perp_{D_k}, & k \neq j. \end{cases}$$

Thus u^j is the tuple which is bottom in its j-th coordinate and top elsewhere, while v^j is top in its j-th coordinate and bottom elsewhere.

All the tuples $u^1, \ldots, u^m$ belong to the same coherence class, and $\bigwedge_{j=1}^m u^j = (\perp, \ldots, \perp)$.

Since f is bistable and strict, $\bigwedge_{j=1}^m f(u^j) = f\left(\bigwedge_{j=1}^m u^j\right) = \perp$. As the codomain is the two-element lattice Σ, there is some i such that $f(u^i) = \perp$.

Dually, $\bigvee_{j=1}^m v^j = (\top, \ldots, \top)$, so there is some k such that $f(v^k) = \top$. If $i \neq k$, then $v^k \sqsubseteq u^i$. Monotonicity would therefore imply $\top = f(v^k) \sqsubseteq f(u^i) = \perp$, which is impossible. Hence $i = k$.

Now suppose that $x_i = \perp_{D_i}$. Then $x \sqsubseteq u^i$, and therefore $f(x) \sqsubseteq f(u^i) = \perp$. Thus $f(x) = \perp$. Similarly, if $x_i = \top_{D_i}$, then $v^i \sqsubseteq x$, so $\top = f(v^i) \sqsubseteq f(x)$, and hence $f(x) = \top$. Therefore f is i-strict. □

Corollary 3.5.14 (Semantic head-variable classification)**.** *Let $A = [A_1, \ldots, A_m]$ and let $a \in [\![A]\!]$. Then one of the following holds:*

1. *$a = \perp_A$;*
2. *$a = \top_A$;*
3. *a is i-strict for some $1 \leq i \leq m$.*

Proof. The assertion is immediate when $m = 0$, since $[\![o]\!] = \{\perp, \top\}$. Suppose that $m \geq 1$, and regard a as its uncurried function $\bar{a} : [\![A_1]\!] \times \cdots \times [\![A_m]\!] \longrightarrow \Sigma$.

If $\bar{a}(\perp, \ldots, \perp) = \top$, then monotonicity implies that $\bar{a}$ is constantly $\top$, so $a = \top_A$.

Dually, if $\bar{a}(\top, \ldots, \top) = \perp$, then $a = \perp_A$.

Consequently, if a is neither $\perp_A$ nor $\top_A$, then $\bar{a}(\perp, \ldots, \perp) = \perp$, $\bar{a}(\top, \ldots, \top) = \top$. The strong-sequentiality lemma therefore implies that a is i-strict for some i. □

Semantic strictness detection

We next introduce a term which detects the strictness coordinate of an element.

For $A = [A_1, \ldots, A_m]$, let $F_m = [\underbrace{o, \ldots, o}_{m \text{ times}}]$, where $F_0 = o$, and define

$$\Sigma^A = \lambda f^A u_1^o \cdots u_m^o.\, f\,(K^{A_1} u_1) \cdots (K^{A_m} u_m) : A \to F_m.$$

For $1 \leq i \leq m$, let $\pi_i^m = \lambda u_1^o \cdots u_m^o.\, u_i : F_m$.

Lemma 3.5.15. *Let $a \in [\![A]\!]$, where $A = [A_1, \ldots, A_m]$. Then*

$$[\![\Sigma^A]\!](\bot_A) = c_\bot, \qquad [\![\Sigma^A]\!](\top_A) = c_\top.$$

Moreover, if a is i-strict, then $[\![\Sigma^A]\!](a) = [\![\pi_i^m]\!]$.

Proof. The first two equations follow immediately from the definitions of $\bot_A$ and $\top_A$.

Suppose that a is i-strict. Given $u_1, \ldots, u_m \in \Sigma$, the i-th argument $[\![K^{A_i}]\!](u_i)$ is equal to $\bot_{A_i}$ when $u_i = \bot$, and to $\top_{A_i}$ when $u_i = \top$. By i-strictness, $[\![\Sigma^A]\!](a)(u_1, \ldots, u_m) = u_i$. Thus $[\![\Sigma^A]\!](a) = [\![\pi_i^m]\!]$. □

Now we have enough tools to give a concrete description of denotations of first-order types.

Lemma 3.5.16. *Let $F_n = [\underbrace{o, \ldots, o}_{n}]$ with $n \geq 1$. Every element of $[\![F_n]\!]$ is one of $c_\bot$, $c_\top$, $\pi_1, \ldots, \pi_n$.*

Proof. An element of $[\![F_n]\!]$ is a monotone bistable function $f : \Sigma^n \longrightarrow \Sigma$.

If f is constant, then $f = c_\bot$ or $f = c_\top$.

Suppose that f is not constant. Since f is monotone and $(\bot, \ldots, \bot) \sqsubseteq x \sqsubseteq (\top, \ldots, \top)$ for every $x \in \Sigma^n$, non-constancy implies $f(\bot, \ldots, \bot) = \bot$ and $f(\top, \ldots, \top) = \top$. Thus f is strict. By the strong sequentiality lemma, every strict monotone bistable function $\Sigma^n \to \Sigma$ is i-strict for some $1 \leq i \leq n$. Hence, for every $x \in \Sigma^n$,

$$x_i = \bot \implies f(x) = \bot, \qquad x_i = \top \implies f(x) = \top.$$

Since $\Sigma = \{\bot, \top\}$, it follows that $f(x) = x_i$ for every $x \in \Sigma^n$. Therefore $f = \pi_i$. □

3.5.3 Semantic adaptation of Loader's decomposition

The decomposition argument in this subsection is closely based on Loader's presentation [13] of Schmidt–Schauß's listing method for the minimal model. In particular, the use of strictness, the nested multiset ordering, and the mutually defined reconstruction combinators follows the structure of Loader's argument. Our use of this machinery is semantic rather than syntactic:

the combinators are used to decompose arbitrary elements of the bistable interpretation and thereby prove their definability. A related semantic use of Schmidt–Schauß's method appears in Laird's proof of universality for models of unary PCF [8]. Here we adapt the idea to Laird's bistable model of the two-constant simply typed λ-calculus.

Nested multiset ordering

We now define the combinators used in the semantic decomposition. The recursive calls will be made at types smaller in the nested multiset ordering.

Definition 3.5.17 (Nested multiset ordering)**.** Let $A = [A_1, \ldots, A_m]$. For the purpose of defining the ordering, we regard A as carrying the finite multiset of its immediate argument types $\{A_1, \ldots, A_m\}$.

The *nested multiset ordering*, written $\prec$, is the least transitive relation on types closed under the following replacement rule:

$$\frac{H_\ell \prec B_j \quad \text{for every } 1 \leq \ell \leq k}{[B_1, \ldots, B_{j-1}, H_1, \ldots, H_k, B_{j+1}, \ldots, B_n] \prec [B_1, \ldots, B_n]},$$

where $n \geq 1$, $1 \leq j \leq n$, and $k \geq 0$.

Thus, a type becomes strictly smaller when one of its immediate argument types is replaced by finitely many types that are themselves strictly smaller. The case $k = 0$ is explicitly permitted: in that case the selected argument type is simply deleted, so that

$$[B_1, \ldots, B_{j-1}, B_{j+1}, \ldots, B_n] \prec [B_1, \ldots, B_n].$$

Importantly for our purposes, the nested multiset ordering is well-founded. We omit the proof of this property, the reader can refer to [4].

Data for types

We construct, for every type $A = [A_1, \ldots, A_m]$ and every $1 \leq i \leq m$, the following data:

- a natural number k_i^A
- types $H_{i,1}^A, \ldots, H_{i,k_i^A}^A$
- a closed term $C_i^A : H_{i,1}^A \to \cdots \to H_{i,k_i^A}^A \to A$
- closed terms $D_{i,\ell}^A : A \to H_{i,\ell}^A$ where $1 \leq \ell \leq k_i^A$

They will satisfy $H_{i,\ell}^A \prec A$ and will reconstruct every i-strict element of $[\![A]\!]$.

Construction of data The data are defined by structural induction on types. There is nothing to define at the ground type.

Now let $B = [B_1, \ldots, B_n]$ and fix $1 \leq j \leq n$. Write $B_j = A = [A_1, \ldots, A_m]$. By induction, the data C_i^A, $D_{i,\ell}^A$, and $H_{i,\ell}^A$ have already been constructed.

If $m = 0$, define $k_j^B = 0$ and

$$C_j^B = \lambda y_1^{B_1} \cdots y_n^{B_n}.\, y_j.$$

Suppose now that $m \geq 1$. Set $k_j^B = m$. For each $1 \leq i \leq m$, define

$$H_{j,i}^B = [B_1, \ldots, B_{j-1}, H_{i,1}^A, \ldots, H_{i,k_i^A}^A, B_{j+1}, \ldots, B_n].$$

Inside the scope of variables $\varphi_i : H_{j,i}^B$ and $y_1 : B_1, \ldots, y_n : B_n$, write

$$E_i = \varphi_i\, y_1 \cdots y_{j-1}\, (D_{i,1}^A y_j) \cdots (D_{i,k_i^A}^A y_j)\, y_{j+1} \cdots y_n.$$

Define

$$C_j^B = \lambda \varphi_1^{H_{j,1}^B} \cdots \varphi_m^{H_{j,m}^B}.\, \lambda y_1^{B_1} \cdots y_n^{B_n}.\, \Sigma^A\, y_j\, E_1 \cdots E_m.$$

Finally, for each $1 \leq i \leq m$, define

$$D_{j,i}^B = \lambda \psi^B.\, \lambda y_1^{B_1} \cdots y_{j-1}^{B_{j-1}}\, u_1^{H_{i,1}^A} \cdots u_{k_i^A}^{H_{i,k_i^A}^A}\, y_{j+1}^{B_{j+1}} \cdots y_n^{B_n}.$$
$$\psi\, y_1 \cdots y_{j-1}\, (C_i^A\, u_1 \cdots u_{k_i^A})\, y_{j+1} \cdots y_n.$$

By the induction hypothesis, $H_{i,\ell}^A \prec A$ for every ℓ. Hence the defining property of the nested multiset ordering gives $H_{j,i}^B \prec B$.

Semantic decomposition

The purpose of these terms is expressed by the following proposition.

Proposition 3.5.18 (Semantic decomposition)**.** *For every type $A = [A_1, \ldots, A_m]$, every $1 \leq i \leq m$, and every i-strict element $a \in [\![A]\!]$, $a = [\![C_i^A]\!]\big([\![D_{i,1}^A]\!](a)\big) \cdots \big([\![D_{i,k_i^A}^A]\!](a)\big)$.*

Proof. We prove the proposition by structural induction on the ambient type. There is nothing to prove at ground type, since a ground type has no argument position.

Let $B = [B_1, \ldots, B_n]$ and $B_j = A = [A_1, \ldots, A_m]$, and suppose that $s \in [\![B]\!]$ is j-strict. We must prove

$$s = [\![C_j^B]\!]\big([\![D_{j,1}^B]\!](s)\big) \cdots \big([\![D_{j,k_j^B}^B]\!](s)\big).$$

We first consider the case $m = 0$, so that $A = o$. By definition, $k_j^B = 0$ and

$$C_j^B = \lambda y_1^{B_1} \cdots y_n^{B_n}.\, y_j.$$

It is therefore enough to show that s is the j-th projection.

Let $r_\ell \in [\![B_\ell]\!]$ $(1 \le \ell \le n)$ be arbitrary. Since $r_j \in [\![o]\!] = \Sigma$, either $r_j = \bot$ or $r_j = \top$. If $r_j = \bot$, then j-strictness gives $s(r_1, \ldots, r_n) = \bot = r_j$. If $r_j = \top$, then similarly $s(r_1, \ldots, r_n) = \top = r_j$. Hence $s(r_1, \ldots, r_n) = r_j$ for every tuple of arguments, and therefore $s = [\![C_j^B]\!]$. This proves the required equation when $m = 0$.

Suppose now that $m \ge 1$. Equality in the function space is extensional, so let $r_\ell \in [\![B_\ell]\!]$ $(1 \le \ell \le n)$ be arbitrary, and put $r = r_j \in [\![A]\!]$.

Unfolding the definitions of C_j^B and $D_{j,i}^B$, the value of the right-hand side at $r_1, \ldots, r_n$ is

$$[\![\Sigma^A]\!](r)(q_1, \ldots, q_m),$$

where $q_i = s\big(r_1, \ldots, r_{j-1}, [\![C_i^A]\!]([\![D_{i,1}^A]\!](r), \ldots, [\![D_{i,k_i^A}^A]\!](r)), r_{j+1}, \ldots, r_n\big)$.

We distinguish the cases given by the semantic head-variable classification.

If $r = \bot_A$, then $[\![\Sigma^A]\!](r) = c_\bot$, so the displayed expression is $\bot$. Since s is j-strict and its j-th argument is $\bot_A$, we have $s(r_1, \ldots, r_n) = \bot$.

Similarly, if $r = \top_A$, then $[\![\Sigma^A]\!](r) = c_\top$, so the displayed expression is $\top$, which is again equal to $s(r_1, \ldots, r_n)$ by j-strictness.

It remains to consider the case in which r is i-strict for some $1 \le i \le m$. The preceding lemma gives $[\![\Sigma^A]\!](r) = [\![\pi_i^m]\!]$, and hence $[\![\Sigma^A]\!](r)(q_1, \ldots, q_m) = q_i$.

By the induction hypothesis applied to the proper type component $A = B_j$,

$$r = [\![C_i^A]\!]\big([\![D_{i,1}^A]\!](r)\big) \cdots \big([\![D_{i,k_i^A}^A]\!](r)\big).$$

Substituting this equality into the definition of q_i gives $q_i = s(r_1, \ldots, r_n)$. Thus the two sides agree on arbitrary arguments, and hence they are equal. □

3.5.4 Universality and full abstraction

We have now all the necessary tools to prove universality.

Theorem 3.5.19 (Universality)**.** *For every type A and every element $a \in [\![A]\!]$, there exists a closed term $M : A$ such that $[\![M]\!] = a$.*

Proof. We proceed by well-founded induction on A with respect to the nested multiset ordering.

Let $A = [A_1, \ldots, A_m]$ and let $a \in [\![A]\!]$. If $a = \bot_A$, then it is denoted by the closed term $K^A \bot$. If $a = \top_A$, then it is denoted by $K^A \top$.

Otherwise, the semantic head-variable classification implies that a is i-strict for some $1 \leq i \leq m$. By semantic decomposition, $a = [\![C_i^A]\!]\big([\![D_{i,1}^A]\!](a)\big) \cdots \big([\![D_{i,k_i^A}^A]\!](a)\big)$.

For every $1 \leq \ell \leq k_i^A$, $H_{i,\ell}^A \prec A$. The induction hypothesis therefore gives a closed term $M_\ell : H_{i,\ell}^A$ such that $[\![M_\ell]\!] = [\![D_{i,\ell}^A]\!](a)$. It follows that the closed term $C_i^A\, M_1 \cdots M_{k_i^A} : A$ has denotation $[\![C_i^A\, M_1 \cdots M_{k_i^A}]\!] = a$. Thus every element of $[\![A]\!]$ is definable. □

Remark 3.5.20. The proof shows that the finite syntactic presentation of the minimal model has a direct semantic counterpart. Loader's listing procedure classifies a term by its strictness behaviour and reconstructs it recursively from representatives at smaller types. The argument above performs the same decomposition on elements of the bistable interpretation itself. Thus, in the binary case, the syntactic and semantic approaches describe the same finite observational structure from opposite directions.

We now derive full abstraction from universality.

Lemma 3.5.21 (Ground adequacy)**.** *Let $M : o$ be closed. Then*

$$[\![M]\!] = \bot \iff M =_\beta \bot, \qquad [\![M]\!] = \top \iff M =_\beta \top.$$

Consequently, for closed $M, N : o$, $M \equiv_{\mathcal{C}_2} N \iff [\![M]\!] = [\![N]\!]$.

Proof. By strong normalisation, M has a β-normal form, which, since M is closed and of ground type, is either $\bot$ or $\top$. Soundness of the interpretation under β-conversion gives $M =_\beta \bot \Rightarrow [\![M]\!] = \bot$ and $M =_\beta \top \Rightarrow [\![M]\!] = \top$. Conversely, since $\bot$ and $\top$ have distinct denotations in Σ, a term denoting one cannot have the other as its normal form.

Observational equivalence at ground type is β-equality, so the final equivalence follows. □

Theorem 3.5.22 (Full abstraction)**.** *For all closed terms $M, N : A$, $M \equiv_{\mathcal{C}_2} N \iff [\![M]\!] = [\![N]\!]$.*

Proof. We proceed by induction on A. If $A = o$, the result is ground adequacy.

Suppose $A = B \to C$. For the left-to-right implication, assume $M \equiv_{\mathcal{C}_2} N$. We show that $[\![M]\!] = [\![N]\!]$ extensionally. Let $b \in [\![B]\!]$ be arbitrary. By universality, there is a closed term $P : B$ with $[\![P]\!] = b$. By observational equivalence at function type, $M\,P \equiv_{\mathcal{C}_2} N\,P$, so the induction hypothesis at C gives $[\![M\,P]\!] = [\![N\,P]\!]$. By compositionality, $[\![M]\!](b) = [\![M\,P]\!] = [\![N\,P]\!] = [\![N]\!](b)$. Since b was arbitrary, extensionality gives $[\![M]\!] = [\![N]\!]$.

Conversely, suppose $[\![M]\!] = [\![N]\!]$, and let $P : B$ be closed. By compositionality,

$$[\![M\,P]\!] = [\![M]\!]([\![P]\!]) = [\![N]\!]([\![P]\!]) = [\![N\,P]\!].$$

The induction hypothesis at C yields $M\,P \equiv_{\mathcal{C}_2} N\,P$. Since this holds for every closed $P : B$, observational equivalence at function type gives $M \equiv_{\mathcal{C}_2} N$. □

Remark 3.5.23. As an immediate consequence, at every type A, the map $[M]_{\equiv_{\mathcal{C}_2}} \longmapsto [\![M]\!]$ is injective by full abstraction and surjective by universality. It is therefore a bijection between the observational equivalence classes of closed terms and the elements of $[\![A]\!]$. In this sense, the bistable interpretation presents the minimal model of the two-constant simply typed λ-calculus.

Proposition 3.5.24 (Finiteness and effective presentation)**.** *For every type A, the bistable biorder $[\![A]\!]$ is finite and can be effectively constructed from A. Moreover, the denotation of a term $M : A$ is effectively computable.*

Proof. The claim follows by induction on types. The ground interpretation Σ is finite. If $[\![A]\!]$ and $[\![B]\!]$ are finite and effectively presented, then there are only finitely many functions $[\![A]\!] \to [\![B]\!]$, and monotonicity and bistability are decidable conditions on such functions. Hence $[\![A \to B]\!]$ is finite and effectively presentable. The interpretation of terms is then computed recursively from the cartesian closed operations. □

For the remainder of the chapter, we retain three properties of the binary interpretation established above: it is fully abstract, each type is interpreted by a finite effectively presentable bistable biorder, and term denotations are effectively computable. These properties make the binary model suitable as the basic component of the product interpretation constructed in the next section.

3.6 A fully abstract semantics for finite ground signatures

If the ground type is interpreted by Laird's two-element object $\Sigma = \{\bot, \top\}$, then a language with more than two distinct ground constants cannot be interpreted fully abstractly in this model. Indeed, two distinct constants must receive the same denotation, although they are contextually distinguishable already by the identity ground context. In this section we address this difficulty. Abramsky and Lenisa [1] previously constructed PER models for the maximal theory of simply typed λ-calculus with arbitrary finite ground signatures, in which all semantic elements are definable. Here we take a different route: we reduce finite ground observations to Laird's binary model. The resulting product construction and its full-abstraction argument are original to this dissertation.

Throughout this section we assume $|C| \geq 2$. When $|C| = 1$, every two closed terms of the same type are contextually equivalent, so the singleton case is trivial. For example, the terminal interpretation is fully abstract. Thus the results below, together with this degenerate case, cover every finite non-empty ground signature.

3.6.1 Obstructions to a direct extension

A natural attempt to extend Laird's interpretation to a ground signature $\mathcal{C} = \{a_1, \ldots, a_n\}$ with $n > 2$ is to take $D = \{a_1, \ldots, a_n\}$ as the underlying set of the ground object and equip it with a suitable extensional order and bistable coherence. The shrinking equations derived earlier impose strong necessary conditions on any such interpretation. Already the unary equation requires every element $h \in D \Rightarrow D$ to be idempotent.

This condition rules out two natural extreme choices of coherence. The proofs are given in Appendix B.

Lemma 3.6.1 (Failure for total coherence)**.** *Let $D = [\![o]\!]$ be a finite bistable biorder consisting of a single coherence class. If $|D| > 2$, then the induced interpretation in* **BBO** *is not fully abstract.*

Lemma 3.6.2 (Failure for discrete coherence)**.** *Let $D = [\![o]\!]$ be a bistable biorder with discrete coherence. If $|D| > 2$, then the induced interpretation in* **BBO** *is not fully abstract.*

In both cases the obstruction is the same: the function space $D \Rightarrow D$ contains a non-idempotent endomorphism, contradicting the unary shrinking equation. These results do not classify all possible ground biorders. In particular, mixed coherence remains open here. Rather than pursuing such a classification, we now take a different approach and represent the ground observations through a family of binary recolourings.

3.6.2 Reduction to binary observations

The product construction is based on the observation that distinctions between finitely many ground constants can be detected using only binary observations. If two terms are contextually distinguishable, some ground context evaluates them to distinct constants, which can be separated by a map to a two-element signature. Conversely, a binary distinguishing context can be lifted back to the original signature.

Definition 3.6.3 (Recolouring)**.** Let $r : \mathcal{C} \to \mathcal{D}$ be a map of ground signatures. The induced *recolouring* $r_* : \Lambda_{\mathcal{C}} \to \Lambda_{\mathcal{D}}$ is the homomorphic translation which replaces each ground constant $c \in \mathcal{C}$ by $r(c)$.

Let $\mathcal{C}_2 = \{\bot, \top\}$. For a finite ground signature $\mathcal{C}$ with $|\mathcal{C}| \geq 2$, write

$$P_{\mathcal{C}} = \{p : \mathcal{C} \twoheadrightarrow \mathcal{C}_2\}$$

for the set of surjective binary recolourings.

Lemma 3.6.4 (Compatibility of recolouring)**.** *Let $r : \mathcal{C} \to \mathcal{D}$, extended homomorphically to one-hole contexts. Then recolouring preserves typing and satisfies*

$$r_*(K[M]) = r_*(K)[r_*(M)], \qquad r_*(M[N/x]) = r_*(M)[r_*(N)/x].$$

Moreover, $M \to_\beta^* N$ implies $r_*(M) \to_\beta^* r_*(N)$. Hence, if $M : o$ is closed and $M \Downarrow c$, then $r_*(M) \Downarrow r(c)$. Finally, $(\mathrm{id}_\mathcal{C})_* = \mathrm{id}_{\Lambda_\mathcal{C}}$ and $(r \circ s)_* = r_* \circ s_*$ for every $s : \mathcal{B} \to \mathcal{C}$, with analogous equations for one-hole contexts.

Proof. Compatibility with context filling and substitution follows by structural induction. For reduction, it suffices to consider a β-redex:

$$r_*\big((\lambda x.M)N\big) = (\lambda x.r_*(M))\, r_*(N) \to_\beta r_*(M)[r_*(N)/x] = r_*(M[N/x]).$$

Closure under term contexts and induction on reduction sequences give preservation of $\to_\beta^*$. The statement about normal forms follows immediately. □

We can now express contextual equivalence over $\mathcal{C}$ entirely in terms of its binary recolourings.

Proposition 3.6.5 (Binary reduction of contextual equivalence)**.** *Let $M, N : A$ be closed terms over a finite ground signature $\mathcal{C}$, with $|\mathcal{C}| \geq 2$. Then*

$$M \equiv_\mathcal{C} N \quad \iff \quad p_*(M) \equiv_{\mathcal{C}_2} p_*(N) \text{ for every } p \in P_\mathcal{C}.$$

Proof. We prove both implications by contraposition.

Suppose $M \not\equiv_\mathcal{C} N$. Since observational and contextual equivalence coincide, there is a closed ground context $D[-]$ and distinct $c, d \in \mathcal{C}$ such that $D[M] \Downarrow c$ and $D[N] \Downarrow d$. Choose $p \in P_\mathcal{C}$ with $p(c) = \bot$ and $p(d) = \top$. Then

$$p_*(D)[p_*(M)] = p_*(D[M]) \Downarrow \bot, \qquad p_*(D)[p_*(N)] = p_*(D[N]) \Downarrow \top,$$

so $p_*(M) \not\equiv_{\mathcal{C}_2} p_*(N)$.

Conversely, suppose $p_*(M) \not\equiv_{\mathcal{C}_2} p_*(N)$ for some $p \in P_\mathcal{C}$. Then some closed ground context $E[-]$ over $\mathcal{C}_2$ distinguishes them. Since p is surjective, choose a section $s : \mathcal{C}_2 \to \mathcal{C}$ with $p \circ s = \mathrm{id}_{\mathcal{C}_2}$. If $s_*(E)[M] \Downarrow c$ and $s_*(E)[N] \Downarrow d$, then applying p_* gives

$$E[p_*(M)] \Downarrow p(c), \qquad E[p_*(N)] \Downarrow p(d).$$

Since $E[-]$ distinguishes $p_*(M)$ and $p_*(N)$, we have $p(c) \neq p(d)$, hence $c \neq d$. Thus $s_*(E)[-]$ distinguishes M and N, so $M \not\equiv_\mathcal{C} N$. □

Remark 3.6.6. Equivalently,

$$\equiv_{\mathcal{C}} = \bigcap_{p \in P_{\mathcal{C}}} p_*^{-1}(\equiv_{\mathcal{C}_2}).$$

Thus contextual equivalence over a finite ground signature is completely determined by its binary recolourings. This is the operational decomposition underlying the product interpretation of the next subsection.

3.6.3 The product interpretation

The binary reduction above suggests a corresponding semantic construction. Rather than representing all ground observations in a single bistable biorder, we interpret a term simultaneously in the binary Laird model for each recolouring $p \in P_{\mathcal{C}}$.

Definition 3.6.7 (Product interpretation)**.** Regard $P_{\mathcal{C}}$ as a discrete category and consider

$$[P_{\mathcal{C}}, \mathbf{BBO}] \cong \prod_{p \in P_{\mathcal{C}}} \mathbf{BBO}.$$

Since $P_{\mathcal{C}}$ is discrete, finite products and exponentials are computed componentwise.

We interpret types by

$$[\![o]\!]_{\mathcal{C}} = (\Sigma)_{p \in P_{\mathcal{C}}}, \qquad [\![A \to B]\!]_{\mathcal{C}} = [\![A]\!]_{\mathcal{C}} \Rightarrow [\![B]\!]_{\mathcal{C}},$$

where the exponential is computed componentwise. For $c \in \mathcal{C}$, define

$$([\![c]\!]_{\mathcal{C}})_p = [\![p(c)]\!]_{\mathrm{Laird}}.$$

Variables, abstraction, and application are interpreted by the usual cartesian closed clauses.

Example 3.6.8. Let $\mathcal{C} = \{a, b, c\}$. Every surjection $p : \mathcal{C} \twoheadrightarrow \{\bot, \top\}$ is determined by a non-empty proper subset $S \subsetneq \mathcal{C}$, by setting $p_S(x) = \top$ for $x \in S$ and $p_S(x) = \bot$ otherwise. Hence

$$P_{\mathcal{C}} = \{p_{\{a\}}, p_{\{b\}}, p_{\{c\}}, p_{\{a,b\}}, p_{\{a,c\}}, p_{\{b,c\}}\}.$$

Writing the coordinates in this order, the ground constants are interpreted as

$$[\![a]\!]_{\mathcal{C}} = (\top, \bot, \bot, \top, \top, \bot),$$
$$[\![b]\!]_{\mathcal{C}} = (\bot, \top, \bot, \top, \bot, \top),$$
$$[\![c]\!]_{\mathcal{C}} = (\bot, \bot, \top, \bot, \top, \top).$$

Thus each individual component collapses the ground signature to two values, while the family

of all components still distinguishes the original constants.

The construction is componentwise at every type.

Lemma 3.6.9. *For every type A, context Γ, and $p \in P_{\mathcal{C}}$,*

$$(\llbracket A \rrbracket_{\mathcal{C}})_p = \llbracket A \rrbracket_{\mathrm{Laird}}, \qquad (\llbracket \Gamma \rrbracket_{\mathcal{C}})_p = \llbracket \Gamma \rrbracket_{\mathrm{Laird}}.$$

Proof. By induction on A. The ground case is immediate. At function type, componentwise computation of exponentials gives

$$(\llbracket A \to B \rrbracket_{\mathcal{C}})_p = (\llbracket A \rrbracket_{\mathcal{C}})_p \Rightarrow (\llbracket B \rrbracket_{\mathcal{C}})_p = \llbracket A \rrbracket_{\mathrm{Laird}} \Rightarrow \llbracket B \rrbracket_{\mathrm{Laird}}.$$

The assertion for contexts follows similarly from componentwise computation of finite products. □

Lemma 3.6.10 (Componentwise interpretation)**.** *Let $\Gamma \vdash M : A$ be a typing judgement over $\mathcal{C}$. Then, for every $p \in P_{\mathcal{C}}$,*

$$(\llbracket M \rrbracket_{\mathcal{C}})_p = \llbracket p_*(M) \rrbracket_{\mathrm{Laird}}.$$

Proof. By induction on the typing derivation. For a ground constant $c \in \mathcal{C}$, the equation holds by definition. The variable, abstraction, and application cases follow because products, exponentials, currying, and evaluation in $[P_{\mathcal{C}}, \mathbf{BBO}]$ are computed componentwise. □

Thus the product interpretation may be written succinctly as

$$\llbracket M \rrbracket_{\mathcal{C}} = \left(\llbracket p_*(M) \rrbracket_{\mathrm{Laird}} \right)_{p \in P_{\mathcal{C}}}.$$

Combining this semantic decomposition with the binary reduction of contextual equivalence gives the main result.

Theorem 3.6.11 (Full abstraction for finite ground signatures)**.** *For all closed terms $M, N : A$,*

$$M \equiv_{\mathcal{C}} N \quad \Longleftrightarrow \quad \llbracket M \rrbracket_{\mathcal{C}} = \llbracket N \rrbracket_{\mathcal{C}}.$$

Proof. By the componentwise interpretation lemma, $\llbracket M \rrbracket_{\mathcal{C}} = \llbracket N \rrbracket_{\mathcal{C}}$ if and only if

$$\llbracket p_*(M) \rrbracket_{\mathrm{Laird}} = \llbracket p_*(N) \rrbracket_{\mathrm{Laird}}$$

for every $p \in P_{\mathcal{C}}$. By full abstraction of the binary Laird model, this is equivalent to $p_*(M) \equiv_{\mathcal{C}_2} p_*(N)$ for every $p \in P_{\mathcal{C}}$. The binary reduction proposition now gives $M \equiv_{\mathcal{C}} N$. □

Unlike the binary Laird model, the product interpretation is fully abstract without being universal.

Proposition 3.6.12 (Failure of universality)**.** *The product interpretation is not universal.*

Proof. The failure is already visible at ground type. The interpretation $[\![o]\!]_{\mathcal{C}}$ contains arbitrary families $(e_p)_{p \in P_{\mathcal{C}}}$ with $e_p \in \Sigma$, including the constant family $(\bot)_{p \in P_{\mathcal{C}}}$.

Every closed ground term is β-equivalent to some $c \in \mathcal{C}$, whose denotation is $(p(c))_{p \in P_{\mathcal{C}}}$. For every $c \in \mathcal{C}$, there is some $p \in P_{\mathcal{C}}$ with $p(c) = \top$. Hence no ground term denotes the constantly-$\bot$ family. □

Thus universality is not required for full abstraction: the product model contains non-definable elements, but they introduce no additional distinctions between definable terms. Since $P_{\mathcal{C}}$ is finite and each binary component is finite and effectively presentable, the product interpretation is also finite and effective at every type. We use this in the next section to obtain a semantic decision procedure for contextual equivalence.

3.7 Semantic decidability of contextual equivalence

The preceding full abstraction results have an immediate algorithmic consequence. In Chapter 2, contextual equivalence was decided by reducing terms to finitely many syntactic representatives of observable behaviour. Semantically, the analogous principle is that a fully abstract interpretation in which every type is finite and effectively presentable reduces contextual equivalence to decidable equality of denotations.

For Laird's binary model this already yields a semantic decision procedure.

Corollary 3.7.1 (Semantic decidability in the binary case)**.** *Contextual equivalence in $\Lambda_{\mathcal{C}_2}$ is decidable at every type.*

Proof. For every type A, the bistable biorder $[\![A]\!]_{\mathrm{Laird}}$ is finite and effectively presentable. In particular, at function types its elements may be represented by finite tables, and monotonicity and bistability are decidable conditions. Hence denotations of closed terms $M, N : A$ can be computed compositionally and compared effectively. By full abstraction,

$$M \equiv_{\mathcal{C}_2} N \quad \Longleftrightarrow \quad [\![M]\!]_{\mathrm{Laird}} = [\![N]\!]_{\mathrm{Laird}},$$

so contextual equivalence is decidable. □

The product interpretation extends this argument from the binary calculus to arbitrary finite ground signatures. Abramsky and Lenisa [1] raised the question of obtaining a semantic proof

of decidability, as an alternative to the syntactic approaches of Padovani and Loader, observing that a sufficiently effective semantics in which all semantic elements are definable should support such an argument. To the best of our knowledge, an explicit denotational decision procedure for contextual equivalence over arbitrary finite ground signatures has not previously been given.

Theorem 3.7.2 (Semantic decidability for finite ground signatures)**.** *Let $\mathcal{C}$ be a finite ground signature with $|\mathcal{C}| \geq 2$. Then contextual equivalence in $\Lambda_{\mathcal{C}}$ is decidable at every type.*

Proof. Since $\mathcal{C}$ is finite, so is the set $P_{\mathcal{C}}$ of surjections $p : \mathcal{C} \twoheadrightarrow \mathcal{C}_2$. For every type A, each component of $[\![A]\!]_{\mathcal{C}}$ is the finite and effectively presentable bistable biorder $[\![A]\!]_{\mathrm{Laird}}$. Hence the product $[\![A]\!]_{\mathcal{C}}$ is itself finite and effectively presentable.

Given closed terms $M, N : A$, their denotations can therefore be computed componentwise and compared effectively. By full abstraction of the product interpretation,

$$M \equiv_{\mathcal{C}} N \quad \Longleftrightarrow \quad [\![M]\!]_{\mathcal{C}} = [\![N]\!]_{\mathcal{C}},$$

and the right-hand side is decidable. □

Thus the syntactic and semantic procedures of Chapters 2 and 3 give two finite effective methods for deciding equality in the same observational quotient. Padovani represents the quotient directly by finitely many syntactic representatives, whereas the product semantics embeds it faithfully into a finite semantic space. The latter therefore gives a genuinely denotational route to decidability for finite ground signatures with at least two constants.

Chapter 4

Beyond ground-only signatures

The preceding chapters studied contextual equivalence for simply typed λ-calculi whose primitive constants all have ground type. In that setting, both syntactic and semantic methods led to finite, effective descriptions of observable behaviour. It is natural to ask how robust these phenomena are once the language is enriched with higher-type primitives and additional operational equations.

This chapter investigates that question through two closely related extensions. We first introduce a general framework of typed signatures and equational theories, which allows primitive operations of arbitrary simple type. We then study unary PCF and Boolean PCF as contrasting examples. Unary PCF retains strong sequentiality properties: its stable model is universal, finite at every type, and admits the structural reductions that will be exploited in the next chapter. Boolean PCF, by contrast, shows that pointwise finiteness alone does not imply uniform effective access to contextual equivalence.

The comparison identifies a boundary which will become central in Chapter 5. The issue is not merely whether each type has finitely many observable behaviours, but whether higher-order behaviour can be compressed uniformly and effectively into simpler types. Retractions provide the language for formulating this distinction.

4.1 Typed signatures and equational theories

Let $\mathcal{G}$ be a set of ground types, and let $\mathcal{T}$ be the set of simple types generated by

$$\tau ::= \iota \mid \tau \to \tau, \qquad \iota \in \mathcal{G}.$$

Definition 4.1.1. A *signature* over $\mathcal{G}$ is a family $\mathcal{S} = (\mathcal{S}_\tau)_{\tau \in \mathcal{T}}$, where each $c \in \mathcal{S}_\tau$ is a constant of type τ. We write $\Lambda_\mathcal{S}$ for the simply typed λ-calculus generated by variables and the constants in $\mathcal{S}$.

Definition 4.1.2. An *equational theory* over $\mathcal{S}$ is a well-typed congruence on $\Lambda_{\mathcal{S}}$ closed under substitution. Given a set E of well-typed equations, write $=_E$ for the least equational theory containing $\beta\eta$-equality and the equations in E.

In the examples below the behaviour of primitive constants will often be specified by reduction rules rather than equations. In that case, $=_E$ denotes the least congruence containing $\beta\eta$-equality and the equations induced by those reductions. This syntactic theory should be distinguished from contextual equivalence, which may identify additional terms with the same observable behaviour.

4.2 Unary PCF and its models

4.2.1 The language and observational equivalence

Unary PCF, denoted PCF_1, is the simply typed λ-calculus over a single ground type o with signature

$$\mathcal{S}_1 = \{\top, \bot : o,\ \wedge : o \to o \to o\}.$$

We write $M \wedge N$ for $\wedge\, M\, N$. Its operational semantics is generated by β-reduction together with the ground rewrite rules

$$\top \wedge M \longrightarrow M, \qquad \bot \wedge M \longrightarrow \bot.$$

Thus $\wedge$ is sequential in its first argument.

For a closed term $M : o$, write $M \Downarrow$ if $M \longrightarrow^* \top$. Thus the sole observable ground behaviour is convergence to $\top$, while $\bot$ represents non-observation.

For closed terms $M, N : A$, define the *observational preorder* $M \preceq_{\mathrm{obs}} N$ by

$$M \preceq_{\mathrm{obs}} N \quad \Longleftrightarrow \quad C[M] \Downarrow \Longrightarrow C[N] \Downarrow$$

for every closing context $C[-]$ such that $C[M]$ and $C[N]$ have type o. Its symmetric part is observational equivalence:

$$M \equiv_{\mathrm{obs}} N \quad \Longleftrightarrow \quad M \preceq_{\mathrm{obs}} N \text{ and } N \preceq_{\mathrm{obs}} M.$$

Since this relation is defined by quantification over program contexts, it is precisely contextual equivalence for the chosen ground observation. We therefore write $M \simeq_{\mathrm{ctx}} N$ for $M \equiv_{\mathrm{obs}} N$ from now on.

It is useful to contrast the contextual definition with the corresponding applicative preorder.

For closed terms, define $\preceq_{\mathrm{app}}$ inductively by

$$M \preceq_{\mathrm{app}} N : o \quad \Longleftrightarrow \quad M \Downarrow \Longrightarrow N \Downarrow,$$

and

$$M \preceq_{\mathrm{app}} N : A \to B \quad \Longleftrightarrow \quad MP \preceq_{\mathrm{app}} NP \text{ for every closed } P : A.$$

Lemma 4.2.1 (Context Lemma)**.** *For all closed unary PCF terms $M, N : A$,*

$$M \preceq_{\mathrm{obs}} N \quad \Longleftrightarrow \quad M \preceq_{\mathrm{app}} N.$$

Consequently, $M \simeq_{\mathrm{ctx}} N$ if and only if no sequence of closed arguments reducing M and N to ground type distinguishes their convergence behaviour.

Proof. Jim and Meyer prove the Context Lemma for simply typed λ-calculi whose operational semantics consists of β-reduction together with linear ground rewrite rules [6]. The two rules defining $\wedge$ above are of this form, so unary PCF is a PCF-like rewrite system in their sense and their result applies. □

Although η-reduction is not part of this operational rewrite system, the η-law is contextually valid. If $x \notin FV(M)$, then for every closed $P : A$, $(\lambda x^A.Mx)P =_\beta MP$. Hence $\lambda x^A.Mx$ and M are applicatively equivalent and therefore contextually equivalent by the Context Lemma. Thus the equational theory generated by $\beta\eta$ together with the defining equations for $\wedge$ is sound for contextual equivalence.

The Context Lemma also explains the extensional behaviour of sequential conjunction. Although its defining rules are asymmetric, modulo contextual equivalence $\wedge$ is the meet operation on the two observational ground values:

$$\top \wedge M \simeq_{\mathrm{ctx}} M \wedge \top \simeq_{\mathrm{ctx}} M, \qquad \bot \wedge M \simeq_{\mathrm{ctx}} M \wedge \bot \simeq_{\mathrm{ctx}} \bot.$$

Hence Laird's extensional presentation of unary PCF by the induced equations for $\wedge$ agrees, at the level of contextual equivalence, with the operational presentation used here.

We study unary PCF from three semantic viewpoints. The Scott model first shows what happens when all monotone behaviour is admitted: parallel composition appears. We then recall Laird's abstract classification of order-extensional models, in which the presence or absence of parallel composition determines universality. Finally, we instantiate the sequential branch with the stable-biorder model and examine its finite structure explicitly.

4.2.2 The Scott model and parallel behaviour

We first recall the standard Scott interpretation, specialised to unary PCF. The presentation and its use to contrast sequential and parallel behaviour follow Laird's discussion in [9]. The underlying cpo semantics is the classical Scott model of PCF.

The Scott model of unary PCF interprets types by cpos and Scott-continuous functions. Since all type interpretations arising here are finite, continuity is automatic, so it is enough to work with finite posets and monotone functions.

The ground type is interpreted by the two-element chain $[\![o]\!] = \Sigma = \{\bot \sqsubseteq \top\}$. Function types are interpreted by the poset of monotone functions $[\![\sigma \to \tau]\!] = [[\![\sigma]\!] \to [\![\tau]\!]]$, ordered pointwise: $f \sqsubseteq g$ iff $f(x) \sqsubseteq g(x)$ for every $x \in [\![\sigma]\!]$. All type interpretations are finite by induction.

The constants $\bot$ and $\top$ are interpreted by the corresponding points of Σ, while sequential conjunction is interpreted by binary meet on Σ. Thus the operational rules $\top \wedge M \longrightarrow M$ and $\bot \wedge M \longrightarrow \bot$ are sound. Extensionally, meet is commutative, so the stronger equations $\top \wedge x = x \wedge \top = x$ and $\bot \wedge x = x \wedge \bot = \bot$ also hold in the model.

The Scott model contains more functions than are definable in unary PCF. The basic example is *parallel disjunction*, the binary join $\vee_\Sigma : \Sigma \times \Sigma \to \Sigma$, characterised by

$$\bot \vee_\Sigma \bot = \bot, \qquad \bot \vee_\Sigma \top = \top \vee_\Sigma \bot = \top \vee_\Sigma \top = \top.$$

This map is monotone, and hence belongs to $[\![o \to o \to o]\!]$, but it is not definable in unary PCF. Intuitively, either argument may force convergence, so neither argument is an unavoidable sequential dependency.

Consequently, the Scott model is not universal for unary PCF. It may instead be viewed as a model of the larger language obtained by adjoining parallel disjunction. In fact Laird proves that the Scott model of $\Lambda^{\top}_{\bot}(\wedge, \vee)$ is universal [8]. Together with soundness, universality yields full abstraction. Thus the Scott model is fully abstract for unary PCF with parallel composition, although it is not fully abstract for ordinary unary PCF. This contrast motivates the stable-biorder model introduced below: both models have the same extensional ground order, but the Scott model admits all monotone maps, whereas the stable model restricts the function spaces by an additional stability condition.

4.2.3 Standard order-extensional models

We now introduce the abstract setting in which Laird's universality result for unary PCF is formulated [9].

Definition 4.2.2. A *standard model* of unary PCF consists of a cartesian closed category, an

object Σ interpreting o, points $\bot, \top : 1 \to \Sigma$, and a morphism $\wedge : \Sigma \times \Sigma \to \Sigma$ satisfying

$$\bot \wedge x = x \wedge \bot = \bot, \qquad \top \wedge x = x \wedge \top = x.$$

These are the extensional equations for sequential conjunction. As observed above, they hold modulo contextual equivalence in the operational presentation of unary PCF.

Every standard model carries an intrinsic relation on its points.

Definition 4.2.3. For a standard model $\mathcal{M}$, define $\preceq_A$ inductively by

$$e \preceq_o e' \quad \Longleftrightarrow \quad e = \bot \text{ or } e' = \top,$$

$$f \preceq_{A \to B} g \quad \Longleftrightarrow \quad e \preceq_A e' \Longrightarrow f(e) \preceq_B g(e')$$

for all points e, e' of type A. The model is *order-extensional* if $\preceq_A$ is a partial order at every type A.

In an order-extensional model we write $\sqsubseteq$ for this intrinsic order. Every type then has least and greatest points, again denoted $\bot$ and $\top$.

The distinction between sequential and parallel behaviour can be stated directly at ground type.

Definition 4.2.4. A standard model *contains parallel composition* if it contains a morphism $\vee : \Sigma \times \Sigma \to \Sigma$ satisfying

$$\bot \vee \bot = \bot, \qquad \bot \vee \top = \top \vee \bot = \top \vee \top = \top.$$

Thus the Scott model contains parallel composition, witnessed by $\vee_\Sigma$. Laird proves that, for order-extensional models, the absence of this operation is already sufficient to force universality.

Theorem 4.2.5 (Laird)**.** *Every standard order-extensional model of unary PCF which does not contain parallel composition is universal.*

This theorem will be applied below to the stable-biorder model. The retraction-based proof underlying it will be revisited in the next chapter.

4.2.4 The stable-biorder model

The Scott model admits all monotone maps, including parallel disjunction. The stable-biorder model restricts its function spaces by equipping each type with a second order which records stable, or sequential, dependence. We follow Laird's presentation of stable biorders in the definitions below [9] with small notational changes.

Definition 4.2.6. A *stable biorder* is a set D equipped with an *extensional order* $\sqsubseteq$ and a *stable order* $\leq$ such that:

1. $(D, \sqsubseteq)$ is a meet-semilattice, with meet $x \sqcap y$;

2. $x \leq y$ implies $x \sqsubseteq y$;

3. $\sqcap$ is monotone with respect to the stable order.

Write $x \uparrow_s y$ when x and y have a common upper bound in the stable order. In this case $x \sqcap y$ is also their greatest lower bound with respect to the stable order.

Definition 4.2.7. A function $f : D \to E$ between stable biorders is *monotone and stable* if it is monotone with respect to both orders and, whenever $x \uparrow_s y$,

$$f(x \sqcap y) = f(x) \sqcap f(y).$$

For stable biorders D and E, let $D \Rightarrow E$ be the set of monotone and stable functions $D \to E$. Its extensional order is pointwise:

$$f \sqsubseteq g \quad \iff \quad f(x) \sqsubseteq g(x) \text{ for every } x \in D,$$

while its stable order is given by

$$f \leq g \quad \iff \quad x \leq y \Longrightarrow f(x) = f(y) \sqcap g(x) \text{ for all } x, y \in D.$$

Proposition 4.2.8. *Stable biorders and monotone stable functions form a cartesian closed category, denoted* **SBi***, with products defined componentwise and exponentials given by* $D \Rightarrow E$.

Definition 4.2.9 (Stable-biorder model)**.** The stable-biorder model of unary PCF interprets the ground type by

$$D_o = \Sigma = \{\bot, \top\}, \qquad \bot \sqsubseteq \top, \qquad \bot \leq \top,$$

and function types by $D_{A \to B} = D_A \Rightarrow D_B$. The constants $\bot$ and $\top$ are interpreted by the corresponding points of Σ, and $\wedge$ by binary meet on Σ.

The stable-biorder model is standard and order-extensional. Its intrinsic order agrees with the extensional order $\sqsubseteq$: this is immediate at ground type and follows inductively at function types from the pointwise definition of the extensional order.

Lemma 4.2.10 (Observational soundness)**.** *The stable-biorder model of unary PCF is observationally sound.*

Proof. Suppose that $M, N : A$ are closed and $[\![M]\!] = [\![N]\!]$. Let $C[-] : A \Rightarrow o$ be any closed ground context. By compositionality, $[\![C[M]]\!] = [\![C[N]]\!]$.

Every closed ground term of unary PCF has observable result either $\bot$ or $\top$, and these are interpreted by distinct elements of $\Sigma = \{\bot, \top\}$. Hence $C[M]$ and $C[N]$ cannot have different observable results. Since $C[-]$ was arbitrary, $M \simeq_{\mathrm{ctx}} N$. □

4.2.5 Universality of the stable model

We now show that the stable-biorder model is universal. The key structural property is *boundedness*, which forces monotone stable maps to behave sequentially.

Definition 4.2.11. A stable biorder D is *pointed* if it has a least element $\bot_D$ with respect to the stable order. Since the stable order refines the extensional order, $\bot_D$ is also extensionally least.

A pointed stable biorder is *bounded* if it has a greatest element $\top_D$ with respect to the extensional order.

Lemma 4.2.12. *For every unary PCF type A, the stable biorder D_A is pointed and bounded.*

Proof. By induction on A. At ground type, $D_o = \Sigma = \{\bot, \top\}$, so the result is immediate.

Suppose D_A and D_B are pointed and bounded. The constant function $\lambda x.\bot_{D_B}$ is the least element of $D_A \Rightarrow D_B$ with respect to the stable order, while $\lambda x.\top_{D_B}$ is its greatest element with respect to the extensional order. Hence $D_A \Rightarrow D_B$ is pointed and bounded. □

Every unary PCF type has a unique curried form $A_1 \to \cdots \to A_n \to o$. This allows sequential dependence to be defined uniformly at all types.

Definition 4.2.13. Let $f : D_{A_1} \times \cdots \times D_{A_n} \to \Sigma$. The map f is *strict* if $f(\bot, \ldots, \bot) = \bot$, and is *$i$-strict* if

$$f(x_1, \ldots, x_{i-1}, \bot, x_{i+1}, \ldots, x_n) = \bot$$

for every choice of the remaining arguments.

The stable-biorder model is *sequential* if every element of every type, regarded in this curried form, is either constant or i-strict for some i.

If f is i-strict, then its i-th argument is an unavoidable sequential dependency: divergence of that argument forces the whole computation to diverge. Boundedness is precisely the additional condition which forces stable maps to have such a dependency.

Lemma 4.2.14 (Laird)**.** *Let $D_1, \ldots, D_n$ be bounded stable biorders. If $f : D_1 \times \cdots \times D_n \to \Sigma$ is monotone, stable, and strict, then f is i-strict for some $i \in \{1, \ldots, n\}$.*

Proposition 4.2.15. *The stable-biorder model of unary PCF is sequential.*

Proof. Let $f \in D_{A_1 \to \cdots \to A_n \to o}$. By Cartesian closure, we may regard f as a monotone stable map

$$f : D_{A_1} \times \cdots \times D_{A_n} \to \Sigma.$$

Each D_{A_i} is bounded by the preceding lemma.

If $f(\bot, \ldots, \bot) = \top$, then extensional monotonicity implies $f(x_1, \ldots, x_n) = \top$ for every tuple, so f is constant. Otherwise $f(\bot, \ldots, \bot) = \bot$, so f is strict, and Laird's lemma gives an i for which f is i-strict. Hence every element is either constant or i-strict. □

In particular, the stable-biorder model cannot contain parallel composition. Indeed, parallel disjunction $\vee_\Sigma : \Sigma \times \Sigma \to \Sigma$ is neither constant nor 1-strict nor 2-strict, since $\bot \vee_\Sigma \top = \top$ and $\top \vee_\Sigma \bot = \top$.

Theorem 4.2.16 (Laird)**.** *The stable-biorder model of unary PCF is universal.*

Proof. The stable-biorder model is standard and order-extensional, as observed above. By the preceding proposition it is sequential, and therefore does not contain parallel composition. Laird's universality theorem for standard order-extensional models without parallel composition therefore applies. Hence every element of D_A is denoted by a closed unary PCF term of type A. □

4.2.6 Failure of full abstraction of the Scott model

The contrast between the stable and Scott models is not merely a failure of universality. The additional parallel elements of the Scott model can distinguish terms which no unary PCF context can distinguish.

Example 4.2.17. Consider the closed terms

$$F_1 = \lambda f^{o \to o \to o}.f\,(f \perp \top)\,(f \top \perp), \qquad F_2 = \lambda f^{o \to o \to o}.f \perp \perp.$$

Then $F_1 \simeq_{\mathrm{ctx}} F_2$ in unary PCF.

Indeed, by the Context Lemma it is enough to consider closed unary PCF arguments $f : o \to o \to o$. Such arguments denote sequential elements: they are either constant or strict in one of their arguments. A direct case analysis therefore shows that $F_1 f$ and $F_2 f$ have the same observable behaviour.

In the Scott model, however, we may apply the denotations of F_1 and F_2 to parallel disjunction $\vee_\Sigma$. Since

$$\bot \vee_\Sigma \top = \top \vee_\Sigma \bot = \top, \qquad \bot \vee_\Sigma \bot = \bot,$$

we obtain

$$[\![F_1]\!](\vee_\Sigma) = \top, \qquad [\![F_2]\!](\vee_\Sigma) = \bot.$$

Hence $[\![F_1]\!] \neq [\![F_2]\!]$ in the Scott model, although $F_1 \simeq_{\mathrm{ctx}} F_2$. Therefore the Scott model is not fully abstract for unary PCF.

4.2.7 Concrete structure at first-order types

The stable-biorder interpretation is finite at every unary PCF type.

Lemma 4.2.18. *For every unary PCF type A, the stable biorder D_A is finite.*

Proof. The ground biorder Σ is finite. If D_A and D_B are finite, then there are only finitely many functions $D_A \to D_B$, and hence only finitely many monotone stable functions. Thus $D_A \Rightarrow D_B$ is finite. □

For later use, we derive explicitly the first-order structure of this model. Let

$$\phi_n = \underbrace{o \to \cdots \to o}_{n \text{ arguments}} \to o, \qquad n \geq 0.$$

For each $S \subseteq \{1, \ldots, n\}$, define $c_S(x_1, \ldots, x_n) = \bigwedge_{i \in S} x_i$, with $c_\varnothing = \top$.

Proposition 4.2.19. *The elements of D_{ϕ_n} are precisely*

$$D_{\phi_n} = \{\bot\} \cup \{c_S \mid S \subseteq \{1, \ldots, n\}\}.$$

Hence $|D_{\phi_n}| = 2^n + 1$.

Moreover,

$$c_S \sqsubseteq c_T \quad \Longleftrightarrow \quad T \subseteq S,$$

while the stable order is flat above $\bot$:

$$\bot \leq c_S \quad \textit{for every } S, \qquad c_S \leq c_T \quad \Longleftrightarrow \quad S = T.$$

Proof. An element of D_{ϕ_n} is a monotone stable map $f : \Sigma^n \to \Sigma$. Its truth set $U_f = \{\vec{x} \mid f(\vec{x}) = \top\}$ is upward closed and closed under meets. If U_f is nonempty, finiteness implies that it has a least element m, and therefore $U_f = {\uparrow} m$. Such a principal upset is determined by the coordinates of m which are equal to $\top$, and hence is the truth set of a unique c_S. The empty truth set gives the constant $\bot$ map.

The description of the extensional order follows from inclusion of truth sets. Since every nonempty truth set contains $(\top, \ldots, \top)$, the defining condition for the stable order implies that distinct non-bottom elements are stably incomparable. □

In particular,

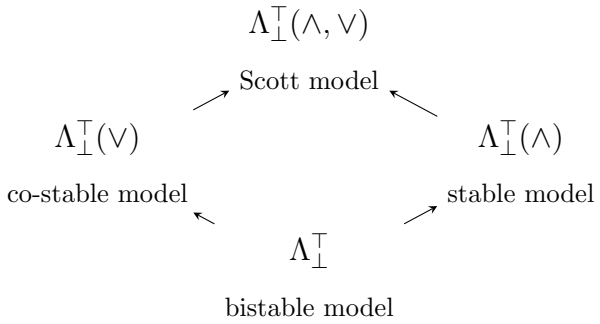
$$D_{o\to o} = \{c_\bot, \mathrm{id}, c_\top\},$$

where $c_\bot$ and $c_\top$ are the two constant functions. This three-element function space will reappear in the next chapter as the basis type for the retraction analysis of unary PCF.

4.3 A hierarchy of two-point calculi and models

The models considered above form part of a small hierarchy of two-point calculi studied by Laird. Starting from $\Lambda^\top_\bot$, the simply typed λ-calculus over one ground type o with constants $\bot, \top : o$, one may adjoin sequential conjunction $\wedge$, its order-dual $\vee$, or both.

At ground type, $\wedge$ is meet and $\vee$ is join on $\Sigma = \{\bot \sqsubseteq \top\}$. Thus $\Lambda^\top_\bot(\wedge)$ is the extensional presentation of unary PCF considered above, while $\Lambda^\top_\bot(\vee)$ is its dual. The four calculi and their corresponding universal models are arranged as follows:



Unary PCF occupies the right-hand branch, whose universal model is the stable-biorder model developed above. Dually, $\Lambda^\top_\bot(\vee)$ has a co-stable model. When both $\wedge$ and $\vee$ are available, the corresponding universal model is the Scott model, whose function spaces contain all monotone maps. At the bottom, $\Lambda^\top_\bot$ has the bistable-biorder model as its universal model.

The hierarchy therefore relates increasing expressive power in the language to progressively weaker semantic restrictions on function spaces. Bistability sits at the symmetric base; adjoining $\wedge$ selects the stable branch, adjoining $\vee$ selects its dual, and adjoining both removes the stability restriction altogether and yields the Scott model.

This hierarchy will reappear in the next chapter from a different perspective: adjoining parallel composition not only enlarges the universal model, but also changes how far higher-order types can be compressed by definable retractions.

4.4 Boolean PCF

The favourable situation for unary PCF changes sharply when the ground type has two distinct terminating values. Boolean PCF provides the simplest example: Laird's bounded-biorder argu-

ment no longer applies, and Loader's undecidability theorem gives a much stronger obstruction to effective fully abstract semantics.

4.4.1 The language and observational equivalence

Boolean PCF, denoted PCF_2, is the simply typed λ-calculus over a single ground type $\mathbb{B}$ with constants

$$\mathsf{tt}, \mathsf{ff}, \bot : \mathbb{B}$$

and a conditional

$$\mathsf{if} : \mathbb{B} \to \mathbb{B} \to \mathbb{B} \to \mathbb{B}.$$

We write $\mathsf{if}\ M\ \mathsf{then}\ N\ \mathsf{else}\ P$ for $\mathsf{if}\, M\, N\, P$.

Besides the usual $\beta\eta$-conversions, the conditional satisfies

$$\mathsf{if}\ \mathsf{tt}\ \mathsf{then}\ M\ \mathsf{else}\ N \longrightarrow M, \qquad \mathsf{if}\ \mathsf{ff}\ \mathsf{then}\ M\ \mathsf{else}\ N \longrightarrow N,$$

and

$$\mathsf{if}\ \bot\ \mathsf{then}\ M\ \mathsf{else}\ N \longrightarrow \bot,$$

together with the commuting conversion

$$\begin{aligned}&\mathsf{if}\,(\mathsf{if}\ M\ \mathsf{then}\ N\ \mathsf{else}\ P)\,\mathsf{then}\ Q\ \mathsf{else}\ R\\ &\quad\longrightarrow \mathsf{if}\ M\ \mathsf{then}\,(\mathsf{if}\ N\ \mathsf{then}\ Q\ \mathsf{else}\ R)\,\mathsf{else}\,(\mathsf{if}\ P\ \mathsf{then}\ Q\ \mathsf{else}\ R).\end{aligned}$$

We write $=_{E_2}$ for the least congruence containing these conversions.

Loader presents if as a ternary ground-type construct rather than as a curried constant [15]. The two presentations are equivalent for the purposes considered here, since partial applications of the curried constant may be η-expanded.

Loader defines the observational preorder as a logical relation. At ground type it is the flat order

$$\bot \preceq_{\mathrm{obs}} \mathsf{tt}, \qquad \bot \preceq_{\mathrm{obs}} \mathsf{ff},$$

with tt and ff incomparable. At function type, for closed terms $F, G : A \to B$,

$$F \preceq_{\mathrm{obs}} G \quad\Longleftrightarrow\quad FM \preceq_{\mathrm{obs}} GN \text{ whenever } M \preceq_{\mathrm{obs}} N.$$

By the logical-relations lemma this is equivalently characterised by

$$F \preceq_{\mathrm{obs}} G \quad\Longleftrightarrow\quad FM \preceq_{\mathrm{obs}} GM \text{ for every closed } M : A,$$

and the resulting relation coincides with the usual contextual preorder [15]. Its symmetric part is therefore contextual equivalence, which we again write as $\simeq_{\mathrm{ctx}}$.

4.4.2 Failure of boundedness

The first contrast with unary PCF is structural. The natural extensional order on the Boolean ground type is

$$D_{\mathbb{B}} = \{\bot, \mathsf{tt}, \mathsf{ff}\}, \qquad \bot \sqsubseteq \mathsf{tt}, \quad \bot \sqsubseteq \mathsf{ff},$$

with tt and ff incomparable. In particular, $D_{\mathbb{B}}$ has no greatest element and is therefore not bounded.

Thus the hypothesis which forced monotone stable maps to be sequential in unary PCF is already lost at ground type. Laird's bounded-biorder argument for universality consequently does not extend directly to Boolean PCF. This observation alone does not rule out other fully abstract models; a substantially stronger obstruction is provided by undecidability.

4.4.3 Undecidability and effective semantics

Theorem 4.4.1 (Loader, [15])**.** *Contextual equivalence in* PCF_2 *is undecidable.*

Loader also locates the onset of undecidability in the type hierarchy. His convention assigns order 1 to the ground type, whereas we use $\mathrm{ord}(\mathbb{B}) = 0$. In Loader's convention, contextual equivalence is decidable at types of order at most 4, while undecidability occurs already at order 5. Thus, under our convention, contextual equivalence is decidable at all types of order at most 3, but is undecidable at order 4 [15].

Loader in fact proves undecidability of both the observational preorder and its induced equivalence. Since observational and contextual equivalence coincide, this immediately constrains how effectively a fully abstract model of Boolean PCF can be presented.

Corollary 4.4.2. *There is no fully abstract model of* PCF_2 *for which both term interpretation is computable and equality of semantic elements is uniformly decidable in the type.*

Proof. Otherwise, given closed terms $M, N : A$, we could compute $[\![M]\!]$ and $[\![N]\!]$ and decide whether they are equal. Full abstraction would give

$$M \simeq_{\mathrm{ctx}} N \quad \Longleftrightarrow \quad [\![M]\!] = [\![N]\!],$$

yielding a decision procedure for contextual equivalence and contradicting Loader's theorem. □

An important feature of Loader's result is that undecidability does not arise from infinite observational quotients.

Lemma 4.4.3 (Loader, [15])**.** *At every* PCF_2 *type there are only finitely many contextual equivalence classes of closed terms.*

Thus finiteness at each fixed type does not imply effective presentability uniformly in the type. Loader's theorem shows that these finite quotients cannot be uniformly presented in a way that makes contextual equivalence decidable. This distinction between *pointwise finiteness* and *uniform effectiveness* will be important in the next chapter.

4.4.4 The stable-biorder interpretation

There is a natural stable-biorder interpretation of Boolean PCF obtained by taking

$$D_{\mathbb{B}} = \{\bot, \mathsf{tt}, \mathsf{ff}\}$$

with both orders given by the flat order above, interpreting function types by stable-biorder exponentials, and interpreting the conditional by

$$\mathsf{if}_{\mathbb{B}}(\bot, x, y) = \bot, \qquad \mathsf{if}_{\mathbb{B}}(\mathsf{tt}, x, y) = x, \qquad \mathsf{if}_{\mathbb{B}}(\mathsf{ff}, x, y) = y.$$

The conditional is monotone and stable, so this gives a sound interpretation of PCF_2.

Corollary 4.4.4. *The finite stable-biorder interpretation of* PCF_2 *is not fully abstract.*

Proof. The interpretation is effectively constructible from the type. Indeed, $D_{\mathbb{B}}$ is finite, and once finite biorders D_A and D_B have been constructed, the functions $D_A \to D_B$ can be enumerated and monotonicity and stability checked effectively. Hence $D_{A\to B}$ and its two orders are uniformly computable.

Term denotations are therefore computable and equality of denotations is uniformly decidable. Full abstraction would contradict Loader's undecidability theorem by the preceding corollary. □

The examples in this chapter expose two distinct questions which were largely conflated in the finite-ground setting. First, does each type admit only finitely much observable behaviour? Second, can that behaviour be accessed uniformly and effectively through simpler types?

Unary PCF answers both questions positively. Stability together with boundedness forces sequential behaviour, yielding a universal finite model and strong structural reductions between types. Boolean PCF shows that the two questions come apart: every type still has only finitely many observational classes, but Loader's theorem rules out a uniform effective presentation capable of deciding contextual equivalence.

The next chapter formulates this distinction in terms of definable retractions. Retraction bases will provide a general notion of higher-order compression, allowing the positive results for finite ground signatures and unary PCF to be treated uniformly, while Boolean PCF will supply an obstruction to effective compression.

Chapter 5

Retractions

In our considerations so far, types have played an important technical role but have remained largely in the background. In this chapter, however, they become an object of interest in their own right. Even in the simple setting considered here, there are interesting relations between types. The main idea is that, given a suitable pair of types, one can construct an information-preserving encoding of one type into another by means of a so-called *definable retraction*. Definable retractions have been used in denotational semantics to reduce questions at complicated higher-order types to corresponding questions at simpler types [10, 9]. In particular, such reductions underlie universality and full-abstraction arguments for sequential models [10, 9], while closely related definable embeddings have also been used in decidability proofs [11]. The aim of this chapter is to isolate this methodology from its particular semantic applications and develop it into a general framework for simply typed, arrow-only languages.

The central idea is to regard a retraction as a form of definable information-preserving compression. Because the languages considered here do not contain product types, ordinary retractions into a single simple type are in general too restrictive. We therefore introduce finite-component retractions, in which the information contained in a value of type T is distributed among finitely many simpler types and can be reconstructed by a definable decoder. We show that these are exactly ordinary retractions into finite products when products are admitted as a conservative metatheoretical device.

This leads to the notion of a retraction basis: a family of types into finite collections of which every type can be definably compressed. A general basis criterion separates the construction into a uniform structural argument, common to all simply typed languages considered here, and a language-specific compression problem for function spaces between products of basis types. When the compression is effective, decidability of contextual equivalence on the basis lifts to decidability at all types; similarly, full abstraction and universality can be transferred from the basis to arbitrary types.

The framework is instantiated for several languages. For the simply typed lambda calculus over a finite ground signature we obtain an effective basis of first-order types and thereby a decision procedure for its maximal theory. Laird's retractions for unary PCF [9] fit the same framework and yield an effective basis consisting of the single low-order type $o \to o$. With parallel composition, compression becomes stronger still: the ground type itself forms an effective basis. Finally, Loader's undecidability theorem for Boolean PCF [15] yields a converse obstruction: an undecidable language cannot admit an effective retraction basis contained in a fragment with uniformly decidable contextual equivalence. Thus the framework accounts both for successful reductions of higher-order equivalence and for principled limitations on such reductions.

5.1 Definable retractions

Definition 5.1.1. Given types S and T, a *definable retraction* $S \trianglelefteq T$ in a model $\mathcal{M}$ consists of closed terms $\mathsf{C} : S \to T$ and $\mathsf{D} : T \to S$ such that $[\![\mathsf{C}]\!]; [\![\mathsf{D}]\!] = \mathrm{id}_{[\![S]\!]}$, where ; denotes diagrammatic composition.

We refer to C as the *coder* and to D as the *decoder*. Thus C gives a definable information-preserving encoding of S into T, while D reconstructs the original element. The notion is model-dependent: the same pair of terms may witness a retraction in one model but not in another.

Definable retractions are useful because properties established at the target type can often be transferred back to the retract. This principle has been used in arguments for universality and full abstraction, while closely related definable embeddings have also been used to transfer decidability results [10, 11, 9]. The corresponding transfer principles will be formulated below in the more general setting needed in this chapter.

A representative example is Laird's treatment of the bistable model of $\Lambda^{\top}_{\perp}$. Universality is first established at types of the form $o^n \to o^m$, after which definable retractions of arbitrary types into such types are used to obtain universality at all types [10]. This illustrates the basic strategy: reduce a problem at an arbitrary higher-order type to the same problem at a simpler family of types.

For the arrow-only languages considered here, however, this strategy cannot in general be implemented using ordinary retractions into a single simple type. Natural compression targets may require finite products. We therefore turn to a product-free generalisation in which the encoded information is distributed among finitely many component types.

5.2 Finite-component retractions

The preceding strategy encounters a difficulty in the arrow-only setting: an arbitrary type need not be a retract of any single first-order type. The following example illustrates the obstruction.

Example 5.2.1. Let $F_n = \underbrace{[o, \ldots, o]}_{n}$ be a first-order type. In Laird's bistable model, $[\![F_n]\!]$ consists of $\bot, \top, \pi_1, \ldots, \pi_n$, so every extensional chain in $[\![F_n]\!]$ contains at most three elements.

Now let $H = (o \to o) \to o$. The three elements of $[\![o \to o]\!]$ are $\bot \sqsubseteq \mathrm{id} \sqsubseteq \top$. We represent a map $[\![o \to o]\!] \to [\![o]\!]$ by a bitstring abc, where $a, b, c \in \{0, 1\}$ are its respective values on $\bot, \mathrm{id}, \top$, with $0 = \bot$ and $1 = \top$. Then the four maps

$$000 \sqsubseteq 001 \sqsubseteq 011 \sqsubseteq 111$$

form an extensional chain in $[\![H]\!]$. Since the coder of a retraction must be an order embedding, H cannot be a definable retract of any F_n.

This motivates allowing the encoded information to be distributed among finitely many target types.

Definition 5.2.2 (Definable finite-component retraction)**.** Let $\mathcal{M}$ be a Cartesian closed model and let $T, A_1, \ldots, A_m$ be types, with $m \geq 1$. A *definable finite-component retraction* $T \trianglelefteq (A_1, \ldots, A_m)$ in $\mathcal{M}$ consists of closed terms $\mathsf{C}_i : T \to A_i$ $(1 \leq i \leq m)$ and $\mathsf{D} : A_1 \to \cdots \to A_m \to T$ such that

$$[\![\lambda x^T.\mathsf{D}(\mathsf{C}_1 x) \cdots (\mathsf{C}_m x)]\!] = \mathrm{id}_{[\![T]\!]}.$$

We call the C_i the *component encoders* and D the *decoder*.

Thus a finite-component retraction distributes the information contained in a value of type T among finitely many types $A_1, \ldots, A_m$, from which it can be reconstructed definably. Ordinary definable retractions are precisely the case $m = 1$.

5.2.1 Products as a metatheoretical device

Finite products provide a convenient way to package the components of a finite-component retraction. Let $\mathcal{L}$ be an arrow-only simply typed λ-calculus whose primitive constants have product-free types, and let $\mathcal{L}^\times$ be its extension by finite products, pairing, and projections, with no additional primitive constants.

We use the following product-elimination property. Its proof is given in Appendix C.1.

Lemma 5.2.3 (Product elimination)**.** *Suppose that every type in* Γ*, the type* A*, and the types of all primitive constants are product-free. If* $\Gamma \vdash_{\to,\times} M : A$*, then there exists an arrow-only term* M° *such that* $\Gamma \vdash_{\to} M^\circ : A$ *and* $M =_{\beta\pi} M^\circ$*.*

As a consequence, adjoining finite products is conservative for contextual equivalence at the original product-free types.

Lemma 5.2.4 (Conservativity of finite products)**.** *If $M, N : A$ are closed arrow-only terms and A is product-free, then $M \simeq^{\mathcal{L}}_{\mathrm{ctx}} N$ iff $M \simeq^{\mathcal{L}^\times}_{\mathrm{ctx}} N$.*

Proof. The left-to-right direction follows by applying product elimination to an arbitrary ground context of $\mathcal{L}^\times$; the converse is immediate since every context of $\mathcal{L}$ is also a context of $\mathcal{L}^\times$. □

Finite-component retractions are exactly ordinary retractions into finite products in this conservative extension.

Lemma 5.2.5 (Products and finite-component retractions)**.** *Let $\mathcal{M}$ be a Cartesian closed model of $\mathcal{L}$, extended canonically to $\mathcal{L}^\times$, and let $T, A_1, \ldots, A_m$ be arrow-only types. Then*

$$T \trianglelefteq (A_1, \ldots, A_m) \quad \Longleftrightarrow \quad T \trianglelefteq A_1 \times \cdots \times A_m$$

where the retraction on the right is witnessed in $\mathcal{L}^\times$. Moreover, in the right-to-left direction the component encoders and decoder may be chosen to be arrow-only.

Proof. Given component encoders $\mathsf{C}_i : T \to A_i$ and decoder $\mathsf{D} : A_1 \to \cdots \to A_m \to T$, package the encoders as $\mathsf{C}x = \langle \mathsf{C}_1 x, \ldots, \mathsf{C}_m x \rangle$ and define $\mathsf{D}^\times z = \mathsf{D}(\pi_1 z) \cdots (\pi_m z)$.

Conversely, from a retraction $\mathsf{C} : T \to A_1 \times \cdots \times A_m$ and $\mathsf{D} : A_1 \times \cdots \times A_m \to T$, take $\mathsf{C}_i x = \pi_i(\mathsf{C}x)$ and curry the decoder. These terms have arrow-only types, so product elimination yields equivalent arrow-only witnesses. □

Hence finite products may be used purely metatheoretically: reasoning about a retraction into $A_1 \times \cdots \times A_m$ is equivalent to reasoning about a finite-component retraction into $(A_1, \ldots, A_m)$ in the original arrow-only language.

5.3 General retraction-based framework

The definitions and general transfer results in this section constitute the abstract retraction framework introduced in this dissertation. Subsequent sections distinguish new applications of this framework from compression results taken from the literature.

Throughout this section, let $\mathcal{L}$ be an arrow-only simply typed λ-calculus over a ground type o, and let $\mathcal{M}$ be a Cartesian closed model of $\mathcal{L}$. We use the finite-product extension $\mathcal{L}^\times$ introduced above purely as a metatheoretical device. Unless stated otherwise, all retractions are understood relative to $\mathcal{M}$.

5.3.1 Retraction bases and compression

Finite-component retractions reduce a type to finitely many simpler component types. We now isolate when a fixed family of component types suffices uniformly for all types.

Definition 5.3.1. A family $\mathcal{R}$ of arrow-only types is a *retraction basis in* $\mathcal{M}$ if, for every arrow-only type T, there exist $R_1, \ldots, R_m \in \mathcal{R}$ such that $T \trianglelefteq (R_1, \ldots, R_m)$.

Write $\mathcal{R}^\times$ for the class of finite non-empty products $R_1 \times \cdots \times R_m$ with $R_i \in \mathcal{R}$.

The interest is in finding a small family $\mathcal{R}$ into which all types can be compressed. The structural part of this problem is simplified by the following standard closure property.

Lemma 5.3.2 (Preservation by arrows and products)**.** *If $S_1 \trianglelefteq S_2$ and $T_1 \trianglelefteq T_2$, then $S_1 \to T_1 \trianglelefteq S_2 \to T_2$ and $S_1 \times T_1 \trianglelefteq S_2 \times T_2$ in the product extension.*

Proof. For function spaces, if $\mathsf{C}_S, \mathsf{D}_S$ and $\mathsf{C}_T, \mathsf{D}_T$ witness the two retractions, take $\mathsf{C}(f) = \lambda x.\mathsf{C}_T(f(\mathsf{D}_S x))$ and $\mathsf{D}(g) = \lambda x.\mathsf{D}_T(g(\mathsf{C}_S x))$. For products, take the componentwise encoders and decoders. □

The following proposition is the main structural criterion for constructing retraction bases.

Proposition 5.3.3 (Retraction basis criterion)**.** *Let $\mathcal{R}$ be a family of arrow-only types. Suppose that:*

1. *$o \trianglelefteq P$ for some $P \in \mathcal{R}^\times$;*
2. *for every $P, Q \in \mathcal{R}^\times$ there exists $S \in \mathcal{R}^\times$ such that $P \to Q \trianglelefteq S$.*

Then $\mathcal{R}$ is a retraction basis in $\mathcal{M}$.

Proof. By structural induction on T, we construct $P_T \in \mathcal{R}^\times$ such that $T \trianglelefteq P_T$. The ground case is assumption (1). If $T = A \to B$, the induction hypothesis gives $A \trianglelefteq P_A$ and $B \trianglelefteq P_B$ for some $P_A, P_B \in \mathcal{R}^\times$. Preservation by function spaces gives $A \to B \trianglelefteq P_A \to P_B$, and assumption (2) gives $P_A \to P_B \trianglelefteq P_T$ for some $P_T \in \mathcal{R}^\times$. Composing the retractions and unpacking P_T into its factors yields the required finite-component retraction. □

The criterion separates the construction into a uniform structural argument and a language-specific one. Once a family $\mathcal{R}$ has been chosen, the only substantive language-dependent step is to construct the *compression retractions* $P \to Q \trianglelefteq S$ for $P, Q, S \in \mathcal{R}^\times$. Higher-order types then require no separate analysis.

5.3.2 Transfer principles

We first isolate the soundness property required in the transfer arguments.

Definition 5.3.4. A model $\mathcal{M}$ is *observationally sound at a type* T if, for all closed terms $M, N : T$, $[\![M]\!] = [\![N]\!] \implies M \simeq_{\mathrm{ctx}} N$. We say that $\mathcal{M}$ is *observationally sound* if it is observationally sound at every type.

The use of definable embeddings and retractions to transfer properties from simpler types to more complicated ones is well established in Laird's work. For example, in [11] he observes that full abstraction and decidability of observational equivalence may be transferred along definable monomorphisms, and uses such reductions to lift first-order results to arbitrary types. Related retraction arguments are used for universality in [10, 9].

The following proposition packages the same basic methodology for the finite-component retractions introduced above. Thus the transfer idea is inherited from these earlier arguments. The generalisation here consists in allowing finitely many target types and, subsequently, organising such reductions through retraction bases.

Proposition 5.3.5 (Transfer along finite-component retractions)**.** *Suppose* $T \trianglelefteq (A_1, \ldots, A_m)$ *is witnessed by* $\mathsf{C}_i : T \to A_i$ *and* $\mathsf{D} : A_1 \to \cdots \to A_m \to T$.

1. *If* $\mathcal{M}$ *is observationally sound at* T *and fully abstract at each* A_i*, then it is fully abstract at* T.

2. *If* $\mathcal{M}$ *is universal at each* A_i*, then it is universal at* T.

3. *If* $\mathcal{M}$ *is observationally sound at* T *and contextual equivalence is decidable at each* A_i*, then contextual equivalence is decidable at* T.

Proof. For (1), if $M \simeq_{\mathrm{ctx}} N : T$, congruence and full abstraction at each A_i give $[\![\mathsf{C}_i M]\!] = [\![\mathsf{C}_i N]\!]$ for every i. Applying the decoder and using the retraction equation yields $[\![M]\!] = [\![N]\!]$; the converse follows from observational soundness.

For (2), given $t \in [\![T]\!]$, universality at the A_i provides closed terms $M_i : A_i$ denoting $[\![\mathsf{C}_i]\!](t)$. Then $\mathsf{D}\, M_1 \cdots M_m$ denotes t.

For (3), for closed $M, N : T$ we have $M \simeq_{\mathrm{ctx}} N$ iff $\mathsf{C}_i M \simeq_{\mathrm{ctx}} \mathsf{C}_i N$ for every i. The forward implication is congruence. Conversely, applying the decoder to equivalent components and using observational soundness together with the retraction equation yields $M \simeq_{\mathrm{ctx}} N$. Thus finitely many decidable tests at the component types decide equivalence at T. □

5.3.3 Effective retraction bases

For decidability, the compression itself must be effective.

Definition 5.3.6. A retraction basis $\mathcal{R}$ is *effective* if, given an arrow-only type T, one can effectively construct types $R_1, \ldots, R_m \in \mathcal{R}$ and arrow-only terms $\mathsf{C}_i : T \to R_i$ and $\mathsf{D} : R_1 \to \cdots \to R_m \to T$ witnessing $T \trianglelefteq (R_1, \ldots, R_m)$.

If the retractions required by the retraction basis criterion can be constructed uniformly, then the resulting basis is effective: the structural induction is effective and product elimination is effective by normalisation.

The preceding results give the main consequence of the framework.

Theorem 5.3.7 (Retraction-basis transfer principle)**.** *Let $\mathcal{R}$ be a retraction basis in $\mathcal{M}$.*

1. *If $\mathcal{M}$ is observationally sound and fully abstract at every type in $\mathcal{R}$, then $\mathcal{M}$ is fully abstract at every type.*

2. *If $\mathcal{M}$ is universal at every type in $\mathcal{R}$, then $\mathcal{M}$ is universal at every type.*

3. *If $\mathcal{R}$ is effective, $\mathcal{M}$ is observationally sound, and contextual equivalence is uniformly decidable at the types in $\mathcal{R}$, then contextual equivalence is decidable at every type.*

Proof. For every type T, the basis property gives a finite-component retraction $T \trianglelefteq (R_1, \ldots, R_m)$ with $R_i \in \mathcal{R}$. Apply the corresponding part of the preceding proposition. In the decidability case, effectiveness ensures that the required component types and encoders can be constructed from T. □

Thus an application of the framework has two language-specific tasks: choose a small family $\mathcal{R}$ and construct the required compression retractions between products of basis types; then establish the desired property directly at the basis types. The structural reduction from arbitrary higher-order types is supplied uniformly by the framework.

We will also freely use the canonical Cartesian closed isomorphisms of the finite-product extension, such as currying, associativity and symmetry of products, and $A \to (B \times C) \cong (A \to B) \times (A \to C)$. These isomorphisms are definable: both directions are represented by closed terms constructed uniformly from the types involved. Moreover, these constructions are effective. For proof see Appendix C.2.

Lemma 5.3.8 (Effectivity of canonical CCC isomorphisms)**.** *Suppose an isomorphism between types in $\mathcal{L}^{\times}$ is obtained by a finite composition of instances of currying, product associativity and symmetry, and $A \to (B \times C) \cong (A \to B) \times (A \to C)$, together with their inverses. Then closed terms witnessing both directions of the isomorphism can be constructed effectively from the derivation.*

5.4 Recovering universality of the bistable model

As a first illustration of the framework, we recover Laird's universality result for the bistable model of $\Lambda_{\mathcal{C}_2}$ [10]. Let $\mathcal{R} = \{F_n : n \geq 1\}$ be the family of first-order types.

Proposition 5.4.1. *The family $\mathcal{R}$ is a retraction basis in the bistable model.*

Proof. Let T be an arrow-only type. By Laird's Lemma 2.19, there exist $n \geq 0$ and $m \geq 1$ such that $T \trianglelefteq o^n \to o^m$ in the finite-product calculus. If $n = 0$, the target is itself a retract of $o \to o^m$, by encoding an element as a constant function and decoding by application to $\bot$, so we may assume $n \geq 1$.

Using the canonical definable isomorphisms $o^n \to o^m \cong (o^n \to o)^m \cong F_n^m$, we obtain $T \trianglelefteq F_n^m$. By the correspondence between product and finite-component retractions, this yields an arrow-only retraction $T \trianglelefteq (\underbrace{F_n, \ldots, F_n}_{m})$. Hence $\mathcal{R}$ is a retraction basis. □

Lemma 5.4.2. *The bistable model is universal at every F_n, for $n \geq 1$.*

Proof. Every element of $[\![F_n]\!]$ is one of $c_\bot, c_\top, \pi_1, \ldots, \pi_n$, and these are denoted respectively by the constant-$\bot$ function, the constant-$\top$ function, and the projections. □

Corollary 5.4.3. *The bistable model of $\Lambda_{\mathcal{C}_2}$ is universal at every type.*

Proof. The result follows from the retraction-basis transfer principle, since $\mathcal{R}$ is a retraction basis and the model is universal at every type in $\mathcal{R}$. □

5.5 Finite ground signatures

Let $\mathcal{C} = \{c_1, \ldots, c_k\}$, where $k \geq 1$, be a finite non-empty set of constants of ground type o, and let $\Lambda_{\mathcal{C}}$ be the corresponding arrow-only simply typed λ-calculus. For $n \geq 0$, write $F_n = \underbrace{o \to \cdots \to o}_{n} \to o$, so that $F_0 = o$. When reasoning in the finite-product extension, A^m denotes the m-fold product of A.

We shall show that $\mathcal{R} = \{F_n : n \geq 1\}$ is an effective retraction basis in the minimal model $\mathcal{M}_{\mathcal{C}}$ of $\Lambda_{\mathcal{C}}$. By the retraction-basis criterion, the substantive language-specific step is the construction of suitable compression retractions between first-order types.

5.5.1 First-order compression

The following lemma provides the basic compression from which the retraction basis will be constructed. The retraction equation is first established syntactically up to contextual equivalence and hence holds in every model realising the maximal theory.

Lemma 5.5.1 (First-order compression)**.** *For every $n \geq 1$ there is a definable retraction $F_n \to o \trianglelefteq (o \to o) \times o^n$ in the canonical product extension of every model of $\Lambda_{\mathcal{C}}$ realising its maximal theory.*

Proof. For $y : o$ and $1 \leq i \leq n$, write $K_y^n = \lambda x_1^o \ldots x_n^o.y$ and $p_i^n = \lambda x_1^o \ldots x_n^o.x_i$. Define

$$\mathsf{C}_n = \lambda f^{F_n \to o}.\, \big\langle \lambda y^o.f\, K_y^n, \langle f\, p_1^n, \ldots, f\, p_n^n \rangle \big\rangle$$

and

$$\mathsf{D}_n = \lambda z^{(o\to o)\times o^n}.\lambda g^{F_n}.\pi_1 z\big(g\, \pi_1(\pi_2 z) \cdots \pi_n(\pi_2 z)\big).$$

It suffices to show that $\mathsf{D}_n(\mathsf{C}_n f) \simeq_{\mathrm{ctx}} f$ for every closed $f : F_n \to o$.

By strong normalisation, take f in β-normal form. Then $f = \lambda h^{F_n}.M$, where $M : o$ is β-normal in the context $h : F_n$. Since all primitive constants have ground type, the head of M is either a constant c_j or the variable h.

If $M = c_j$, then $f =_\beta \lambda h.c_j$ and $\lambda y.f\, K_y^n =_\beta \lambda y.c_j$. Hence $\mathsf{D}_n(\mathsf{C}_n f) =_\beta f$.

Suppose instead that the head of M is h. Then $f =_\beta \lambda h.h\, M_1 \cdots M_n$ for suitable terms $M_1, \ldots, M_n : o$. For a variable $y : o$, $f\, K_y^n =_\beta y$, and therefore

$$\mathsf{D}_n(\mathsf{C}_n f) =_\beta \lambda g^{F_n}.g\, (f\, p_1^n) \cdots (f\, p_n^n).$$

Every closed β-normal term $g : F_n$ is either a constant function $K_{c_j}^n$ or a projection p_i^n. If $g =_\beta K_{c_j}^n$, then $\mathsf{D}_n(\mathsf{C}_n f)\, g =_\beta c_j =_\beta f\, g$. If $g =_\beta p_i^n$, then $\mathsf{D}_n(\mathsf{C}_n f)\, g =_\beta f\, p_i^n =_\beta f\, g$. Thus $\mathsf{D}_n(\mathsf{C}_n f)\, g =_\beta f\, g$ for every closed $g : F_n$.

By the applicative characterisation of contextual equivalence, $\mathsf{D}_n(\mathsf{C}_n f) \simeq_{\mathrm{ctx}} f$ for every closed $f : F_n \to o$. Applying the same characterisation once more gives $\lambda f.\mathsf{D}_n(\mathsf{C}_n f) \simeq_{\mathrm{ctx}} \lambda f.f$.

Since contextual equivalence is the maximal theory of $\Lambda_{\mathcal{C}}$, every model $\mathcal{M}$ realising that theory satisfies $[\![\mathsf{C}_n]\!]; [\![\mathsf{D}_n]\!] = \mathrm{id}_{[\![F_n \to o]\!]}$. Thus C_n and D_n witness the required retraction. □

The minimal model is used here only to validate the retraction equation modulo the maximal theory. The resulting decision procedure does not require an effective presentation of the model.

5.5.2 An effective first-order retraction basis

Proposition 5.5.2 (First-order retraction basis)**.** *Let $\mathcal{M}_{\mathcal{C}}$ be the minimal model of $\Lambda_{\mathcal{C}}$. Then $\mathcal{R} = \{F_n : n \geq 1\}$ is an effective retraction basis in $\mathcal{M}_{\mathcal{C}}$. Moreover, for every arrow-only type T there exist $n, m \geq 1$ such that $T \trianglelefteq (\underbrace{F_n, \ldots, F_n}_{m})$ can be constructed effectively.*

Proof. We verify the retraction-basis criterion.

Ground type. Fix $c \in \mathcal{C}$. The retraction $o \trianglelefteq F_1$ is witnessed by $\mathsf{C} = \lambda x^o y^o.x$ and $\mathsf{D} = \lambda f^{F_1}.f\,c$, since $\mathsf{D}(\mathsf{C}x) =_\beta x$.

We shall also use the padding retractions $F_p \trianglelefteq F_q$ for $1 \le p \le q$. They are witnessed by

$$\mathsf{C}_{p,q} = \lambda f^{F_p} x_1^o \dots x_q^o.f\,x_1 \dots x_p$$

and

$$\mathsf{D}_{p,q} = \lambda g^{F_q} x_1^o \dots x_p^o.g\,x_1 \dots x_p \underbrace{c \dots c}_{q-p}.$$

Indeed, $\mathsf{D}_{p,q}(\mathsf{C}_{p,q} f) =_{\beta\eta} f$.

Compression. By the preceding lemma, for every $a \ge 1$, $F_a \to o \trianglelefteq (o \to o) \times o^a$. Hence, using canonical definable Cartesian closed isomorphisms and preservation of retractions by function spaces,

$$F_a \to F_b \cong o^b \to (F_a \to o) \trianglelefteq o^b \to \big((o \to o) \times o^a\big) \cong F_{b+1} \times F_b^a.$$

Thus $F_a \to F_b$ retracts into an object of $\mathcal{R}^\times$.

We extend this to arbitrary $P, Q \in \mathcal{R}^\times$. First, by induction on the number of factors of P, for every $b \ge 1$ there exists $S \in \mathcal{R}^\times$ such that $P \to F_b \trianglelefteq S$.

The base case $P = F_a$ is the retraction above. For the induction step, write $P = P' \times F_a$. Then

$$(P' \times F_a) \to F_b \cong P' \to (F_a \to F_b) \trianglelefteq P' \to (F_{b+1} \times F_b^a) \cong (P' \to F_{b+1}) \times (P' \to F_b)^a.$$

By the induction hypothesis, both $P' \to F_{b+1}$ and $P' \to F_b$ retract into objects of $\mathcal{R}^\times$, and preservation by products therefore gives the required compression.

Now let $Q = F_{b_1} \times \dots \times F_{b_s}$. By the canonical product isomorphism,

$$P \to Q \cong (P \to F_{b_1}) \times \dots \times (P \to F_{b_s}).$$

Compressing each factor gives $P \to Q \trianglelefteq S$ for some $S \in \mathcal{R}^\times$. The retraction-basis criterion therefore shows that $\mathcal{R}$ is a retraction basis.

All constructions above are effective. The ground, padding, and first-order compression retractions are given by uniform term schemes, canonical CCC isomorphisms are effective, and product elimination is effective by normalisation. Hence $\mathcal{R}$ is an effective retraction basis.

Finally, let $T \trianglelefteq (F_{n_1}, \dots, F_{n_m})$ be the finite-component retraction produced by the basis construction, and set $n = \max\{n_1, \dots, n_m\}$. Packaging the components gives $T \trianglelefteq F_{n_1} \times \dots \times F_{n_m}$. The padding retractions $F_{n_i} \trianglelefteq F_n$ yield $F_{n_1} \times \dots \times F_{n_m} \trianglelefteq F_n^m$. Composing and unpacking

the resulting product retraction gives effectively $T \trianglelefteq (\underbrace{F_n, \ldots, F_n}_{m})$. □

5.5.3 Decidability at the basis types

Lemma 5.5.3 (Decidability at first-order types)**.** *For every $n \geq 0$, contextual equivalence of closed terms of type F_n is decidable. More precisely, for closed $M, N : F_n$,*

$$M \simeq_{\mathrm{ctx}} N \quad \Longleftrightarrow \quad M\, c_{i_1} \cdots c_{i_n} =_\beta N\, c_{i_1} \cdots c_{i_n}$$

for every $(i_1, \ldots, i_n) \in \{1, \ldots, k\}^n$.

Proof. By the applicative characterisation of contextual equivalence, $M \simeq_{\mathrm{ctx}} N$ iff $M\, P_1 \cdots P_n =_\beta N\, P_1 \cdots P_n$ for every choice of closed terms $P_1, \ldots, P_n : o$.

Every closed ground term is β-equivalent to one of $c_1, \ldots, c_k$, so it suffices to test the k^n tuples of ground constants. Each test is decidable by normalisation. For $n = 0$, the unique empty tuple gives precisely the test $M =_\beta N$ at $F_0 = o$. □

5.5.4 Decidability of the maximal theory

Theorem 5.5.4. *Contextual equivalence in $\Lambda_{\mathcal{C}}$ is decidable at every type. Consequently, the maximal theory of $\Lambda_{\mathcal{C}}$ is decidable.*

Proof. The preceding proposition gives an effective retraction basis $\mathcal{R} = \{F_n : n \geq 1\}$ in the minimal model $\mathcal{M}_{\mathcal{C}}$, while contextual equivalence is uniformly decidable at the basis types by the preceding lemma. Since $\mathcal{M}_{\mathcal{C}}$ is observationally sound, the retraction-basis transfer principle gives decidability of contextual equivalence for closed terms at every type.

For open terms, let $\Gamma = x_1 : A_1, \ldots, x_r : A_r$ and suppose $\Gamma \vdash M, N : B$. Since contextual equivalence is a λ-theory, $\Gamma \vdash M \simeq_{\mathrm{ctx}} N$ iff the closed abstractions $\lambda x_1^{A_1} \ldots x_r^{A_r}.M$ and $\lambda x_1^{A_1} \ldots x_r^{A_r}.N$ are contextually equivalent. Hence contextual equivalence, and therefore the maximal theory, is decidable for arbitrary well-typed terms. □

Remark 5.5.5. The resulting decision procedure is explicit. Given closed $M, N : T$, construct effectively a finite-component retraction $T \trianglelefteq (\underbrace{F_n, \ldots, F_n}_{m})$ with component encoders $\mathsf{C}_1, \ldots, \mathsf{C}_m$. Then $M \simeq_{\mathrm{ctx}} N$ iff $\mathsf{C}_i M \simeq_{\mathrm{ctx}} \mathsf{C}_i N$ for every $1 \leq i \leq m$. Each comparison at F_n reduces to k^n ground tests, so the procedure performs mk^n such tests after the retraction has been constructed.

Corollary 5.5.6. *Equality of denotations of closed terms is decidable at every type in the minimal model $\mathcal{M}_{\mathcal{C}}$.*

Proof. In the minimal model, equality of denotations coincides with contextual equivalence, so the result follows from the preceding theorem. □

5.6 Unary PCF

We now apply the general framework to unary PCF. The compression results used in this section are due to Laird [9].

5.6.1 An effective retraction basis

Proposition 5.6.1. *Let $\mathcal{M}$ be a standard order-extensional model of unary PCF without parallel composition. Then $\mathcal{R} = \{o \to o\}$ is an effective retraction basis in $\mathcal{M}$.*

Proof. By Laird's Lemma 3.6, for every unary PCF type T there exist $n \geq 1$ and a definable retraction $T \trianglelefteq (o \to o)^n$ in the finite-product calculus. Laird's construction is by structural induction on T, so n and the witnessing terms can be constructed effectively from T.

By the correspondence between product and finite-component retractions, this yields an arrow-only finite-component retraction $T \trianglelefteq (\underbrace{o \to o, \ldots, o \to o}_{n})$. The conversion is effective by product elimination. Hence $\{o \to o\}$ is an effective retraction basis in $\mathcal{M}$. □

5.6.2 Decidability

It remains to decide contextual equivalence at the single basis type $o \to o$.

Lemma 5.6.2 (Decidability at $o \to o$)**.** *Contextual equivalence of closed unary PCF terms of type $o \to o$ is decidable.*

Proof. By the Context Lemma established in the preceding chapter, $M \simeq_{\mathrm{ctx}} N : o \to o$ iff $MP \simeq_{\mathrm{ctx}} NP$ for every closed $P : o$. Every closed term of type o is contextually equivalent to either $\bot$ or $\top$. Hence

$$M \simeq_{\mathrm{ctx}} N \quad \Longleftrightarrow \quad M\bot \simeq_{\mathrm{ctx}} N\bot \text{ and } M\top \simeq_{\mathrm{ctx}} N\top.$$

Since contextual equivalence at ground type is decidable, these two tests decide contextual equivalence at $o \to o$. □

We now recover the known decidability result [14] as an application of the retraction-basis transfer principle.

Theorem 5.6.3. *Contextual equivalence is decidable for unary PCF.*

Proof. Let $\mathcal{B}$ be the stable biorder model of unary PCF. It is standard, order-extensional, observationally sound, and contains no parallel composition. Hence $\{o \to o\}$ is an effective retraction basis in $\mathcal{B}$ by the preceding proposition. Contextual equivalence is decidable at the basis type by the preceding lemma, so the retraction-basis transfer principle yields decidability at every unary PCF type. □

5.6.3 Universality

The same retraction basis also recovers Laird's universality result.

Lemma 5.6.4. *The stable biorder model of unary PCF is universal at $o \to o$.*

Proof. The interpretation of $o \to o$ consists of the three monotone stable maps $c_\perp$, id, and $c_\top$, denoted respectively by $\lambda x^o.\perp$, $\lambda x^o.x$, and $\lambda x^o.\top$. Hence every element of $[\![o \to o]\!]$ is definable. □

Corollary 5.6.5. *The stable biorder model is universal for unary PCF.*

Proof. The family $\{o \to o\}$ is a retraction basis in the stable biorder model, and the model is universal at its single basis type. Universality at every type therefore follows from the retraction-basis transfer principle. □

5.7 Unary PCF with parallel composition

Laird's analysis of unary PCF distinguishes sharply between the sequential and parallel cases [9]. In the absence of parallel composition, his universality argument proceeds by definable retractions into finite powers of $o \to o$. The crucial compression step explicitly relies on the exclusion of parallel composition. When parallel composition is present, however, Laird shows that the appropriate order-extensional model is instead the Scott model of finite posets and monotone functions, and proves this model universal.

We revisit the parallel case from the perspective of the retraction framework developed above. The semantic characterisation is due to Laird. The new observation here is that the availability of parallel composition permits a substantially stronger explicit compression. Every first-order monotone functional $o^n \to o$ can be reconstructed from its finite table using a monotone disjunctive normal form. Consequently, $o^n \to o \trianglelefteq o^{2^n}$, and the ground type o itself forms an effective retraction basis.

This gives, in particular, a direct retraction-based proof of decidability of contextual equivalence for unary PCF with parallel composition.

For readability, write $\wedge$ for sequential composition at ground type and $\vee$ for parallel composition. On the two ground points $\perp$ and $\top$, these operations act respectively as meet and join.

5.7.1 First-order compression

Lemma 5.7.1 (First-order compression)**.** *For every $n \geq 1$, there is a definable retraction $o^n \to o \trianglelefteq o^{2^n}$ in every standard order-extensional model of unary PCF with parallel composition.*

Proof. Let $[n] = \{1, \ldots, n\}$. For each $S \subseteq [n]$, define $\chi_S = \langle b_1^S, \ldots, b_n^S \rangle : o^n$, where

$$b_i^S = \begin{cases} \top & i \in S, \\ \bot & i \notin S. \end{cases}$$

Since the model is standard, the points $[\![\chi_S]\!]$ exhaust $[\![o^n]\!]$.

Index the components of o^{2^n} by subsets of $[n]$, and write $\pi_S : o^{2^n} \to o$ for the corresponding projection. Define

$$\mathsf{C}_n = \lambda f^{o^n \to o}.\langle f\, \chi_S \rangle_{S \subseteq [n]}.$$

For $S \subseteq [n]$, put $m_S(x_1, \ldots, x_n) = \bigwedge_{i \in S} x_i$, with the empty meet equal to $\top$, and define

$$\mathsf{D}_n = \lambda z^{o^{2^n}}.\lambda x_1^o \ldots x_n^o. \bigvee_{S \subseteq [n]} \big(\pi_S(z) \wedge m_S(x_1, \ldots, x_n)\big).$$

Finite meets and joins are represented using any fixed enumeration and bracketing.

Let $f \in [\![o^n \to o]\!]$ and $X \subseteq [n]$. Since $m_S([\![\chi_X]\!]) = \top$ precisely when $S \subseteq X$, we obtain

$$[\![\mathsf{D}_n]\!]([\![\mathsf{C}_n]\!](f))([\![\chi_X]\!]) = \bigvee_{S \subseteq X} f([\![\chi_S]\!]).$$

By order-extensionality, f is monotone. Hence $f([\![\chi_S]\!]) \sqsubseteq f([\![\chi_X]\!])$ whenever $S \subseteq X$, while the term with $S = X$ is exactly $f([\![\chi_X]\!])$. Therefore

$$\bigvee_{S \subseteq X} f([\![\chi_S]\!]) = f([\![\chi_X]\!]).$$

Thus $[\![\mathsf{D}_n]\!]([\![\mathsf{C}_n]\!](f))$ and f agree on every point of $[\![o^n]\!]$. Order-extensionality gives $[\![\mathsf{D}_n]\!]([\![\mathsf{C}_n]\!](f)) = f$, and hence $[\![\mathsf{C}_n]\!]; [\![\mathsf{D}_n]\!] = \mathrm{id}_{[\![o^n \to o]\!]}$. □

5.7.2 An effective ground-type retraction basis

Proposition 5.7.2. *Let $\mathcal{M}$ be a standard order-extensional model of unary PCF with parallel composition. Then $\mathcal{R} = \{o\}$ is an effective retraction basis in $\mathcal{M}$.*

Proof. We verify the retraction-basis criterion. The ground-type condition is immediate, since $o \trianglelefteq o$.

Let $P, Q \in \mathcal{R}^\times$. Then $P = o^p$ and $Q = o^q$ for some $p, q \geq 1$. Using the canonical product isomorphism and the preceding compression retraction componentwise,

$$o^p \to o^q \cong (o^p \to o)^q \trianglelefteq (o^{2^p})^q \cong o^{q 2^p}.$$

Since $o^{q^{2^p}} \in \mathcal{R}^\times$, the compression condition is satisfied, so $\{o\}$ is a retraction basis.

The construction is effective. Given p and q, the subsets of $[p]$ can be enumerated effectively, so the coder and decoder of the preceding lemma are obtained uniformly. Canonical CCC isomorphisms and product elimination are effective by the general results above. Hence $\{o\}$ is an effective retraction basis. □

Remark 5.7.3. This exhibits a stronger compression phenomenon than in ordinary unary PCF. Without parallel composition, Laird's construction compresses arbitrary types into finite powers of $o \to o$. With parallel composition, every monotone first-order function can be reconstructed from its finite table using monotone disjunctive normal form, so the effective retraction basis collapses from $\{o \to o\}$ to the ground type $\{o\}$.

5.7.3 Decidability

Corollary 5.7.4. *Contextual equivalence for unary PCF with parallel composition is decidable at every type.*

Proof. Let $\mathcal{S}$ be the Scott model of unary PCF with parallel composition. It is a standard order-extensional model, so the preceding proposition gives the effective retraction basis $\{o\}$. The Scott model is fully abstract, and contextual equivalence at o is decidable since $[\![o]\!] = \{\bot, \top\}$. The retraction-basis transfer principle therefore yields decidability of contextual equivalence at every type. □

5.8 Boolean PCF and obstruction to effective compression

We now turn to a negative application of the retraction-based framework. In the preceding examples, effective compression into a fragment with decidable contextual equivalence yielded decidability at all types. Conversely, undecidability can rule out the existence of such a compression. The underlying finiteness and undecidability results for Boolean PCF are due to Loader [15]. Our aim here is to reinterpret them in terms of effective retraction bases. In particular, Loader's undecidability theorem rules out effective compression into uniformly decidable types, while his pointwise finiteness theorem allows us to contrast finite observational separation with definable reconstruction.

5.8.1 Undecidability obstruction

Since observational equivalence coincides with contextual equivalence in the setting considered here, taking the contrapositive of the decidability clause of the retraction-basis transfer principle gives an immediate obstruction.

Corollary 5.8.1 (Undecidability obstruction)**.** *Let $\mathcal{M}$ be an observationally sound model of a language $\mathcal{L}$, and suppose that contextual equivalence in $\mathcal{L}$ is undecidable. If $\mathcal{R}$ is an effective retraction basis in $\mathcal{M}$, then contextual equivalence cannot be uniformly decidable at the types in $\mathcal{R}$.*

Proof. If contextual equivalence were uniformly decidable at the types in $\mathcal{R}$, the retraction-basis transfer principle would imply decidability at every type of $\mathcal{L}$, contradicting the assumption. □

We apply this observation to PCF_2. We use the convention $\mathrm{ord}(\mathbb{B}) = 0$. Under this convention, the results reviewed in the previous chapter give decidability of contextual equivalence at all types of order at most 3, while Loader's theorem yields undecidability at order 4 [15].

Corollary 5.8.2 (No effective low-order basis for PCF_2)**.** *Let $\mathcal{R}_{\leq 3} = \{A \mid \mathrm{ord}(A) \leq 3\}$. Then $\mathcal{R}_{\leq 3}$ is not an effective retraction basis for any observationally sound model of* PCF_2.

Proof. Contextual equivalence is uniformly decidable at the types in $\mathcal{R}_{\leq 3}$, whereas contextual equivalence in PCF_2 is undecidable in general. The undecidability obstruction therefore rules out $\mathcal{R}_{\leq 3}$ as an effective retraction basis. □

Remark 5.8.3. The conclusion concerns *uniform effective compression*. It does not show that a particular higher-order type cannot be a definable retract of a particular lower-order type, nor does it rule out a non-effective retraction basis. Rather, it shows that there is no effective procedure which, given an arbitrary type T, constructs a finite-component retraction of T into types whose contextual equivalence is uniformly decidable.

Thus the onset of undecidability marks a failure of effective retraction-based compression into the preceding decidable fragment. This contrasts with unary PCF, where Laird's retractions provide effective compression into finite powers of $o \to o$, and with unary PCF with parallel composition, where compression reaches the ground type itself.

5.8.2 Finite separation without reconstruction

The failure of effective compression does not preclude finite observational descriptions at each fixed type. Loader's finiteness result implies that finitely many ground observations suffice to distinguish contextual-equivalence classes.

Corollary 5.8.4 (Finite observational separation)**.** *For every type T of* PCF_2*, there exist finitely many closed terms $\mathsf{C}_1, \ldots, \mathsf{C}_m : T \to \mathbb{B}$ such that, for all closed $M, N : T$,*

$$M \simeq_{\mathrm{ctx}} N \quad \Longleftrightarrow \quad \mathsf{C}_i M \simeq_{\mathrm{ctx}} \mathsf{C}_i N \text{ for every } 1 \leq i \leq m.$$

Proof. Loader shows that, at every type T, there are only finitely many contextual-equivalence classes of closed terms. For each pair of distinct classes, choose a closed ground context which distinguishes them, and abstract its hole to obtain a closed term $\mathsf{C}_i : T \to \mathbb{B}$. Since there are only finitely many pairs of classes, this yields a finite family of such terms.

If $M \simeq_{\mathrm{ctx}} N$, then $\mathsf{C}_i M \simeq_{\mathrm{ctx}} \mathsf{C}_i N$ for every i by congruence. Conversely, if M and N belong to distinct contextual-equivalence classes, one of the chosen C_i distinguishes them. Hence the family jointly separates contextual-equivalence classes. □

These finite separating families cannot be constructed uniformly and effectively from T. If they could, contextual equivalence at T could be decided by applying the finitely many C_i and comparing the resulting closed ground terms. Since every closed term of type $\mathbb{B}$ is contextually equivalent to one of tt, ff, or $\bot$, each such comparison is decidable. A uniform effective construction at all types would therefore contradict Loader's undecidability theorem.

Remark 5.8.5 (Separation versus reconstruction). Loader's finiteness theorem guarantees finite separation, but not finite-component retraction. The missing ingredient is definable reconstruction. Finite observational separation is weaker than the finite-component retractions used in this chapter. The maps $\mathsf{C}_i : T \to \mathbb{B}$ above jointly separate contextual-equivalence classes, but no definable decoder $\mathsf{D} : \mathbb{B} \to \cdots \to \mathbb{B} \to T$ has been constructed satisfying

$$\mathsf{D}(\mathsf{C}_1 x) \cdots (\mathsf{C}_m x) \simeq_{\mathrm{ctx}} x.$$

A finite-component retraction therefore provides more than a finite collection of observations: it provides *definable reconstruction* from those observations.

Consequently, Loader's finiteness result does not show that $\{\mathbb{B}\}$, or the family of low-order types, forms a non-effective retraction basis. Establishing such a basis would require an additional definable reconstruction result.

The distinction is essential. Boolean PCF admits finitely many separating observations at each individual type, but undecidability prevents these observations from being assembled uniformly and effectively into the kind of compression provided by an effective retraction basis.

Chapter 6

Conclusions

Across the different settings considered in this dissertation, decidability of contextual equivalence has been closely connected with the possibility of replacing arbitrary contextual observations by a finite and effectively accessible representation. Padovani's representatives realise this syntactically, fully abstract finite models realise it semantically, and retractions provide a type-theoretic mechanism for transferring such representations between types. The contrast between unary and Boolean PCF shows, however, that finiteness alone is insufficient: the crucial additional issue is whether the finite observational structure can be accessed uniformly and effectively.

The first part of the dissertation examined this phenomenon for simply typed λ-calculi with finite ground signatures. Padovani's syntactic method replaces the quantification over arbitrary contexts by a finite family of representative behaviours at each type, yielding a direct decision procedure for contextual equivalence. We then considered the corresponding semantic question. The ordinary set-theoretic model was seen to be too discriminating, since its full function spaces contain non-denotable elements capable of separating contextually equivalent terms. Laird's bistable model avoids this problem for the binary ground signature by restricting the admissible higher-order behaviours.

One of the main contributions of this dissertation was to extend this semantic approach to arbitrary finite ground signatures. Rather than attempting to encode all ground distinctions in a single bistable ground object, we considered all surjective binary recolourings of the ground signature simultaneously. Their componentwise product yields a fully abstract interpretation for every finite ground signature. Since each binary component is finite and effectively presentable, so is the resulting product interpretation at every type. This gives a semantic proof of decidability of contextual equivalence for arbitrary finite ground signatures, providing a denotational counterpart to the earlier syntactic decision procedures.

The later chapters broadened the setting beyond languages whose primitive constants all have ground type. Unary PCF and Boolean PCF illustrate two sharply different possibilities. In unary

PCF, stability and boundedness enforce sequential behaviour and lead to finite universal models with strong structural relationships between types. Boolean PCF retains only finitely many contextual-equivalence classes at each type, but Loader's undecidability theorem shows that these finite quotients cannot be accessed uniformly in a way that decides contextual equivalence. This distinction between pointwise finiteness and uniform effectiveness provides an important boundary for the methods developed here.

The second main contribution of this dissertation is a general retraction-based framework for analysing higher-order contextual equivalence. We introduced finite-component retractions and retraction bases, and proved transfer principles reducing full abstraction, universality, and decidability to simpler basis types. The framework recovers existing retraction arguments for unary PCF, yields a stronger ground-type compression in the presence of parallel composition, and turns Loader's undecidability result for Boolean PCF into an obstruction to effective compression. It therefore isolates a common structural principle behind both positive and negative results: decidability depends not only on finite observable behaviour, but on its uniform and effective reconstruction from simpler types.

6.1 Future work

The product construction developed for finite ground signatures raises the question of how far the same idea can be pushed. Possible directions include multiple ground types, infinite ground signatures, and languages with higher-type primitive operations. More generally, it would be interesting to understand when a family of simpler fully abstract models can be combined to recover the observational theory of a richer language.

The general retraction-based framework developed here should be regarded as a starting point rather than a complete theory. Several natural questions remain open.

- **Structure and minimality of retraction bases.** What is the order-theoretic structure of the collection of retraction bases for a given language? Under what conditions does a minimal, or even a least, retraction basis exist? When such bases exist, is there a general effective procedure for constructing them?

- **Extension to richer type systems.** Which features of simple types make finite-component retractions and retraction bases possible, and which parts of the transfer theory survive when the type language is enriched? Natural first cases include product and sum types, followed by more substantial extensions such as recursive or polymorphic types. Understanding which aspects of the present theory depend essentially on the arrow-only structure would help clarify the scope of the method.

- **Effectivity of retractions.** When does the existence of a family of retractions give a uniform effective reduction of contextual equivalence, and when can retractions exist only non-uniformly? The contrast between unary and Boolean PCF suggests that the distinction between structural reducibility and effective reducibility deserves a systematic treatment.

Taken together, these questions suggest a broader programme: to understand which structural properties of a typed language permit its higher-order observational behaviour to be compressed into simpler, finite, and effectively accessible forms. The results of this dissertation provide several instances of such compression, as well as examples showing where finiteness by itself is not enough.

Bibliography

[1] S. Abramsky and M. Lenisa. Fully complete minimal PER models for the simply typed λ-calculus. In *Computer Science Logic (CSL 2001)*, Lecture Notes in Computer Science, vol. 2142, pp. 443–457. Springer, 2001.

[2] H. Barendregt, W. Dekkers, and R. Statman. *Lambda Calculus with Types*. Perspectives in Logic. Cambridge University Press, 2013. 10.1017/CBO9781139032636.

[3] M. Barr and C. Wells. *Category Theory for Computing Science*. 3rd ed. Les Publications CRM, Montreal, 1999.

[4] N. Dershowitz and Z. Manna. Proving termination with multiset orderings. *Communications of the ACM*, 22(8):465–476, 1979.

[5] J.-Y. Girard, Y. Lafont, and P. Taylor. *Proofs and Types*. Cambridge Tracts in Theoretical Computer Science, vol. 7. Cambridge University Press, 1989.

[6] T. Jim and A. R. Meyer. Full abstraction and the context lemma (preliminary report). In *Theoretical Aspects of Computer Software (TACS '91)*, Lecture Notes in Computer Science, vol. 526, pp. 131–151. Springer, 1991.

[7] J. Lambek and P. J. Scott. *Introduction to Higher-Order Categorical Logic*. Cambridge Studies in Advanced Mathematics, vol. 7. Cambridge University Press, Cambridge, 1986.

[8] J. Laird. A fully abstract bidomain model of unary FPC. In *Typed Lambda Calculi and Applications (TLCA 2003)*, Lecture Notes in Computer Science, vol. 2701, pp. 211–225. Springer, 2003.

[9] J. Laird. Sequentiality in bounded biorders. *Fundamenta Informaticae*, 65(1–2):173–191, 2005.

[10] J. Laird. Bistable biorders: A sequential domain theory. *Logical Methods in Computer Science*, 3(2:5):1–22, 2007. 10.2168/LMCS-3(2:5)2007.

[11] J. Laird. Decidability and syntactic control of interference. *Theoretical Computer Science*, 394:64–83, 2008. 10.1016/j.tcs.2007.10.045.

[12] M. Lennon-Bertrand. *Lecture Notes on Denotational Semantics*. Department of Computer Science and Technology, University of Cambridge, 2024.

[13] R. Loader. An algorithm for the minimal model. Unpublished manuscript, 1997.

[14] R. Loader. Unary PCF is decidable. *Theoretical Computer Science*, 206(1–2):317–329, 1998. 10.1016/S0304-3975(98)00048-6.

[15] R. Loader. Finitary PCF is not decidable. *Theoretical Computer Science*, 266(1–2):341–364, 2001. 10.1016/S0304-3975(00)00194-8.

[16] V. Padovani. Decidability of all minimal models. Unpublished note, revised version, 2012. arXiv:1210.3599 [cs.LO].

Appendix A

Auxiliary results on observational equivalence

This appendix collects several auxiliary results concerning the notions of observational and contextual equivalence used in Chapter 1. The underlying facts are standard, or treated as standard in the literature, but we give the arguments in full in order to make the development self-contained. We first compare higher-order and ground observations, then establish the compatibility property of observational equivalence needed for the Context Lemma, and finally use it to prove the coincidence of observational and ground contextual equivalence.

A.1 Higher-order and ground observations

Lemma A.1.1. *For closed terms $M_1, M_2 : \theta$ of the pure simply typed λ-calculus, $M_1 \equiv_{\mathbb{B}} M_2$ iff $M_1 \equiv_{a,b} M_2$.*

Proof. Suppose first that $M_1 \equiv_{\mathbb{B}} M_2$, and let $\vdash_{\{a,b\}} D_{a,b}[-] : \theta \Rightarrow o$ be a closed ground context. Replace, at the meta-level, every occurrence of a and b in $D_{a,b}[-]$ by fresh variables $x, y : o$, respectively, obtaining $D_{x,y}[-]$, and define $C[-] := \lambda x^o.\lambda y^o.D_{x,y}[-]$. Then $\vdash C[-] : \theta \Rightarrow \mathbb{B}$.

Since the original calculus contains no ground constants, the normal form of $D_{x,y}[M_i]$ can only be x or y. By freshness of a and b, we therefore have $D_{a,b}[M_i] \Downarrow a$ iff $D_{x,y}[M_i] \Downarrow x$, equivalently iff $C[M_i] \Downarrow \mathsf{tt}$. Hence $M_1 \equiv_{\mathbb{B}} M_2$ implies $D_{a,b}[M_1] \Downarrow a$ iff $D_{a,b}[M_2] \Downarrow a$. Since $D_{a,b}[-]$ was arbitrary, $M_1 \equiv_{a,b} M_2$.

Conversely, suppose that $M_1 \equiv_{a,b} M_2$, and let $\vdash C[-] : \theta \Rightarrow \mathbb{B}$ be a closed Boolean context. Define $D_{a,b}[-] := C[-]\, a\, b$. Then $\vdash_{\{a,b\}} D_{a,b}[-] : \theta \Rightarrow o$. Since every closed term of type $\mathbb{B}$ normalises either to tt or ff, we have $C[M_i] \Downarrow \mathsf{tt}$ iff $C[M_i]\, a\, b \Downarrow a$. By $M_1 \equiv_{a,b} M_2$, this holds for M_1 iff it holds for M_2, and therefore $M_1 \equiv_{\mathbb{B}} M_2$. □

A.2 Compatibility of observational equivalence

In Chapter 1 we use the following compatibility property of observational equivalence.

Proposition A.2.1 (Context Lemma)**.** *Let* $\vdash t : A \to B$ *and* $\vdash u, u' : A$ *be closed terms. If*

$$u \equiv u',$$

then

$$t\, u \equiv t\, u'.$$

Padovani treats this result as standard [16]. For completeness, we give a proof. The main difficulty is that observational equivalence is defined applicatively: from $u \equiv u'$ it is immediate that equivalent functions behave identically when applied to the same argument, but it is not immediate that an arbitrary function behaves identically when applied to equivalent arguments.

We proceed in three steps. First, we introduce an auxiliary logical relation whose definition builds in the required compatibility with related arguments. Second, we establish its basic properties and show that it coincides with observational equivalence. The Context Lemma then follows immediately.

We therefore introduce an auxiliary logical relation. For every type A, define a relation $\cong_A$ on closed terms of type A recursively by

$$u \cong_o u' \quad \iff \quad u =_\beta u',$$

and

$$f \cong_{C \to D} g \quad \iff \quad \text{for all closed } u, u' : C,\ u \cong_C u' \Longrightarrow f\, u \cong_D g\, u'.$$

Unlike observational equivalence, the definition at function type allows the two functions to be applied to two different, but related, arguments. This is precisely the compatibility property required by the Context Lemma.

We first establish some elementary properties of $\cong_A$.

Lemma A.2.2 (β-invariance)**.** *Let* $M, M', N, N' : A$ *be closed terms. If* $M =_\beta M'$ *and* $N =_\beta N'$*, then*

$$M \cong_A N \quad \iff \quad M' \cong_A N'.$$

Proof. By induction on A.

If $A = o$, the statement follows immediately from the definition of $\cong_o$ and transitivity of $=_\beta$.

Now let $A = C \to D$. Suppose first that $M \cong_A N$, and take arbitrary closed $u, u' : C$ with $u \cong_C u'$. By definition, $M\, u \cong_D N\, u'$. Since $M'\, u =_\beta M\, u$ and $N'\, u' =_\beta N\, u'$, the induction

hypothesis at type D gives $M'\,u \cong_D N'\,u'$. Hence $M' \cong_A N'$. The converse is identical. □

Reflexivity of $\cong_A$ is not immediate from its definition at function types. We obtain it from the following fundamental property.

Lemma A.2.3 (Fundamental lemma)**.** *Suppose* $\Gamma = x_1 : A_1, \ldots, x_n : A_n \vdash M : B$*, and let* $\sigma, \tau \in \mathsf{ClSub}(\Gamma)$ *satisfy* $\sigma(x_i) \cong_{A_i} \tau(x_i)$ *for every* i*. Then*

$$M\sigma \cong_B M\tau.$$

Proof. By induction on the typing derivation of M.

- **Variable.** If $M = x_i$, then $M\sigma = \sigma(x_i) \cong_{A_i} \tau(x_i) = M\tau$ by assumption.
- **Constant.** If $M = a : o$, then $M\sigma = a = M\tau$, and hence $M\sigma \cong_o M\tau$.
- **Application.** Suppose $M = PQ$, with $\Gamma \vdash P : A \to B$ and $\Gamma \vdash Q : A$. By the induction hypothesis, $P\sigma \cong_{A\to B} P\tau$ and $Q\sigma \cong_A Q\tau$. The definition of $\cong_{A\to B}$ therefore gives $(P\sigma)(Q\sigma) \cong_B (P\tau)(Q\tau)$, which is precisely $M\sigma \cong_B M\tau$.
- **Abstraction.** Suppose $M = \lambda x^A.N$, where $\Gamma, x : A \vdash N : B$. We must show $(\lambda x.N)\sigma \cong_{A\to B} (\lambda x.N)\tau$.

  Let $U, V : A$ be arbitrary closed terms with $U \cong_A V$. The extended substitutions $\sigma[U/x]$ and $\tau[V/x]$ are related at every variable of $\Gamma, x : A$, so the induction hypothesis gives $N(\sigma[U/x]) \cong_B N(\tau[V/x])$.

  Moreover, $((\lambda x.N)\sigma)U =_\beta N(\sigma[U/x])$ and $((\lambda x.N)\tau)V =_\beta N(\tau[V/x])$. By Lemma A.2.2, $((\lambda x.N)\sigma)U \cong_B ((\lambda x.N)\tau)V$. Since U, V were arbitrary related arguments, the definition of $\cong_{A\to B}$ gives the result.

□

Taking the context to be empty immediately yields reflexivity.

Corollary A.2.4. *Every closed term* $M : A$ *satisfies* $M \cong_A M$*.*

Proof. Apply Lemma A.2.3 with $\Gamma = \varnothing$ and the empty substitutions. □

We can now show that $\cong_A$ is an equivalence relation.

Lemma A.2.5. *For every type* A*, the relation* $\cong_A$ *is an equivalence relation on closed terms of type* A*.*

Proof. Reflexivity is Corollary A.2.4. We prove symmetry and transitivity by induction on A.

At ground type, $\cong_o$ is $=_\beta$, so both properties are immediate.

Let $A = C \to D$.

For symmetry, suppose $f \cong_A g$, and take closed $u, u' : C$ with $u \cong_C u'$. By symmetry at type C, $u' \cong_C u$. Hence $f\,u' \cong_D g\,u$. By symmetry at D, $g\,u \cong_D f\,u'$. Since u, u' were arbitrary, $g \cong_A f$.

For transitivity, suppose $f \cong_A g$ and $g \cong_A h$, and take closed $u, u' : C$ with $u \cong_C u'$. By reflexivity, $u \cong_C u$, so $f\,u \cong_D g\,u$. From $g \cong_A h$ and $u \cong_C u'$, we also have $g\,u \cong_D h\,u'$. By transitivity at D, $f\,u \cong_D h\,u'$. Hence $f \cong_A h$. □

It remains to relate the auxiliary relation to observational equivalence.

Lemma A.2.6. *For all closed terms $M, N : A$,*

$$M \cong_A N \quad \Longleftrightarrow \quad M \equiv N.$$

Proof. By induction on A.

If $A = o$, both relations are defined by β-equivalence.

Now let $A = C \to D$.

Suppose first that $f \cong_A g$. Let $u : C$ be arbitrary and closed. By reflexivity, $u \cong_C u$, so the definition of $\cong_A$ gives $f\,u \cong_D g\,u$. By the induction hypothesis, $f\,u \equiv g\,u$. Since this holds for every closed $u : C$, the definition of observational equivalence gives $f \equiv g$.

Conversely, suppose $f \equiv g$. To prove $f \cong_A g$, take arbitrary closed $u, u' : C$ with $u \cong_C u'$. By observational equivalence, $f\,u \equiv g\,u$, and hence by the induction hypothesis at D, $f\,u \cong_D g\,u$.

By reflexivity, $g \cong_A g$. Since $u \cong_C u'$, the definition of $\cong_A$ therefore gives $g\,u \cong_D g\,u'$. Transitivity of $\cong_D$ now yields $f\,u \cong_D g\,u'$. Since u, u' were arbitrary, $f \cong_A g$. □

We can finally prove the Context Lemma.

Proof of the Context Lemma. Suppose $u \equiv u' : A$. By Lemma A.2.6, $u \cong_A u'$. By reflexivity, $t \cong_{A\to B} t$. The definition of $\cong_{A\to B}$ therefore gives $t\,u \cong_B t\,u'$. Applying Lemma A.2.6 once more yields $t\,u \equiv t\,u'$. □

A.3 Observational and contextual equivalence

We can now use the Context Lemma to justify the equivalence between the applicative and contextual formulations used in Chapter 1.

Theorem A.3.1. *For closed terms, observational equivalence and ground contextual equivalence coincide.*

Proof. Let $M, N : A$ be closed.

Suppose first that $M \simeq_{\mathrm{ctx}} N$. We prove $M \equiv N$ by induction on the type A.

If $A = o$, take the identity context $C[-] = [-]$. By contextual equivalence, $M =_{\beta\eta} N$. Since closed terms of ground type normalise to ground constants, $M =_{\beta\eta} N$ iff $M =_{\beta} N$. Hence $M \equiv N$.

Now suppose $A = B \to C$, and let $U : B$ be arbitrary and closed. We show that $M\,U \simeq_{\mathrm{ctx}} N\,U$. Given any closed ground context $\vdash D[-] : C \Rightarrow o$, define $E[-] = D[[-]\,U]$. Then $\vdash E[-] : (B \to C) \Rightarrow o$, so $M \simeq_{\mathrm{ctx}} N$ gives $D[M\,U] =_{\beta\eta} D[N\,U]$. Since $D[-]$ was arbitrary, $M\,U \simeq_{\mathrm{ctx}} N\,U$. By the induction hypothesis, $M\,U \equiv N\,U$. Since U was arbitrary, $M \equiv N$.

Conversely, suppose $M \equiv N : A$. It is enough to show that observational equivalence is preserved by arbitrary program contexts. The only non-immediate case is when the hole occurs as the argument of an application; this is precisely where the Context Lemma is needed.

Compatibility claim. Let $\Gamma \vdash C[-] : A \Rightarrow B$ be a typed one-hole context and let $\sigma \in \mathsf{ClSub}(\Gamma)$. If $\vdash M, N : A$ and $M \equiv N$, then $C[M]\sigma \equiv C[N]\sigma : B$.

We prove the claim by structural induction on $C[-]$.

- **Hole.** If $C[-] = [-]$, then $C[M]\sigma = M$ and $C[N]\sigma = N$, since M and N are closed. Hence the result follows from $M \equiv N$.

- **Abstraction.** Suppose $C[-] = \lambda x^{D}.C_0[-]$, where $\Gamma, x : D \vdash C_0[-] : A \Rightarrow E$. We must show $(\lambda x^{D}.C_0[M])\sigma \equiv (\lambda x^{D}.C_0[N])\sigma$. Let $U : D$ be arbitrary and closed. By the definition of observational equivalence, it is enough to compare the two terms after application to U. They are respectively β-equivalent to $C_0[M](\sigma[U/x])$ and $C_0[N](\sigma[U/x])$. Since $\sigma[U/x] \in \mathsf{ClSub}(\Gamma, x : D)$, the induction hypothesis gives $C_0[M](\sigma[U/x]) \equiv C_0[N](\sigma[U/x])$. Hence the required equivalence follows.

- **Application on the right.** Suppose $C[-] = C_0[-]\,P$, where $\Gamma \vdash C_0[-] : A \Rightarrow D \to B$ and $\Gamma \vdash P : D$. By the induction hypothesis, $C_0[M]\sigma \equiv C_0[N]\sigma : D \to B$. Since $P\sigma : D$ is closed, the definition of observational equivalence gives $(C_0[M]\sigma)(P\sigma) \equiv (C_0[N]\sigma)(P\sigma)$.

- **Application on the left.** Suppose $C[-] = P\,C_0[-]$, where $\Gamma \vdash P : D \to B$ and $\Gamma \vdash C_0[-] : A \Rightarrow D$. By the induction hypothesis, $C_0[M]\sigma \equiv C_0[N]\sigma : D$. Since $P\sigma : D \to B$ is closed, the Context Lemma gives $(P\sigma)(C_0[M]\sigma) \equiv (P\sigma)(C_0[N]\sigma)$.

This proves the compatibility claim.

Now let $\vdash C[-] : A \Rightarrow o$ be an arbitrary closed ground context. Applying the claim with $\Gamma = \varnothing$ gives $C[M] \equiv C[N]$. At ground type this means $C[M] =_{\beta} C[N]$, and hence $C[M] =_{\beta\eta} C[N]$. Since $C[-]$ was arbitrary, $M \simeq_{\mathrm{ctx}} N$.

Therefore observational equivalence and ground contextual equivalence coincide on closed terms. □

Appendix B

Obstructions to direct extensions of the bistable model

This appendix contains the proofs of the two obstruction results stated in Chapter 3. Recall that the unary shrinking equation requires every element $h \in D \Rightarrow D$ of the semantic endomorphism space of the ground object to be idempotent.

B.1 Total coherence

Suppose that $D = [\![o]\!]$ is a finite bistable biorder consisting of a single coherence class and that $|D| > 2$. We show that $D \Rightarrow D$ contains a non-idempotent element.

Since D has a single coherence class, $(D, \sqsubseteq)$ is a finite distributive lattice. Let $\bot_D$ and $\top_D$ denote its least and greatest elements, and choose $c \notin \{\bot_D, \top_D\}$. The ideal $\downarrow c$ is proper, so it is contained in a maximal proper ideal M. Since D is a distributive lattice, M is prime, and hence its complement $F = D \setminus M$ is a prime filter.

Define $h : D \to D$ by

$$h(x) = \begin{cases} c, & x \in F, \\ \bot_D, & x \notin F. \end{cases}$$

Since F is a prime filter, h preserves binary meets and joins. Hence it is monotone and, because D consists of a single coherence class, bistable. Thus $h \in D \Rightarrow D$.

Now $c \in M$, whereas $\top_D \notin M$, so

$$h(\top_D) = c \qquad \text{and} \qquad h(c) = \bot_D.$$

Consequently,

$$h^2(\top_D) = \bot_D \neq c = h(\top_D),$$

so h is not idempotent. This contradicts the unary shrinking equation. Therefore the induced interpretation in **BBO** cannot be fully abstract.

B.2 Discrete coherence

Suppose that $D = [\![o]\!]$ is a bistable biorder with discrete coherence and $|D| > 2$. Since every coherence class is a singleton, every monotone function $D \to D$ is automatically bistable. It therefore suffices to construct a monotone non-idempotent endomorphism.

If the extensional order on D is discrete, any transposition of two distinct elements is monotone and non-idempotent.

Otherwise choose $a, b \in D$ with $a \sqsubset b$, and put $U = {\uparrow} a$. Suppose first that $U \neq D$. Define

$$h(x) = \begin{cases} b, & x \in U, \\ a, & x \notin U. \end{cases}$$

The set U is upward closed, so h is monotone. For any $d \notin U$, we have $h(d) = a$ and $h(a) = b$, hence

$$h^2(d) = b \neq a = h(d).$$

Thus h is non-idempotent.

It remains to consider the case $U = D$, so a is a least element. Put $L = {\downarrow} b$. If $L \neq D$, define

$$h(x) = \begin{cases} a, & x \in L, \\ b, & x \notin L. \end{cases}$$

Since L is downward closed, h is monotone. For any $d \notin L$, we have $h(d) = b$ and $h(b) = a$, so

$$h^2(d) = a \neq b = h(d).$$

Finally, suppose $L = D$ as well. Then a is least and b is greatest. Since $|D| > 2$, choose $c \notin \{a, b\}$ and define

$$h(a) = c, \qquad h(x) = b \quad \text{for } x \neq a.$$

This map is monotone: if $x \sqsubseteq y$ and $x = a$, then $h(x) = c \sqsubseteq b = h(y)$ unless $y = a$, while if $x \neq a$ then $h(x) = b$, forcing $h(y) = b$. Moreover,

$$h^2(a) = h(c) = b \neq c = h(a),$$

so h is non-idempotent.

Thus in every possible form of the extensional order, $D \Rightarrow D$ contains a non-idempotent element. The unary shrinking equation therefore fails, and the induced interpretation cannot be fully abstract.

Appendix C

Products and canonical type isomorphisms

This appendix records two technical facts concerning the use of finite products in Chapter 5. First, we show that products can be eliminated from terms whose context, result type, and primitive constants are product-free. Second, we make explicit the effectivity of the canonical type isomorphisms used in the retraction constructions. Together, these results justify the use of products as an auxiliary device without changing the product-free calculi under consideration.

C.1 Elimination of finite products

The following is a standard consequence of normalisation and the subformula property. We include the argument because the result is used explicitly in Chapter 5.

Lemma C.1.1 (Product elimination)**.** *Suppose that every type occurring in the context Γ, the type A, and the types of all primitive constants is product-free. If $\Gamma \vdash_{\to,\times} M : A$, then there exists an arrow-only term M° such that $\Gamma \vdash_{\to} M^\circ : A$ and $M =_{\beta\pi} M^\circ$. Consequently, $M =_{\beta\eta\pi} M^\circ$.*

Proof. We begin with the following technical definition.

Definition C.1.2. The subformula relation $\preceq$ is the least reflexive and transitive relation on types satisfying

$$B \preceq B \to C, \qquad C \preceq B \to C, \qquad B \preceq B \times C, \qquad C \preceq B \times C.$$

By strong normalisation for the simply typed lambda calculus with products, there exists a $\beta\pi$-normal term N such that $M \longrightarrow^*_{\beta\pi} N$. By subject reduction, $\Gamma \vdash_{\to,\times} N : A$.

Under the Curry–Howard correspondence, a typing derivation for the simply typed lambda calculus with arrows and products is a natural-deduction derivation in the $(\Rightarrow, \wedge)$-fragment of

intuitionistic propositional logic: abstraction and application correspond respectively to implication introduction and elimination, while pairing and projection correspond respectively to conjunction introduction and elimination. Moreover, β-redexes and projection redexes correspond to introduction–elimination detours. Since N is $\beta\pi$-normal, the typing derivation induced by N is therefore a normal natural-deduction derivation.

By the subformula property for normal natural deductions in the $(\wedge, \Rightarrow)$-fragment [5, §10.3.1], every type occurring in this derivation is a subformula of the conclusion type A, a type occurring in the context Γ, or the declared type of a primitive constant occurring in N.

By assumption, all these types are product-free. Since every subformula of a product-free type is itself product-free, no product type occurs in the typing derivation of N.

It follows that N contains neither pairing nor projection. A pair $\langle P, Q\rangle$ necessarily has a product type, while a projection $\pi_i P$ requires its argument P to have a product type. Thus N is an arrow-only term. Moreover, its typing derivation uses neither product introduction nor product elimination, and hence it is also a valid derivation in the arrow-only calculus: $\Gamma \vdash_{\to} N : A$. Taking $M^\circ = N$, we obtain $M =_{\beta\pi} M^\circ$, as required. □

C.2 Effective witnesses for canonical type isomorphisms

The retraction arguments in Chapter 5 repeatedly use canonical isomorphisms between types with finite products. For the corresponding decidability arguments, it is useful to record explicitly that witnesses for these isomorphisms can be constructed effectively from their derivations.

Lemma C.2.1 (Effectivity of canonical CCC isomorphisms)**.** *Consider the simply typed λ-calculus with finite products. Suppose an isomorphism between types is obtained by a finite composition of instances of currying, product associativity and symmetry, and*

$$A \to (B \times C) \cong (A \to B) \times (A \to C),$$

together with their inverses. Then, from a finite derivation of the isomorphism, one can effectively construct closed terms witnessing both directions of the isomorphism.

Proof. Types and terms of the simply typed λ-calculus with products are finite syntax trees, so abstraction, application, pairing, projection, and substitution of types into fixed term schemes are computable. It therefore suffices to give uniform witnesses for the generating isomorphisms and observe closure under composition.

For currying, $(A \times B \to C) \cong A \to B \to C$, take $\mathsf{curry}_{A,B,C} = \lambda f^{A\times B\to C}.\lambda x^{A}.\lambda y^{B}.f\langle x, y\rangle$ and $\mathsf{uncurry}_{A,B,C} = \lambda g^{A\to B\to C}.\lambda z^{A\times B}.g(\pi_1 z)(\pi_2 z)$. The function $\beta\eta$-laws and product η-law give $\mathsf{uncurry}_{A,B,C} \circ \mathsf{curry}_{A,B,C} =_{\beta\eta\pi} \mathrm{id}_{A\times B\to C}$ and $\mathsf{curry}_{A,B,C} \circ \mathsf{uncurry}_{A,B,C} =_{\beta\eta\pi} \mathrm{id}_{A\to B\to C}$.

Product associativity, $(A \times B) \times C \cong A \times (B \times C)$, is witnessed by

$$\mathsf{assoc} = \lambda z^{(A\times B)\times C}.\langle \pi_1(\pi_1 z), \langle \pi_2(\pi_1 z), \pi_2 z\rangle\rangle \text{ and } \mathsf{assoc}^{-1} = \lambda z^{A\times(B\times C)}.\langle\langle \pi_1 z, \pi_1(\pi_2 z)\rangle, \pi_2(\pi_2 z)\rangle$$

Their composites are identities by the product $\beta\eta$-laws.

Product symmetry, $A \times B \cong B \times A$, is witnessed in both directions by the appropriate instance of $\mathsf{swap} = \lambda z^{A\times B}.\langle \pi_2 z, \pi_1 z\rangle$.

Finally, $A \to (B \times C) \cong (A \to B) \times (A \to C)$ is witnessed by

$$\mathsf{split} = \lambda f^{A\to B\times C}.\left\langle \lambda x^A.\pi_1(fx), \lambda x^A.\pi_2(fx)\right\rangle$$

and

$$\mathsf{merge} = \lambda p^{(A\to B)\times(A\to C)}.\lambda x^A.\left\langle (\pi_1 p)x, (\pi_2 p)x\right\rangle.$$

Again, $\mathsf{merge} \circ \mathsf{split} =_{\beta\eta\pi} \mathrm{id}_{A\to B\times C}$ and $\mathsf{split} \circ \mathsf{merge} =_{\beta\eta\pi} \mathrm{id}_{(A\to B)\times(A\to C)}$.

All these witnesses are instances of fixed finite syntactic templates, so their syntax trees can be computed from the types involved.

For composition, suppose $A \cong B$ and $B \cong C$ are witnessed by effectively constructed terms $f : A \to B$, $f^{-1} : B \to A$, $g : B \to C$, and $g^{-1} : C \to B$. Then $A \cong C$ is witnessed by $\lambda x^A.g(fx) : A \to C$ and $\lambda z^C.f^{-1}(g^{-1}z) : C \to A$. These terms are computable, and their composites are identities by the equations for the given isomorphisms.

Hence, by induction on the finite derivation of a canonical isomorphism, terms witnessing both directions can be constructed effectively. □